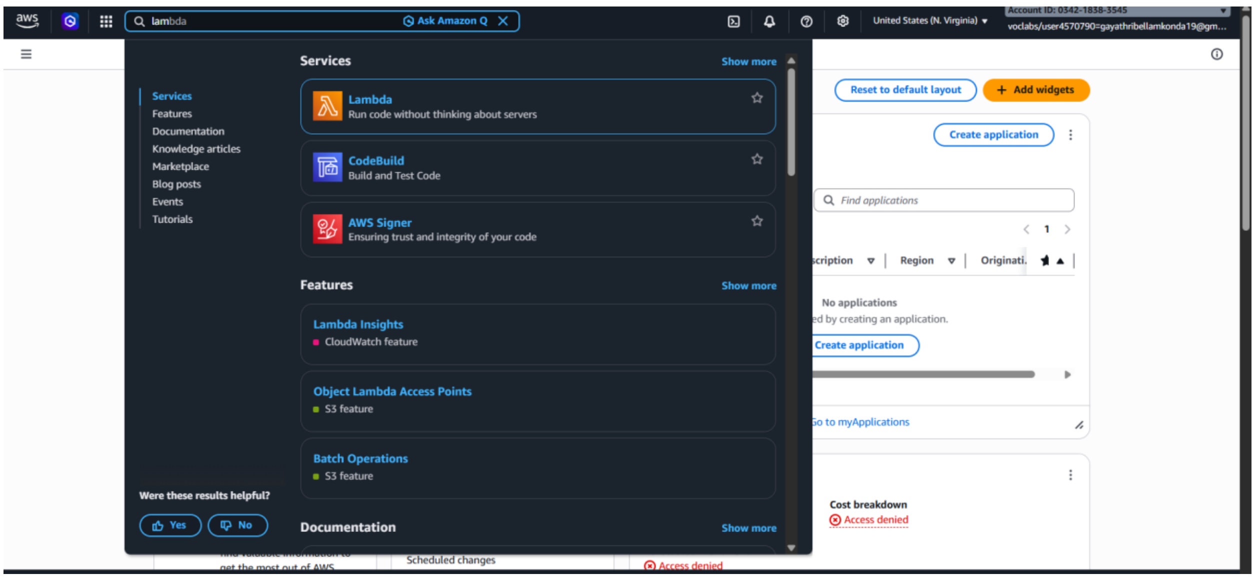
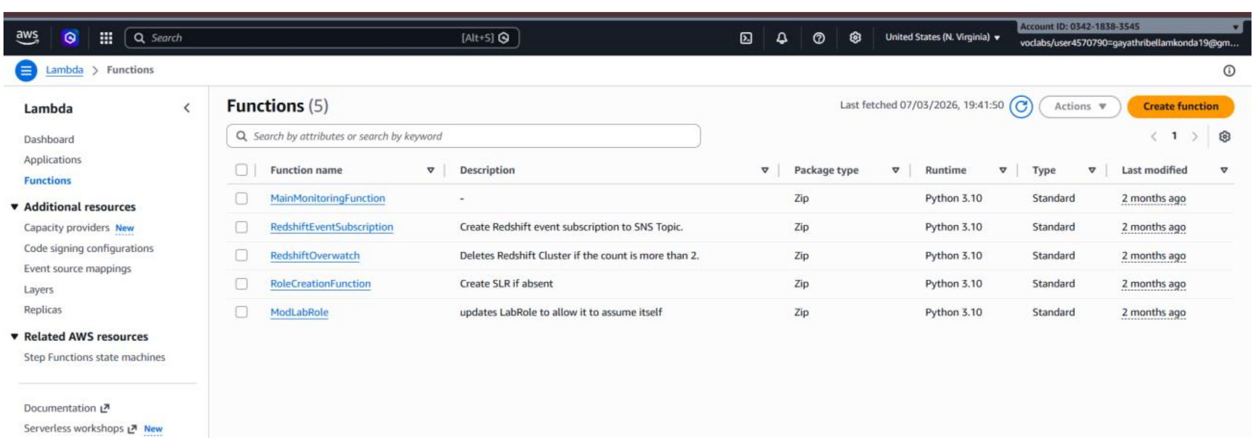
WEEK – 7

AWS Lambda function to trigger whenever an object is uploaded to an S3 bucket and update a DynamoDB table

Open AWS Lambda Console: Navigate to the Lambda service by either searching for "Lambda" in the AWS Management Console



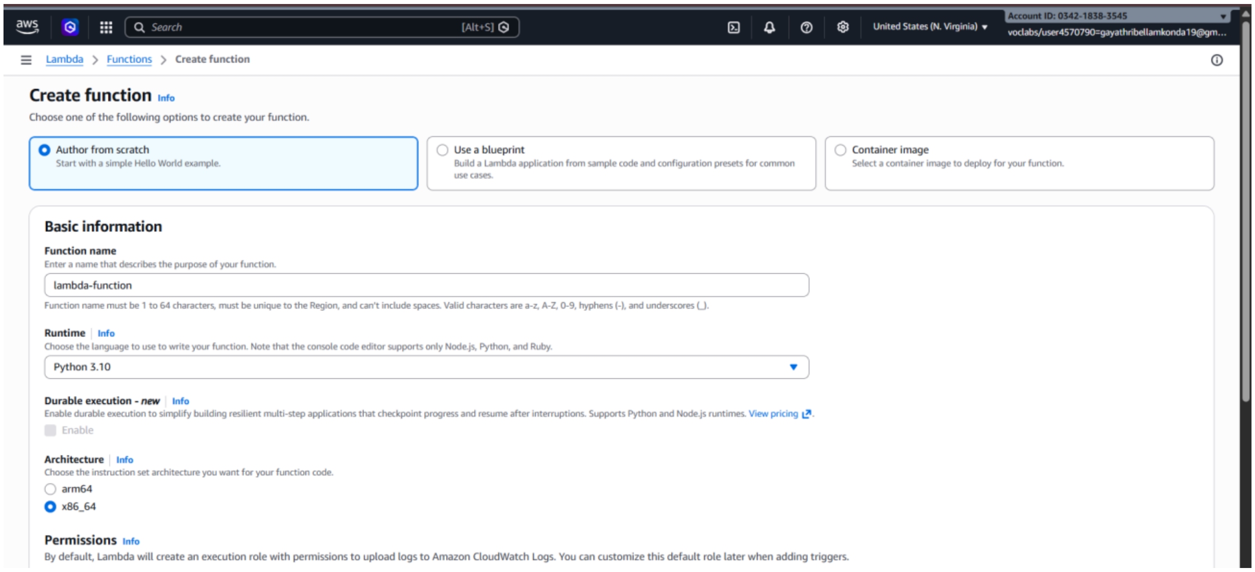
Click on the "Create function" button



- Choose "Author from scratch".

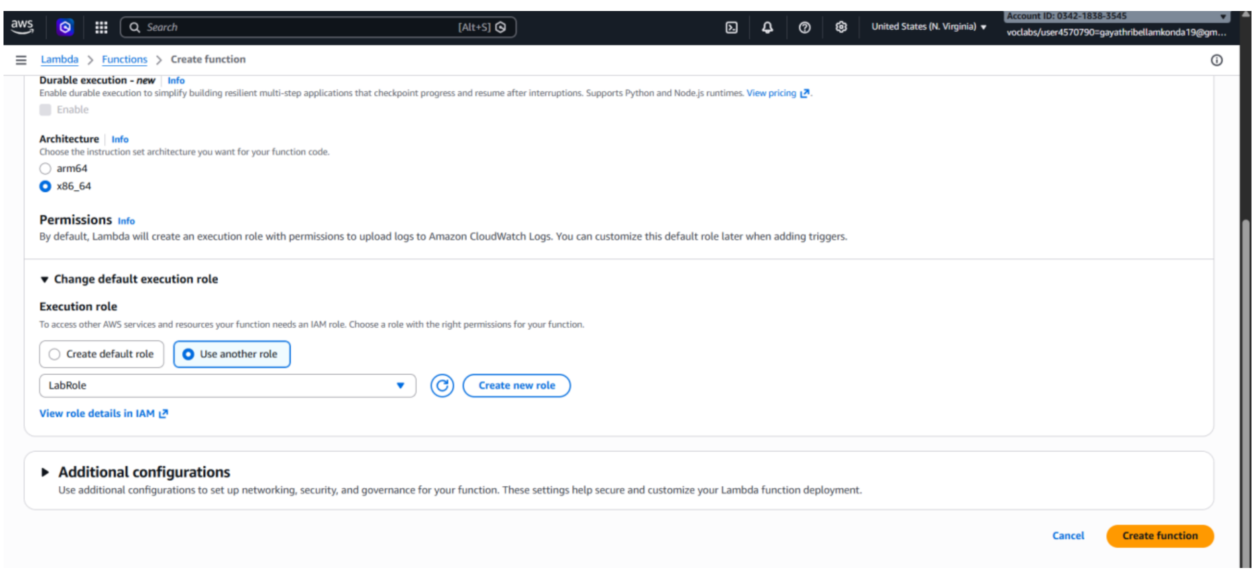
- Enter a name for your function -> lambda-funciton

- Choose Python 3.10 as the runtime



- Under "Permissionscv-> choose use another role that has permissions to access S3 and DynamoDB.

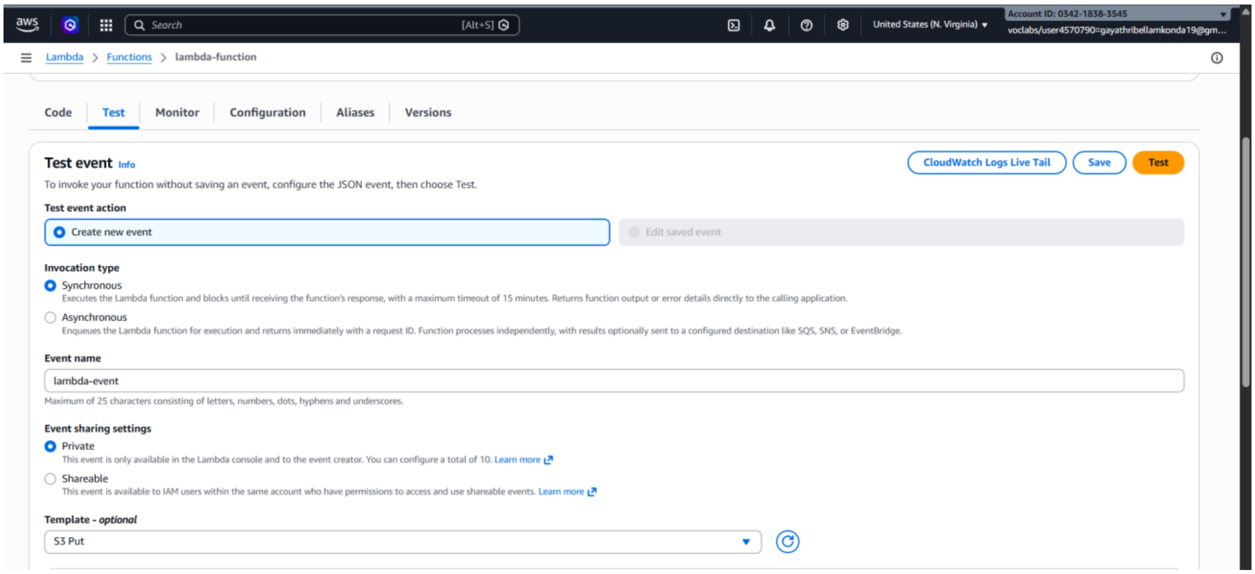
Select LabRole -> Click on create function button

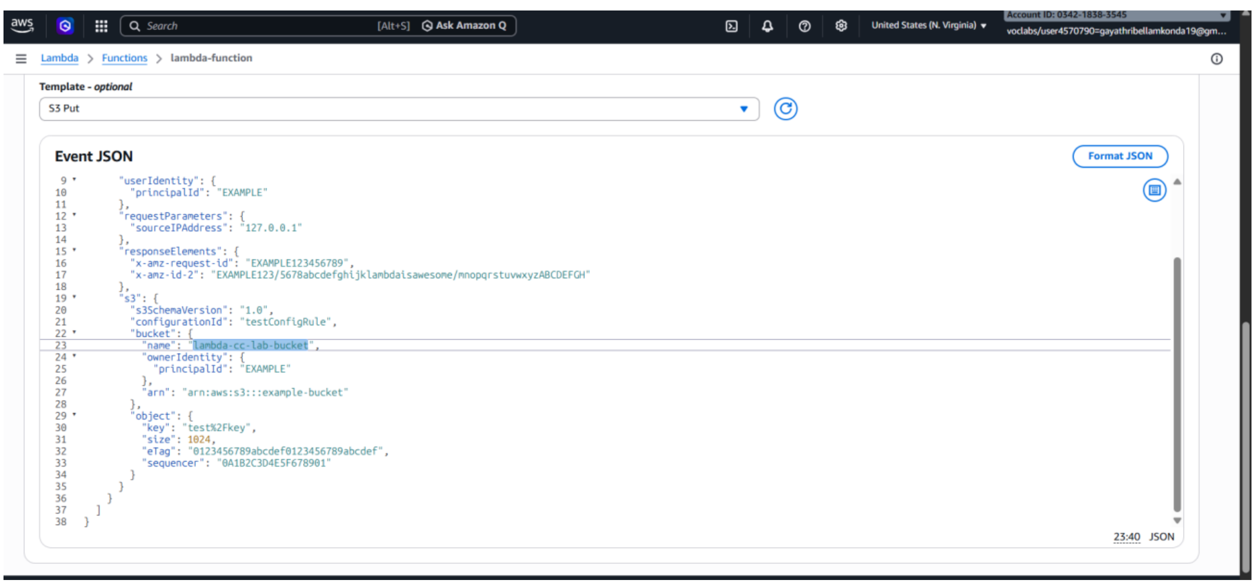


Test Your Function:

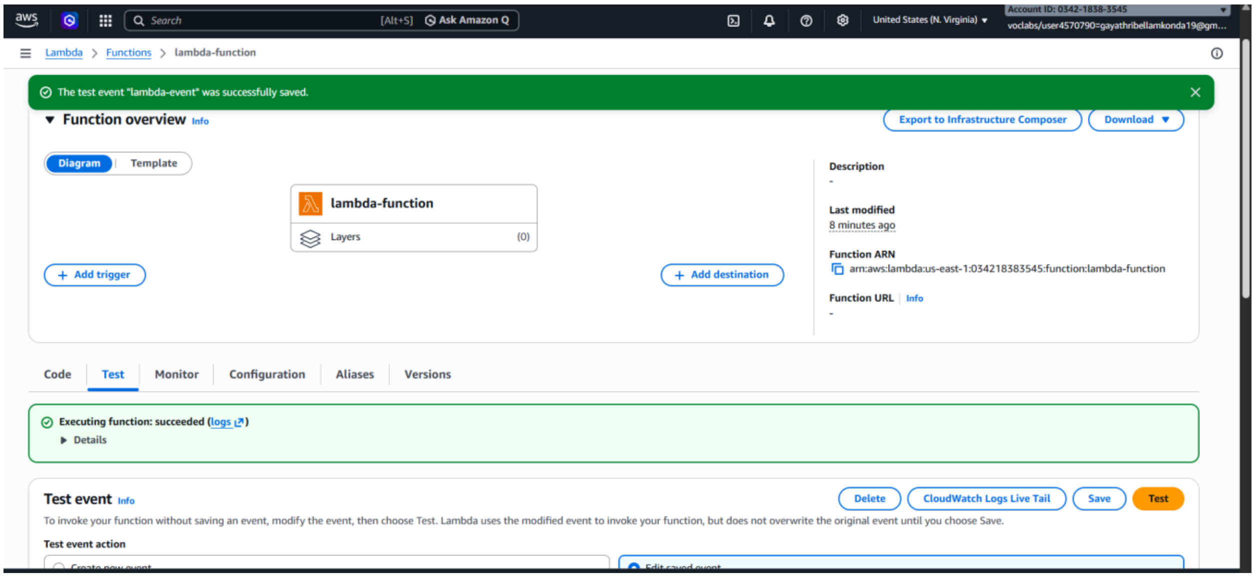
Select create new event -> select template as S3 put

Change the bucket in the code

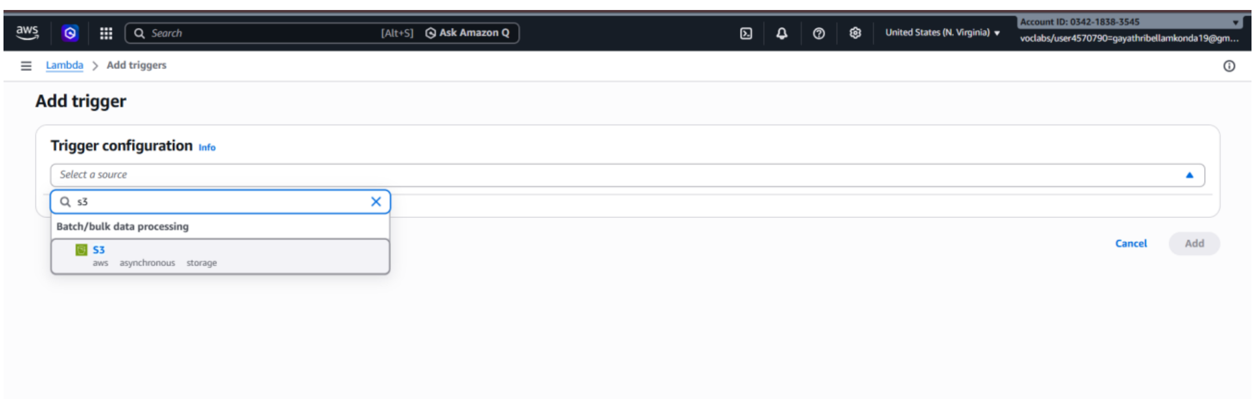




Click on save -> click on test – it will execute the code successfully

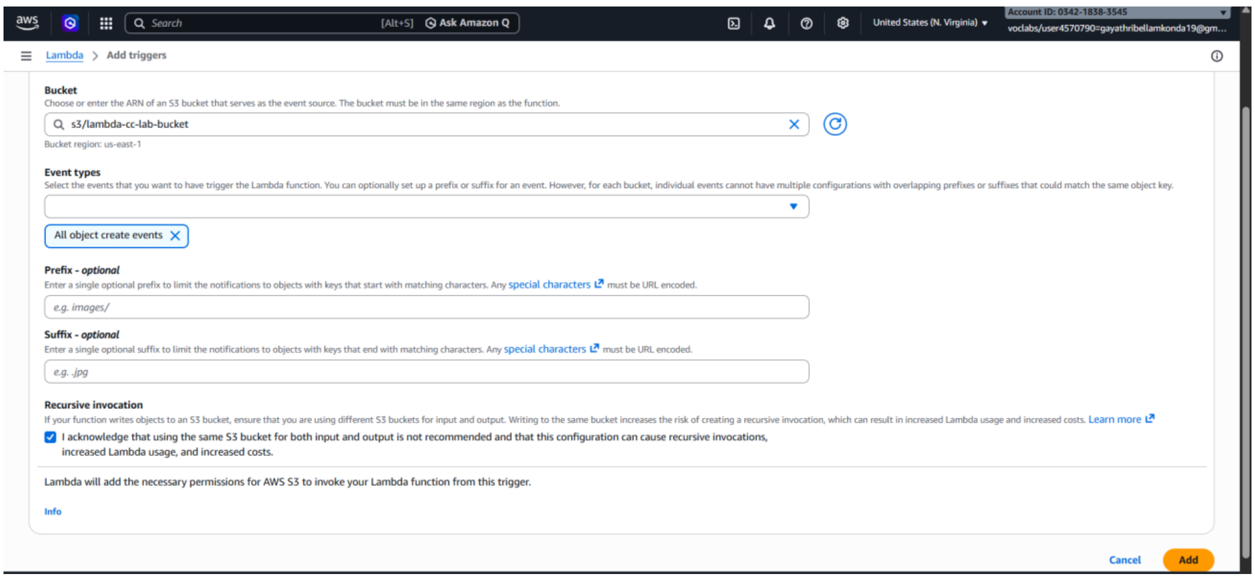


Click on Add trigger -> Select source as S3

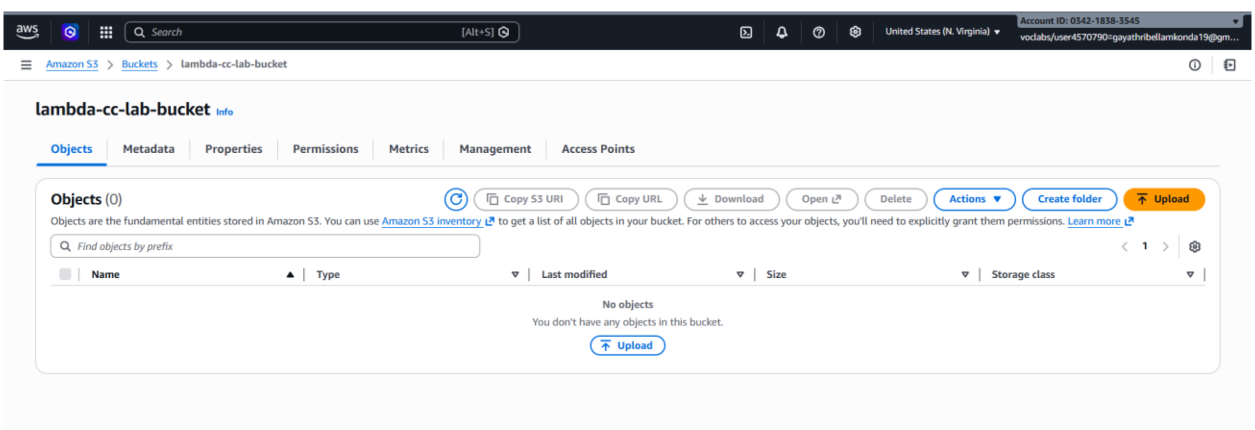


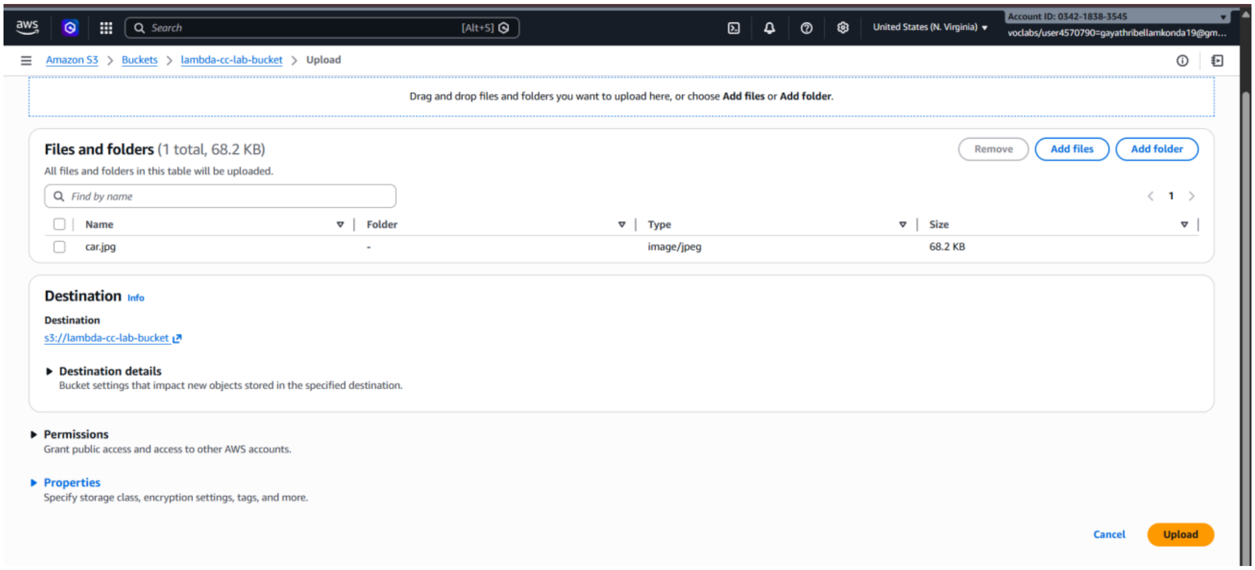
Bucket should be in same region

Select the bucket name -> select I acknowledge -> click on save button

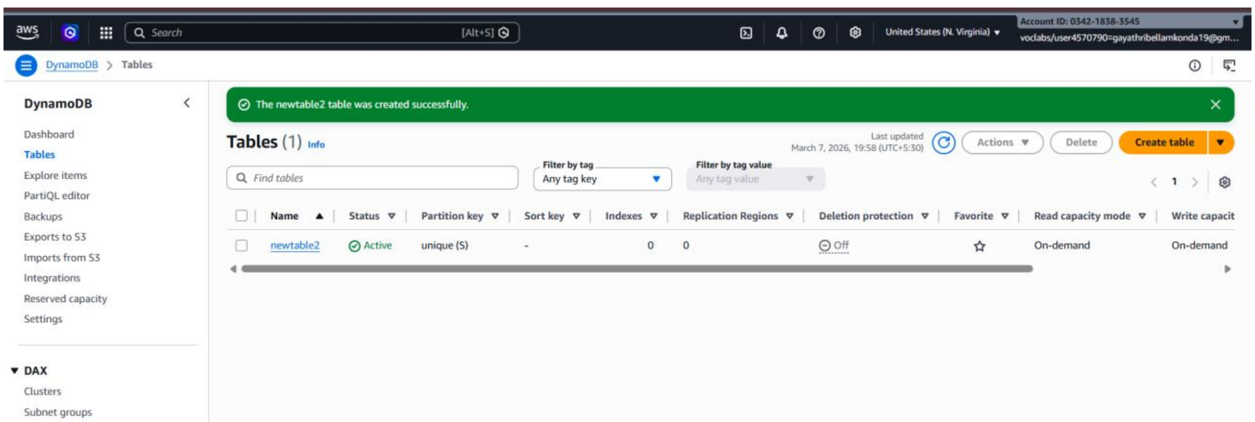


Upload any document in the bucket -> click on upload

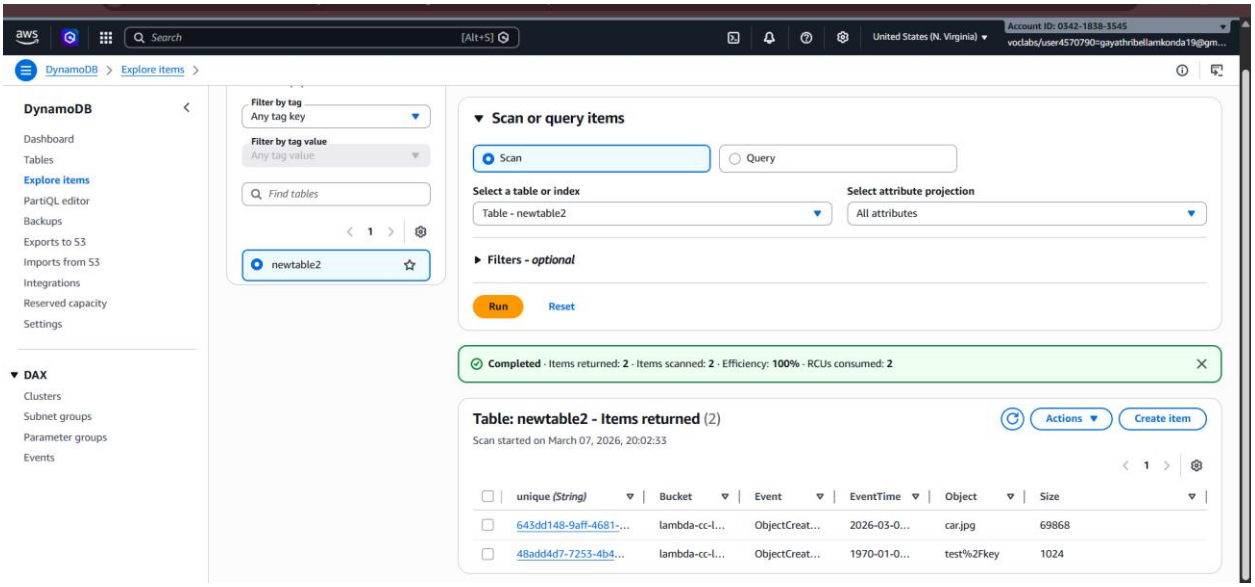




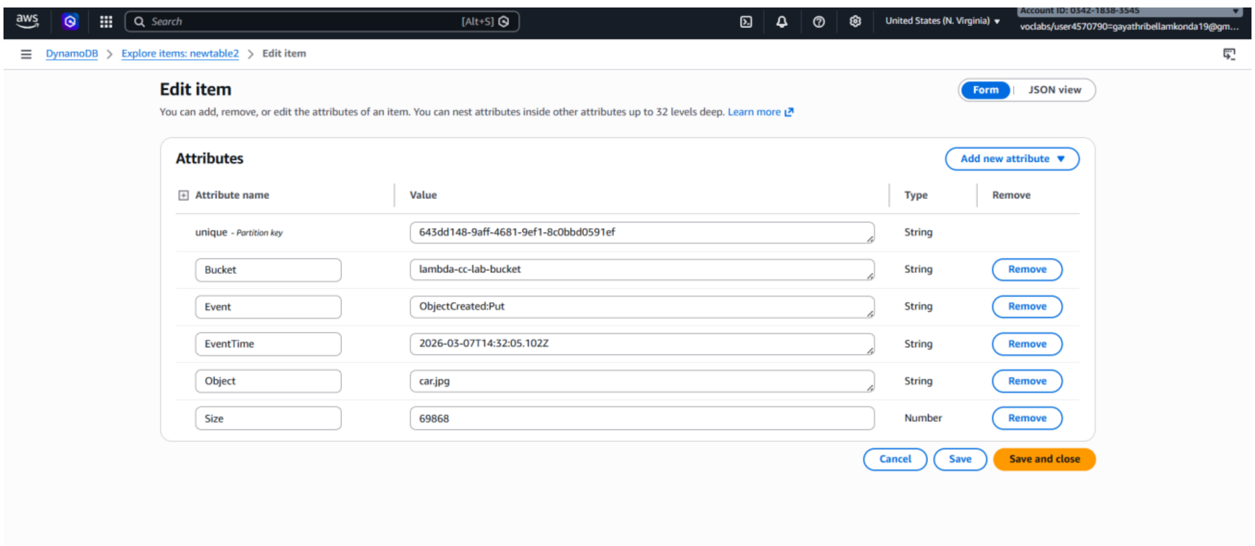
Go to DynamoDb dashboard -> Click on Explore items



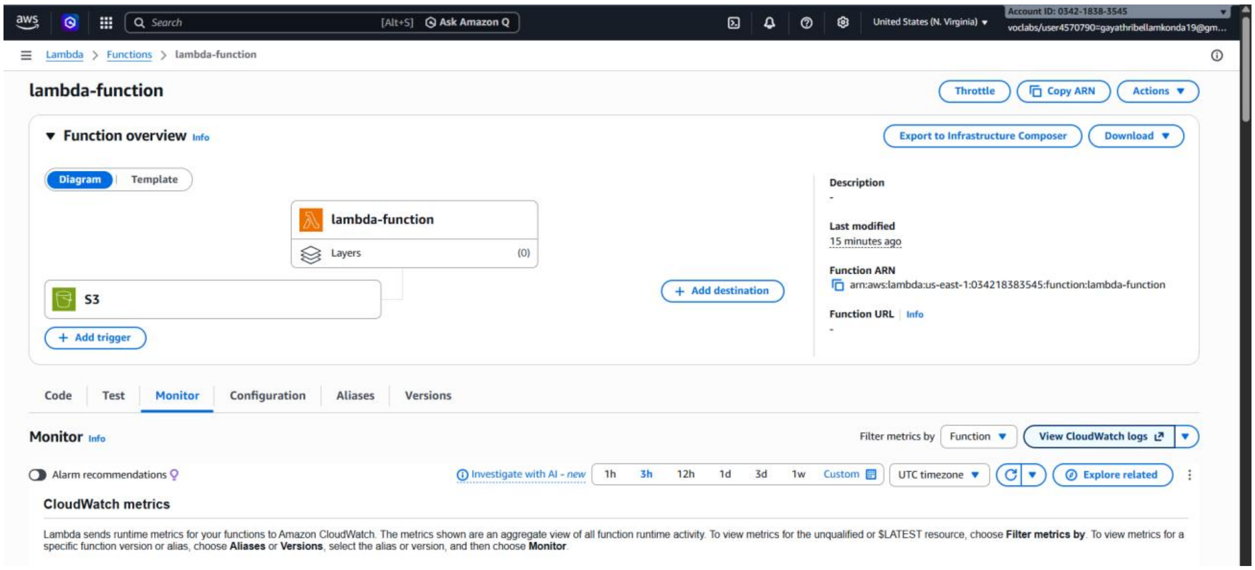
Select the table -> navigate to items returned -> uploaded objects will be visible



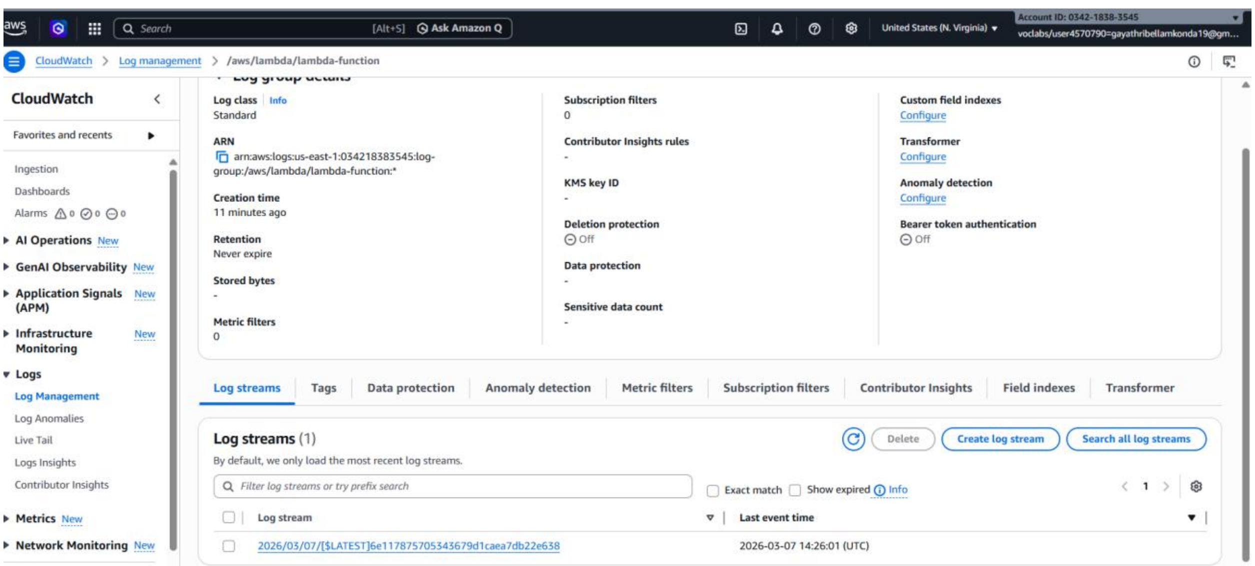
The meta data of the object will be stored in the table

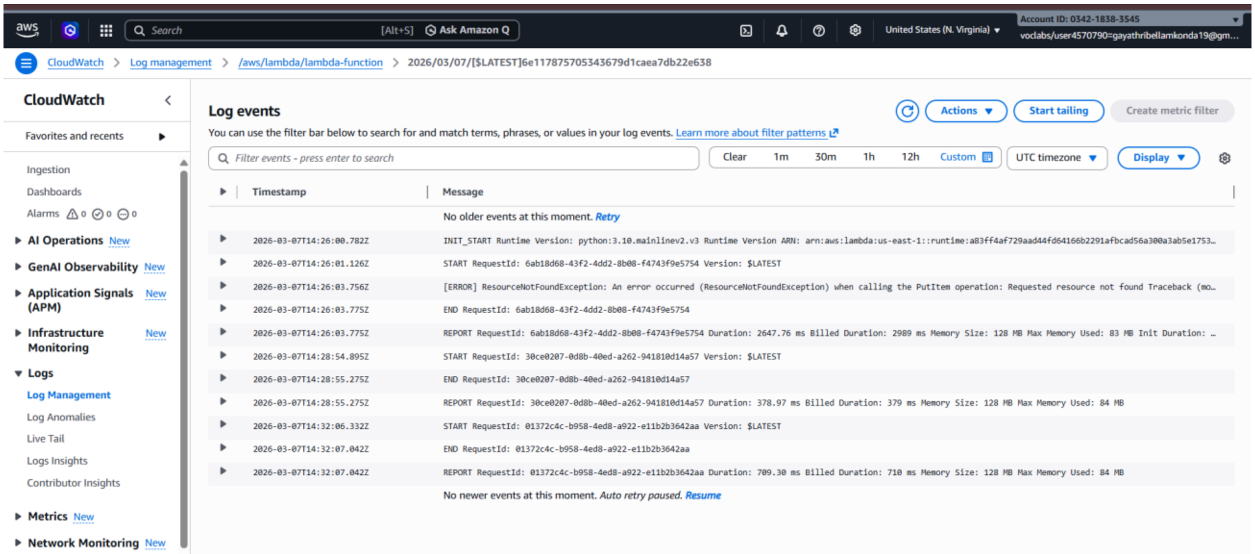


In lambda dashboard -> Go to Monitor Tab -> click on view cloudWatch logs



Select the log stream logs will be appeared



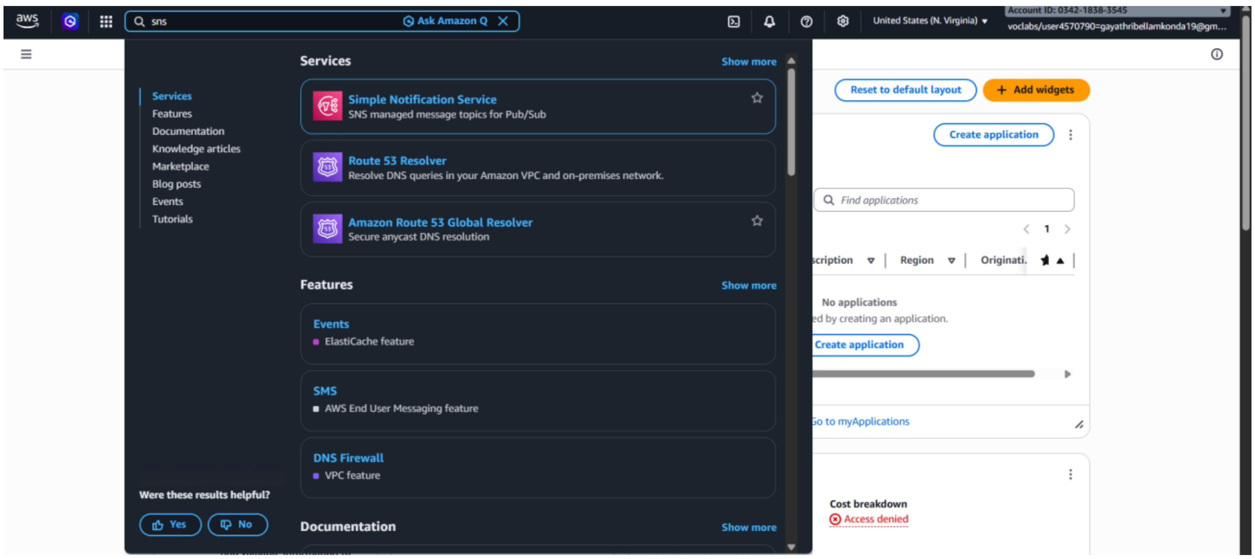


Search
[Alt+S]
Ask Amazon Q
United States (N. Virginia)
Account ID: 0342-1838-3545
voclabs/user4570790=gayathribellamkonda19@gm...
CloudWatch > Log management > /aws/lambda/lambda-function > 2026/03/07/[$LATEST]6e117875705343679d1caea7db22e638
CloudWatch
Favorites and recents
Ingestion
Dashboards
Alarms 0 0 0
AI Operations New
GenAI Observability New
Application Signals (APM) New
Infrastructure Monitoring New
Logs
Log Management
Log Anomalies
Live Tail
Logs Insights
Contributor Insights
Metrics New
Network Monitoring New
Log events
Actions
Start tailing
Create metric filter
You can use the filter bar below to search for and match terms, phrases, or values in your log events. Learn more about filter patterns
Filter events - press enter to search
Clear 1m 30m 1h 12h Custom
UTC timezone
Display
Timestamp
Message
No older events at this moment. Retry
2026-03-07T14:26:00.782Z INIT_START Runtime Version: python:3.10.mainlinev2.v3 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:a83ff4af729aad44fd64166b2291afbcad56a300a3ab5e1753…
2026-03-07T14:26:01.126Z START RequestId: 6ab18d68-43f2-4dd2-8b08-f4743f9e5754 Version: $LATEST
2026-03-07T14:26:03.756Z [ERROR] ResourceNotFoundException: An error occurred (ResourceNotFoundException) when calling the PutItem operation: Requested resource not found Traceback (mo…
2026-03-07T14:26:03.775Z END RequestId: 6ab18d68-43f2-4dd2-8b08-f4743f9e5754
2026-03-07T14:26:03.775Z REPORT RequestId: 6ab18d68-43f2-4dd2-8b08-f4743f9e5754 Duration: 2647.76 ms Billed Duration: 2989 ms Memory Size: 128 MB Max Memory Used: 83 MB Init Duration: …
2026-03-07T14:28:54.895Z START RequestId: 30ce0207-0d8b-40ed-a262-941810d14a57 Version: $LATEST
2026-03-07T14:28:55.275Z END RequestId: 30ce0207-0d8b-40ed-a262-941810d14a57
2026-03-07T14:28:55.275Z REPORT RequestId: 30ce0207-0d8b-40ed-a262-941810d14a57 Duration: 378.97 ms Billed Duration: 379 ms Memory Size: 128 MB Max Memory Used: 84 MB
2026-03-07T14:32:06.332Z START RequestId: 01372c4c-b958-4ed8-a922-e11b2b3642aa Version: $LATEST
2026-03-07T14:32:07.042Z END RequestId: 01372c4c-b958-4ed8-a922-e11b2b3642aa
2026-03-07T14:32:07.042Z REPORT RequestId: 01372c4c-b958-4ed8-a922-e11b2b3642aa Duration: 709.30 ms Billed Duration: 710 ms Memory Size: 128 MB Max Memory Used: 84 MB
No newer events at this moment. Auto retry paused. Resume

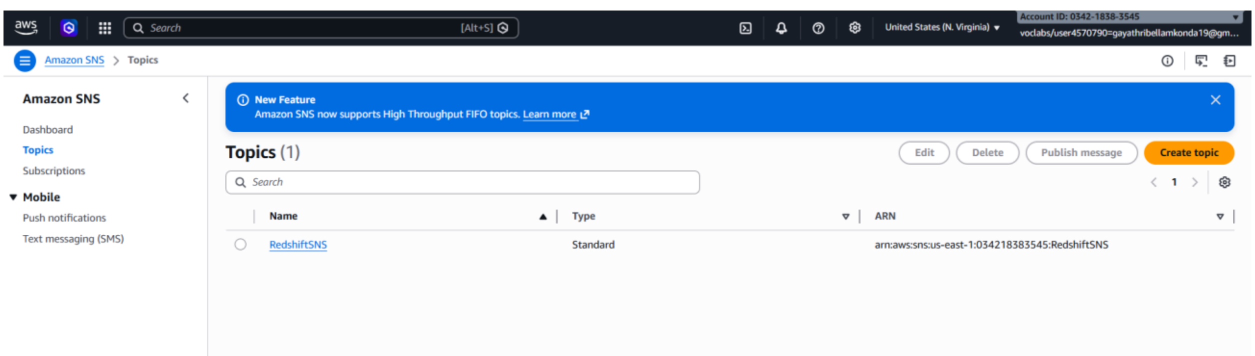
WEEK-8 -- SNS,SQS & S3-SNS-SQS-LAMBDA SERVICES

1) Simple Notification Sevice (SNS)
A) Create SNS Topic and Send Message to Email

Login to **AWS Console ->** In search bar → Type **SNS ->** Open **Amazon SNS (Simple Notification Service**

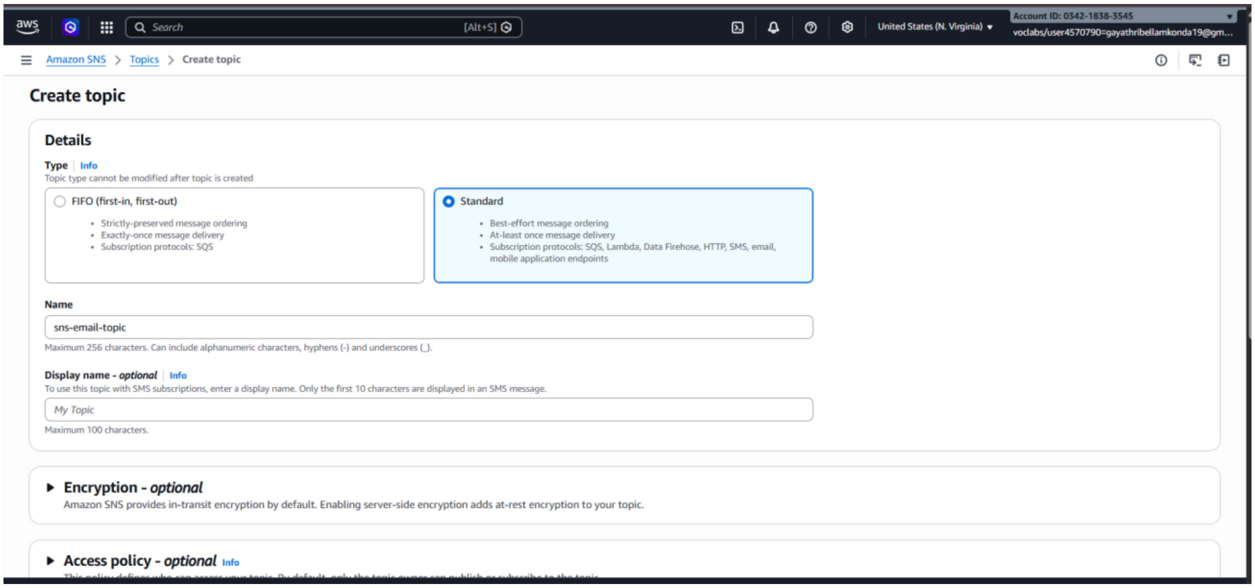


In left panel → Click **Topics ->** Click **Create topic**

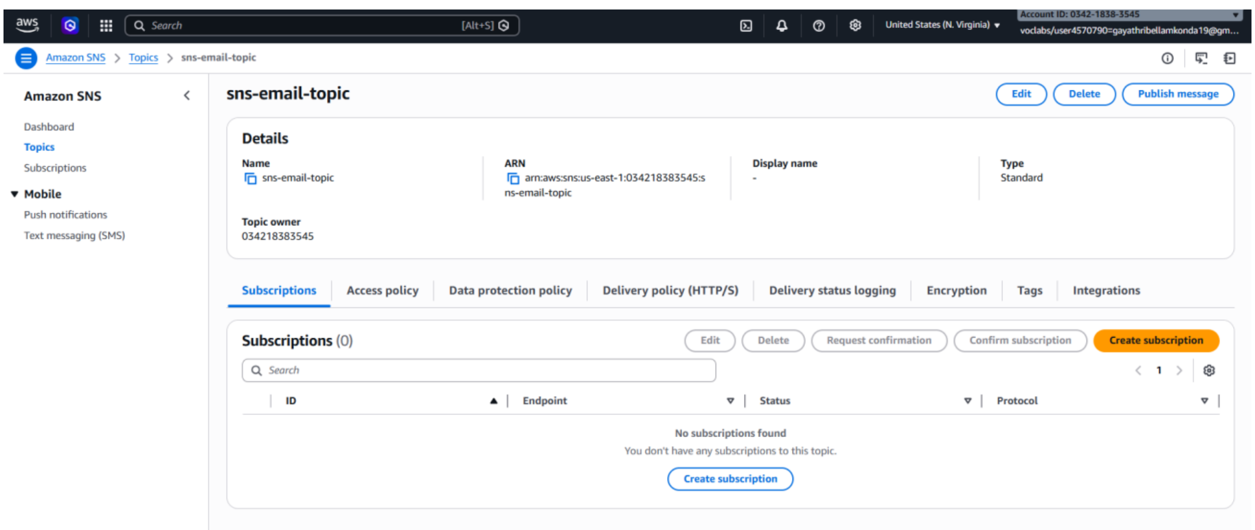


Type - Select **Standard -> Name** – MyEmailTopic -> Leave other settings default

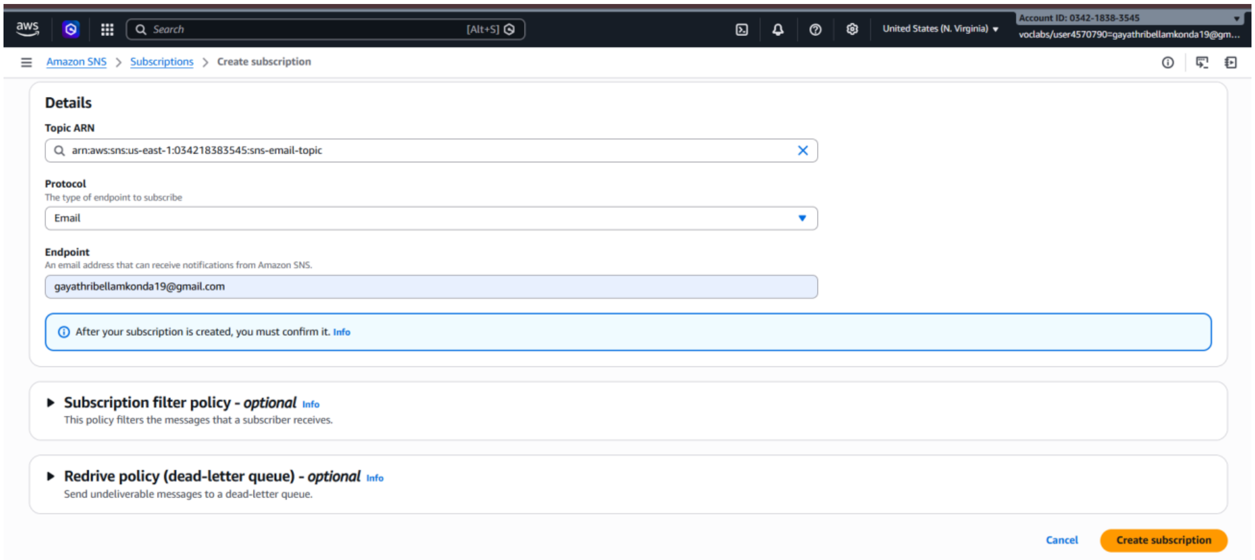
Click Create topic



Open created topic -> Go to **Subscriptions** tab -> Click **Create subscription**

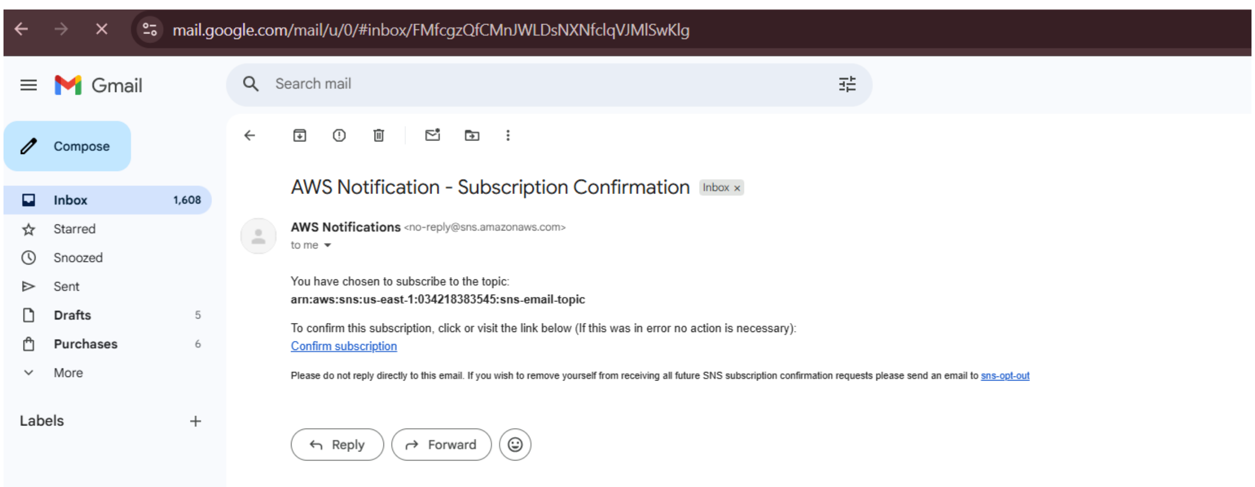


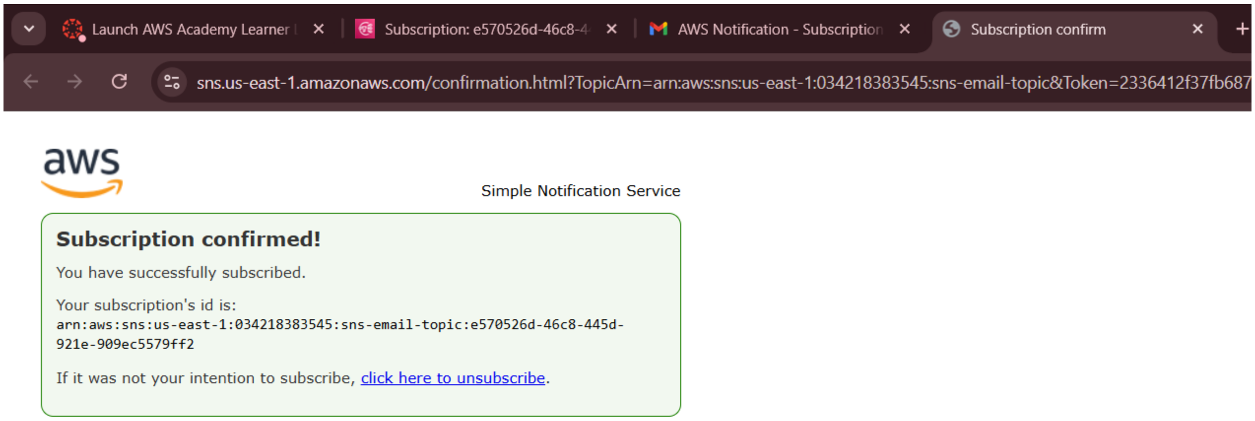
Protocol - Select **Email** -> **Endpoint** - Enter your email address -> Click **Create subscription**

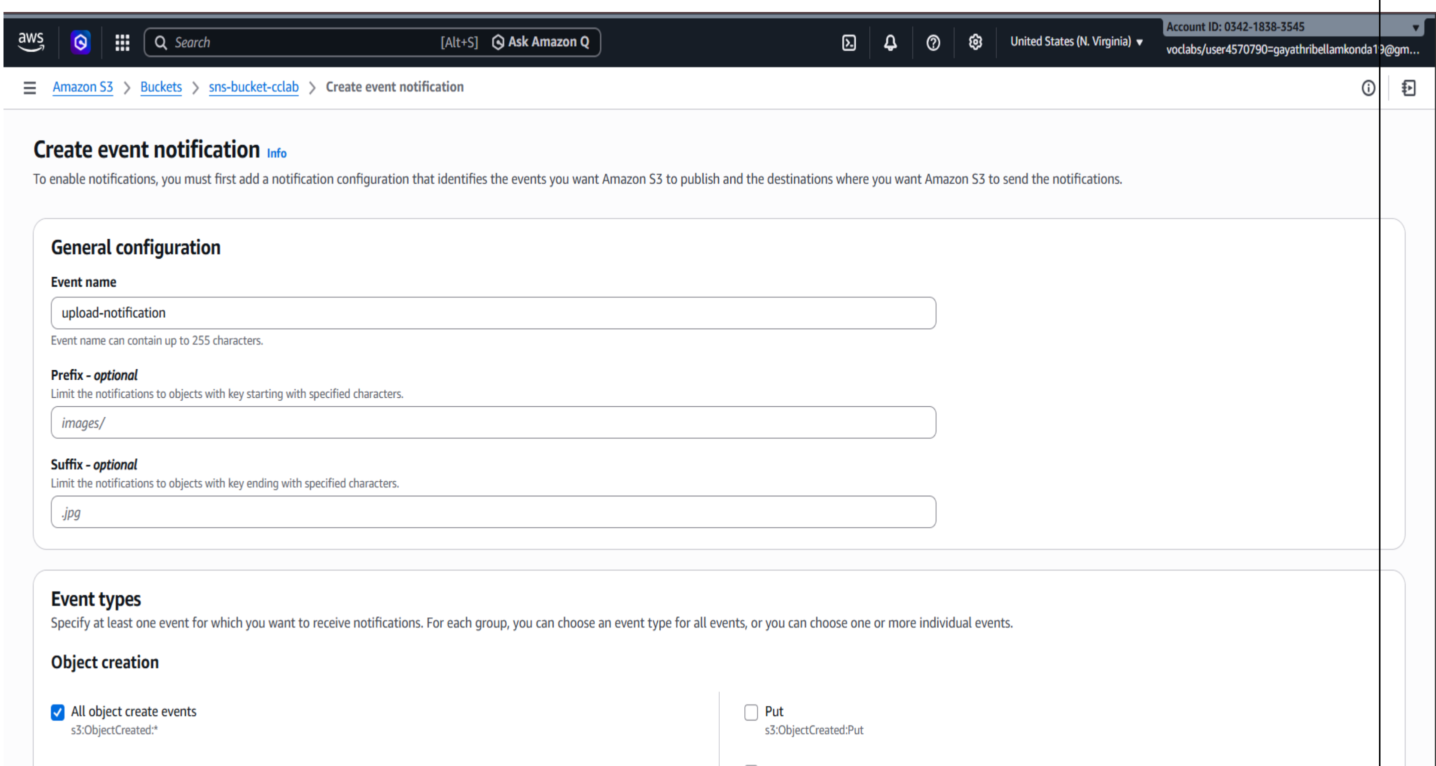


Go to your email inbox -> You will receive a mail from **Amazon SNS**

Click **Confirm subscription**

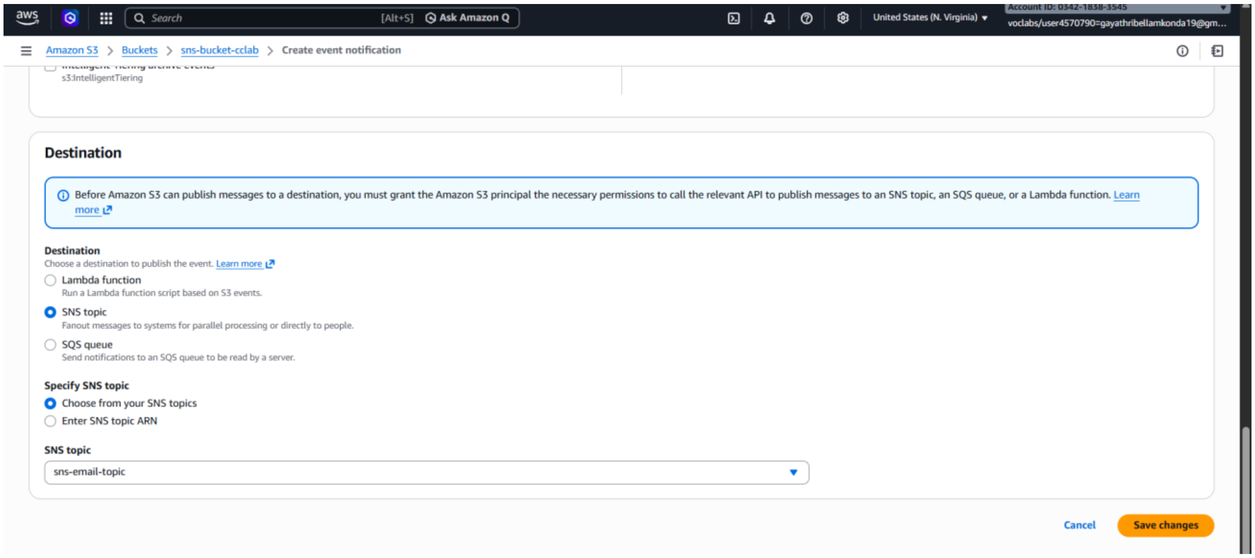




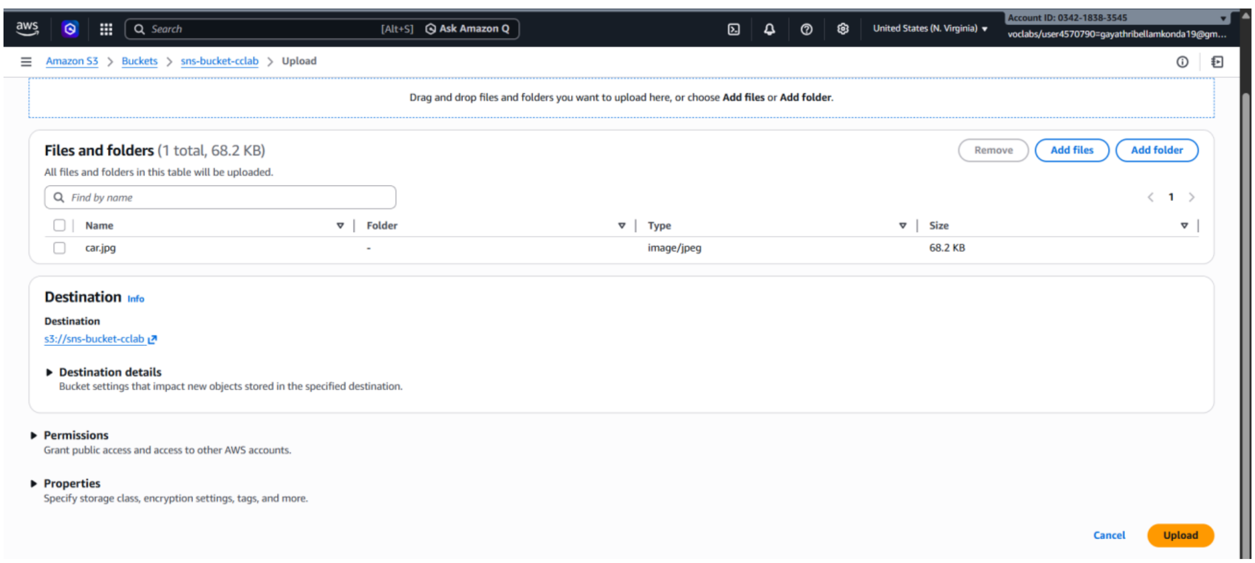


- Destination:
    - Select **SNS topic**
    - Choose your topic (S3UploadNotification)

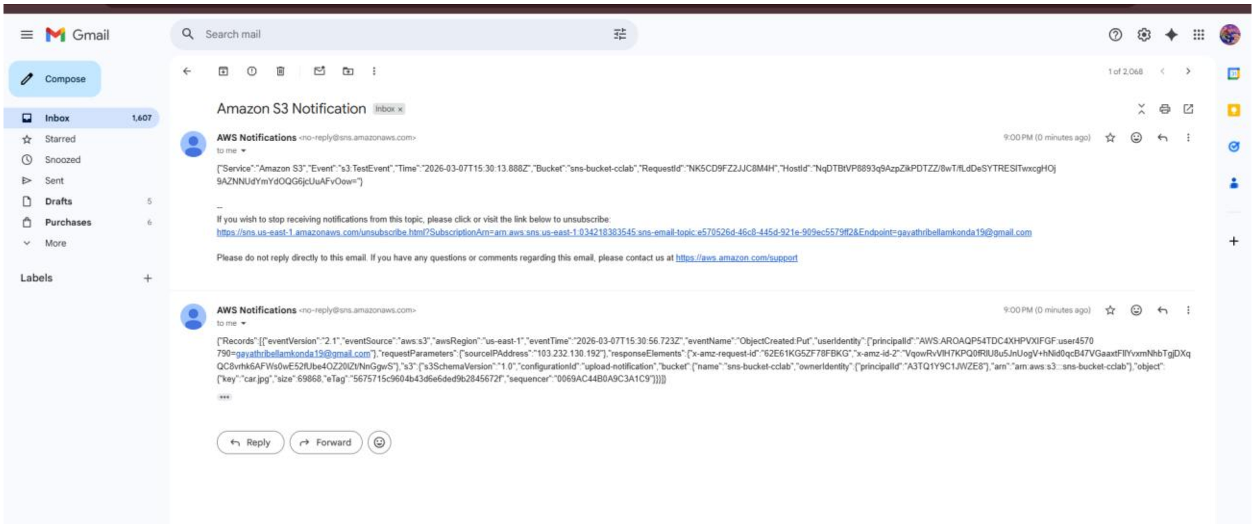
Click **Save changes**



Upload any file into the S3 bucket

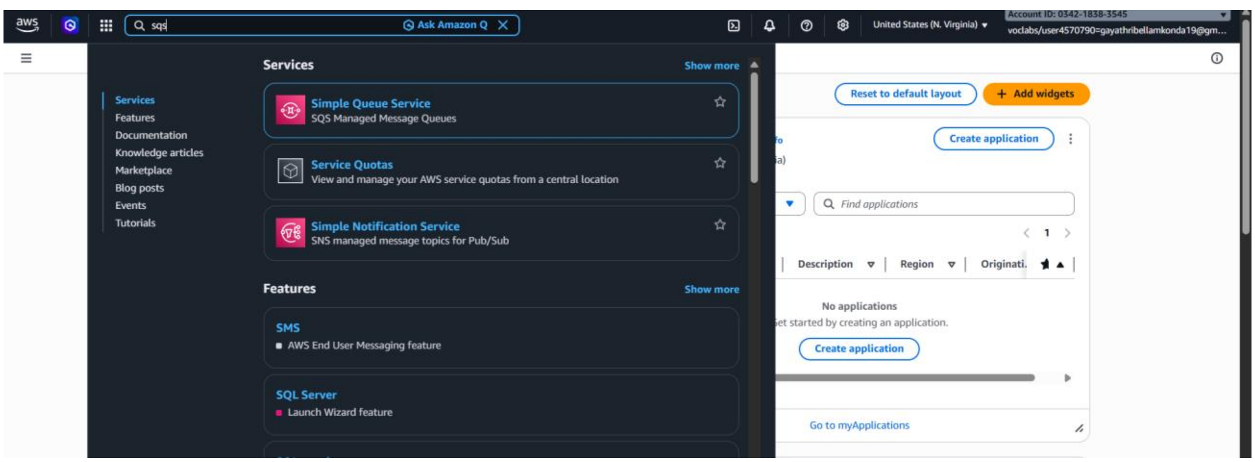


Mail about object upload will be received.

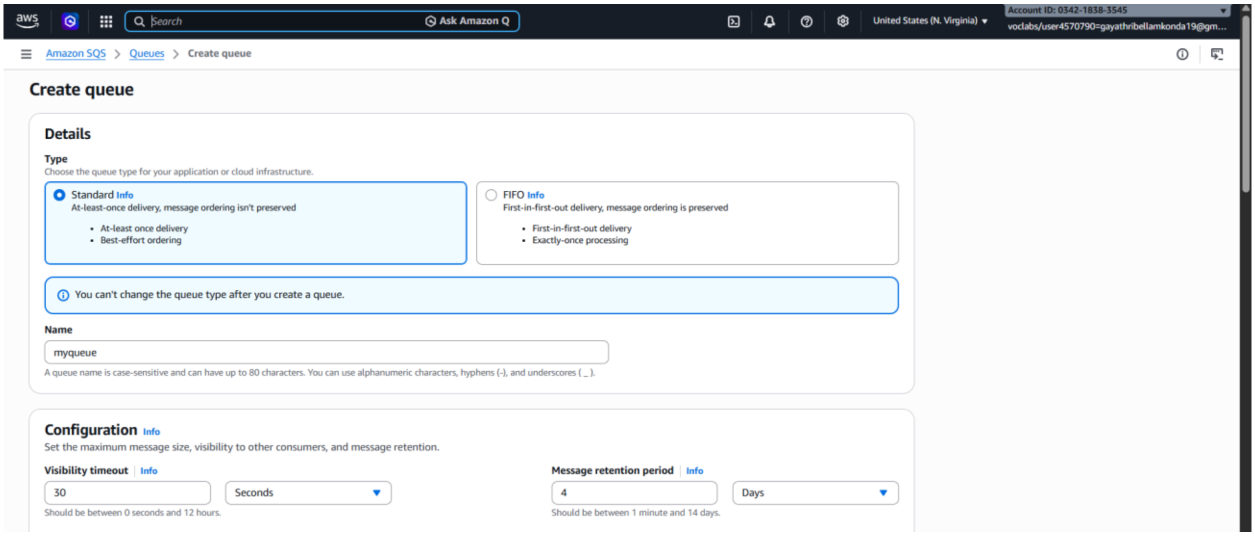


Simple Queue Service (SQS)

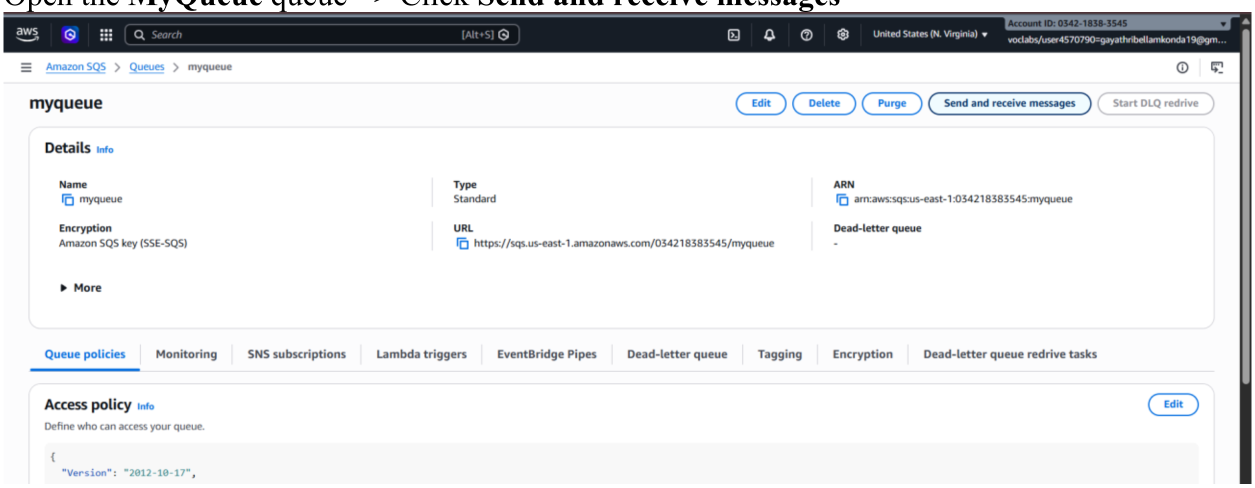
In the **search bar → Type SQS**
Open **Amazon Simple Queue Service**



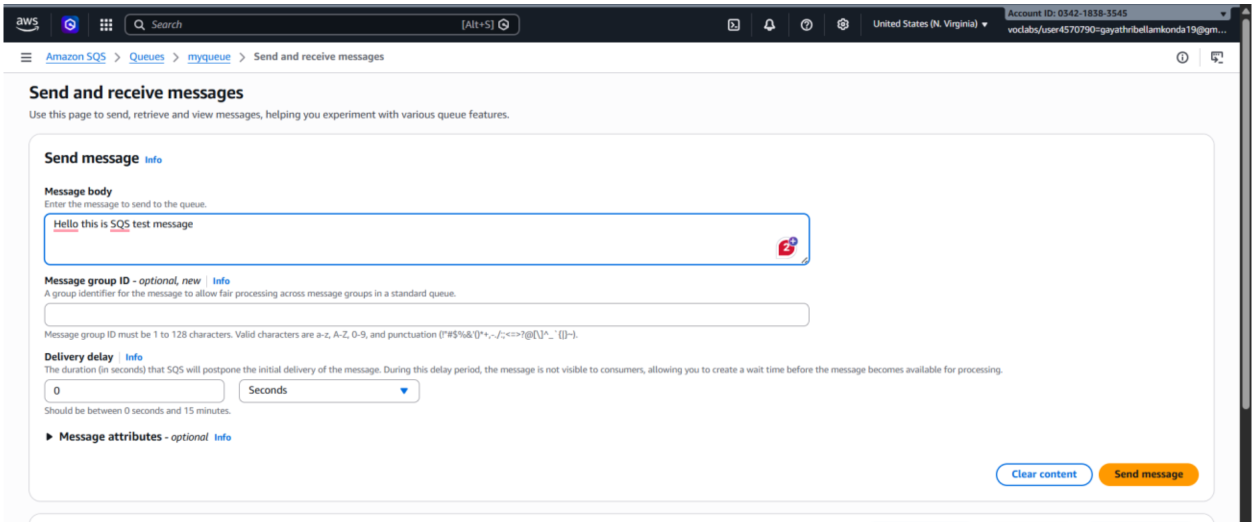
In the dashboard → Click **Create queue**
Select Standard -> Name : MyQueue
Leave other settings as **default**
Click **Create queue**

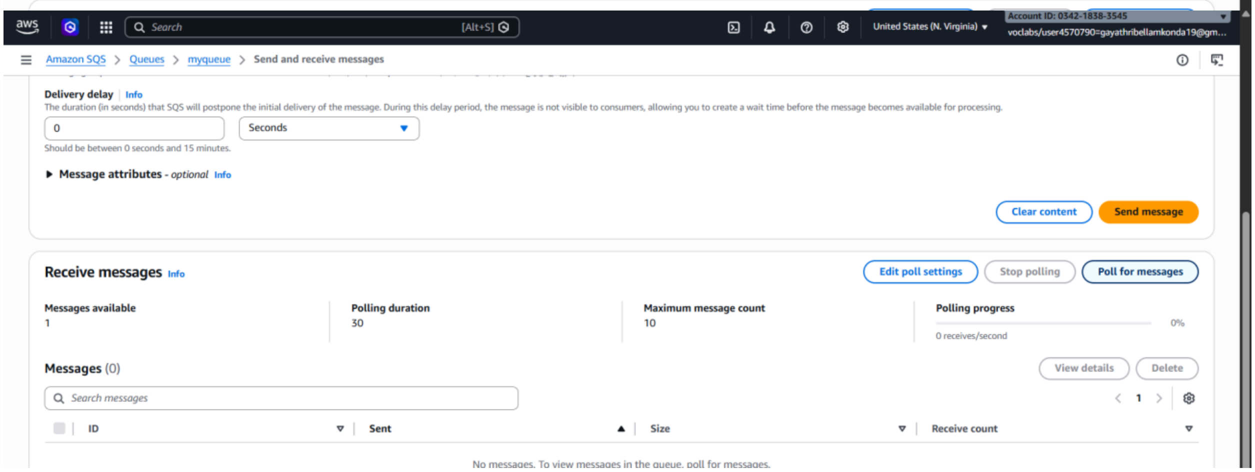


Open the **MyQueue** queue -> Click **Send and receive messages**



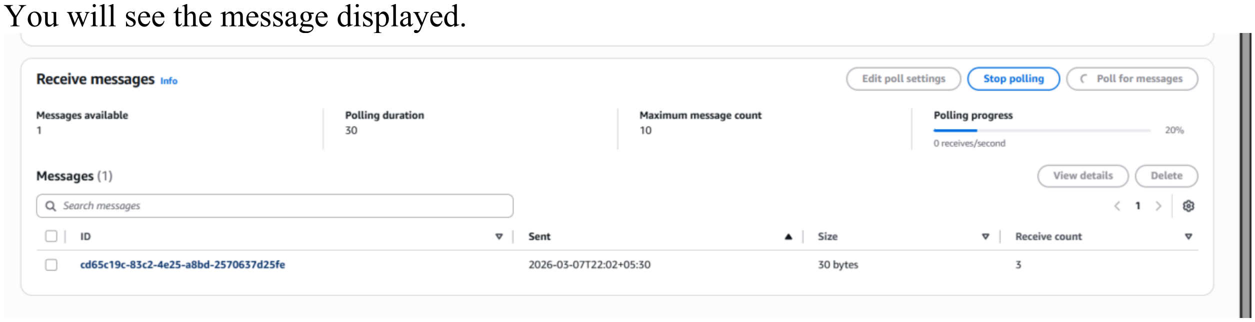
In the **Message body**, type : Hello this is SQS test message
Click **Send message**

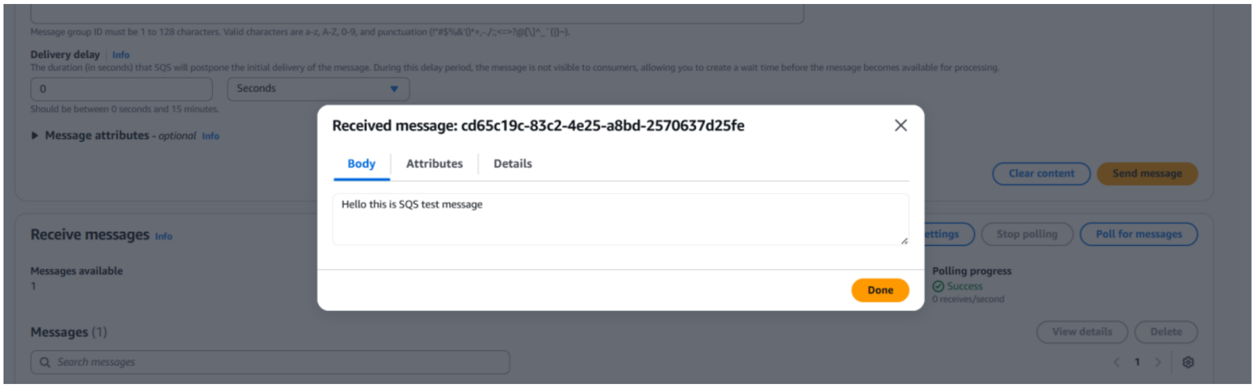




1. In the same page (**Send and receive messages**)
2. Click **Poll for messages**

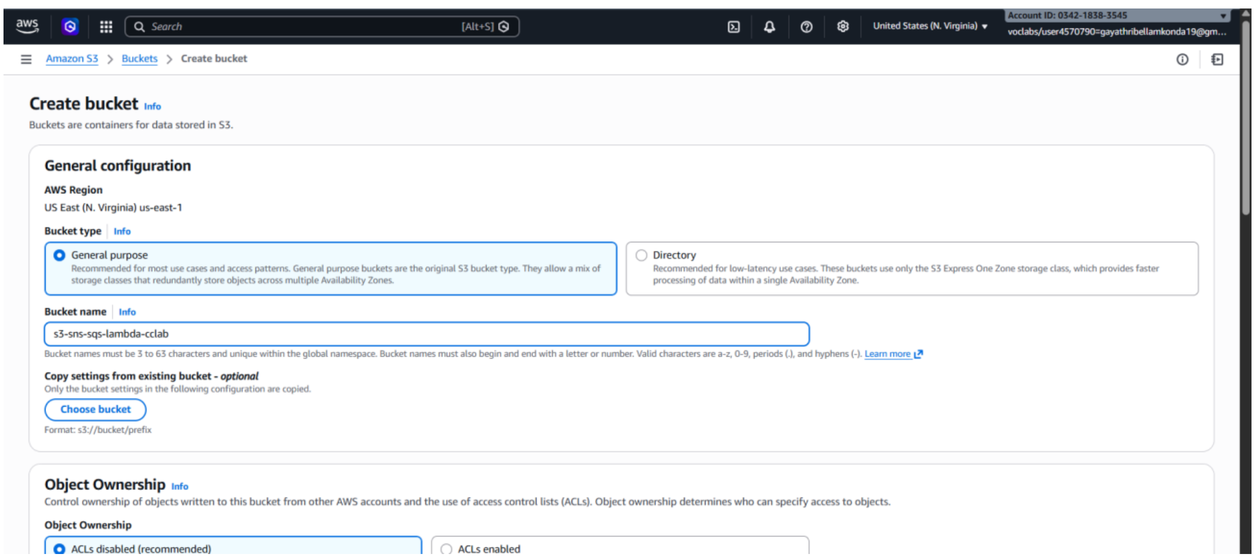
You will see the message displayed.





3) S3 Bucket Event → SNS → SQS → Lambda Processing

- Search **S3** -> Open **Amazon S3** -> Click **Create bucket**
- Bucket name → **s3-sns-sqs-lambda**
- Leave other settings default.



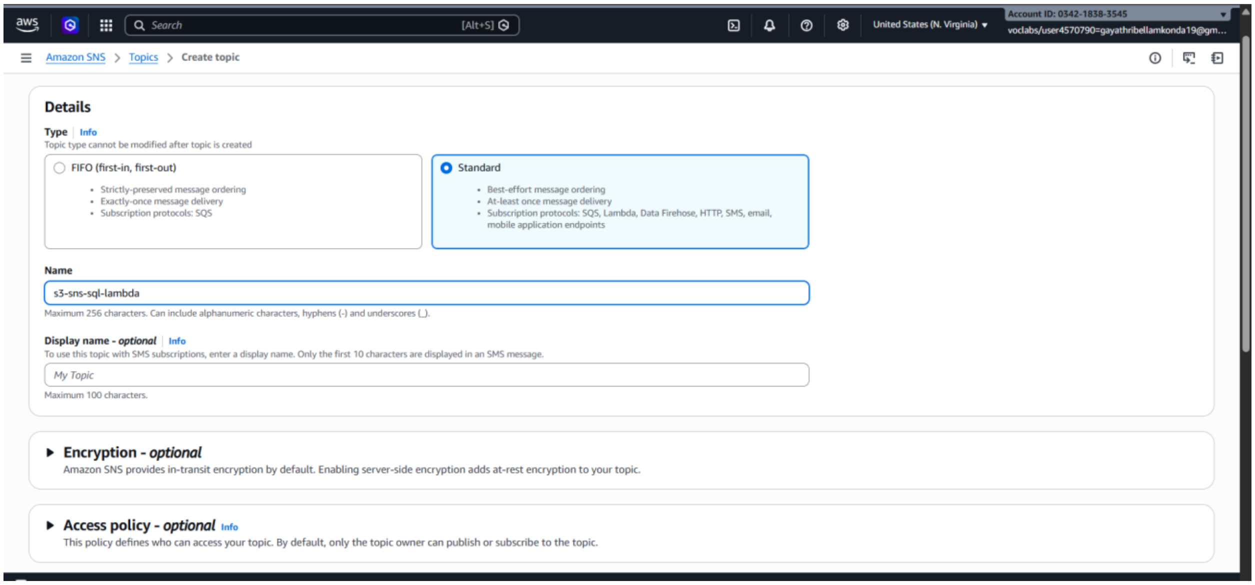
Search **SNS** in AWS Console -> Open **Amazon SNS**

Click **Topics** → **Create topic**

Configure:

• Type → **Standard**
• Name → **MyS3SNSTopic**

Click **Create topic** -> Copy the **SNS Topic ARN**.

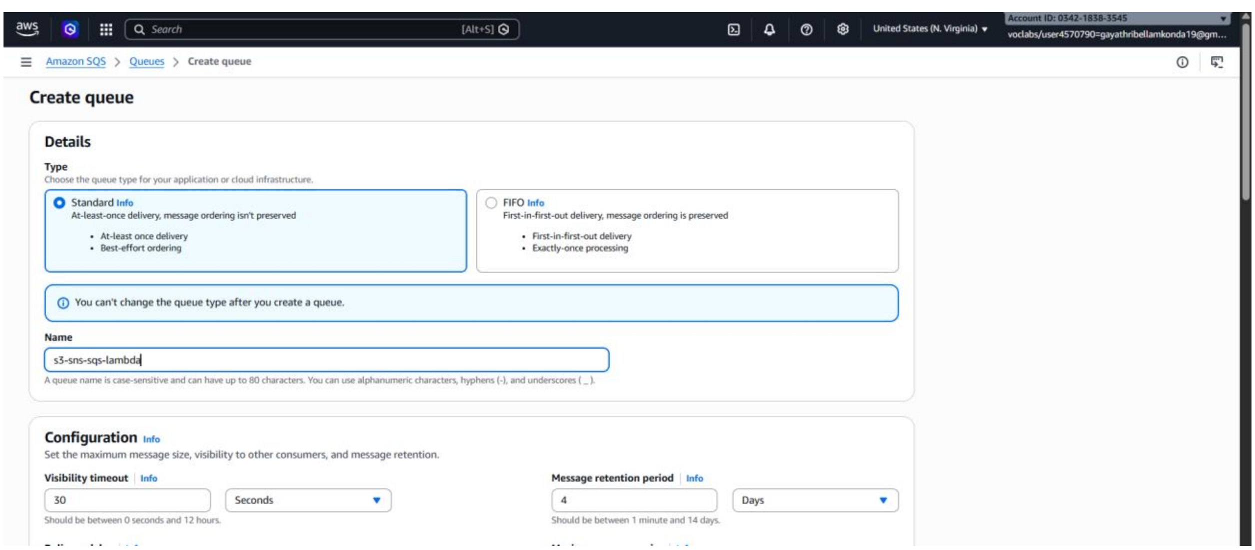


Search **SQS** in AWS Console -> Open **Amazon SQS** -> Click **Create queue**

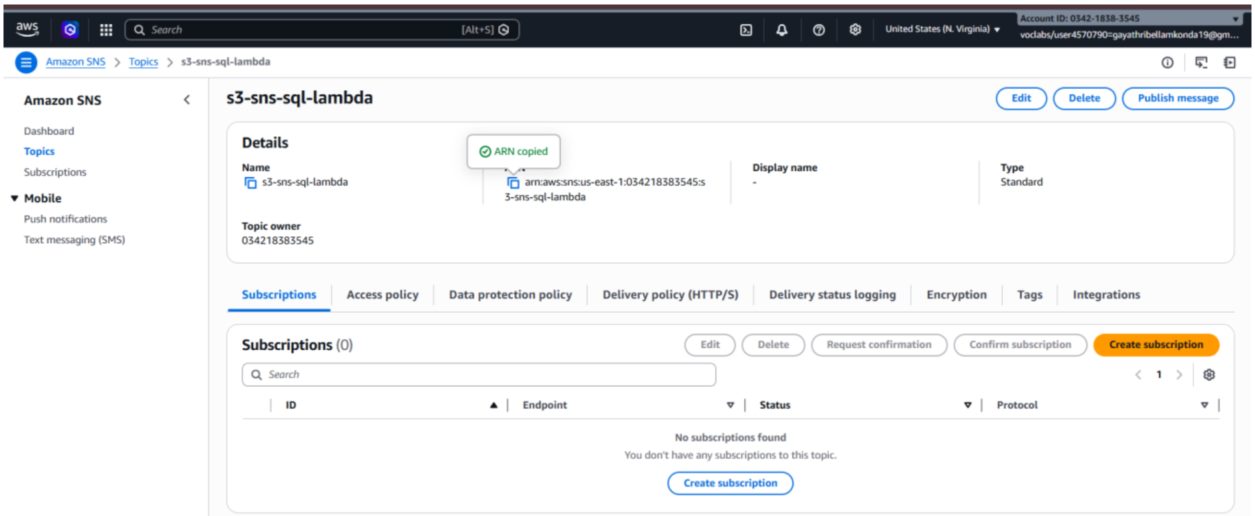
Configure:

- Type → **Standard**
- Name → **MyS3Queue**

Click **Create queue** .



Copy the **Queue ARN**

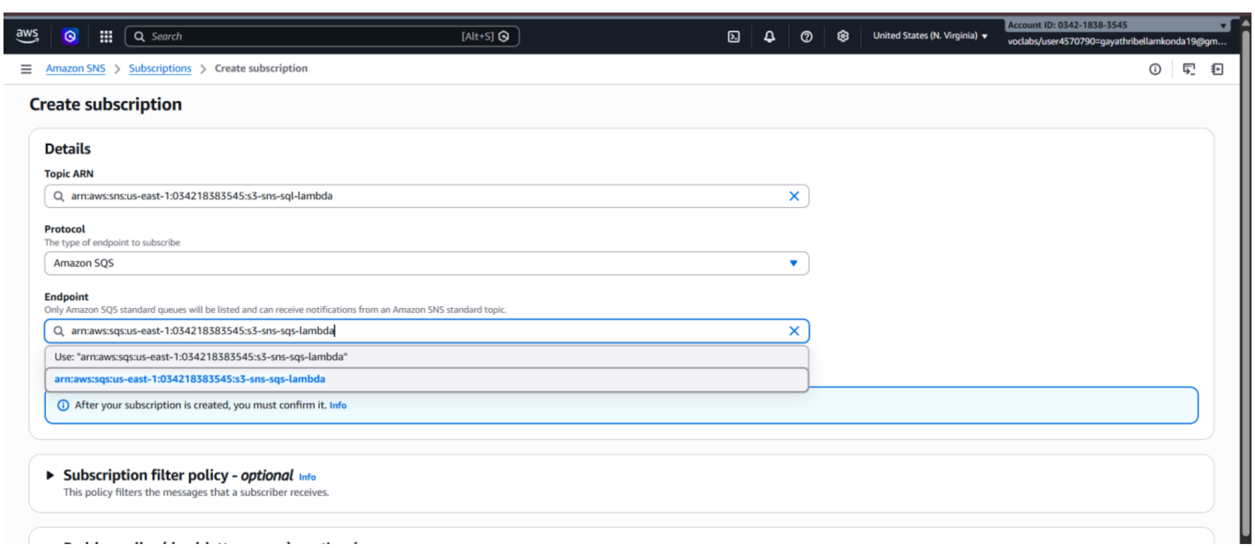


Open **MyS3SNSTopic** -> Click **Create subscription**

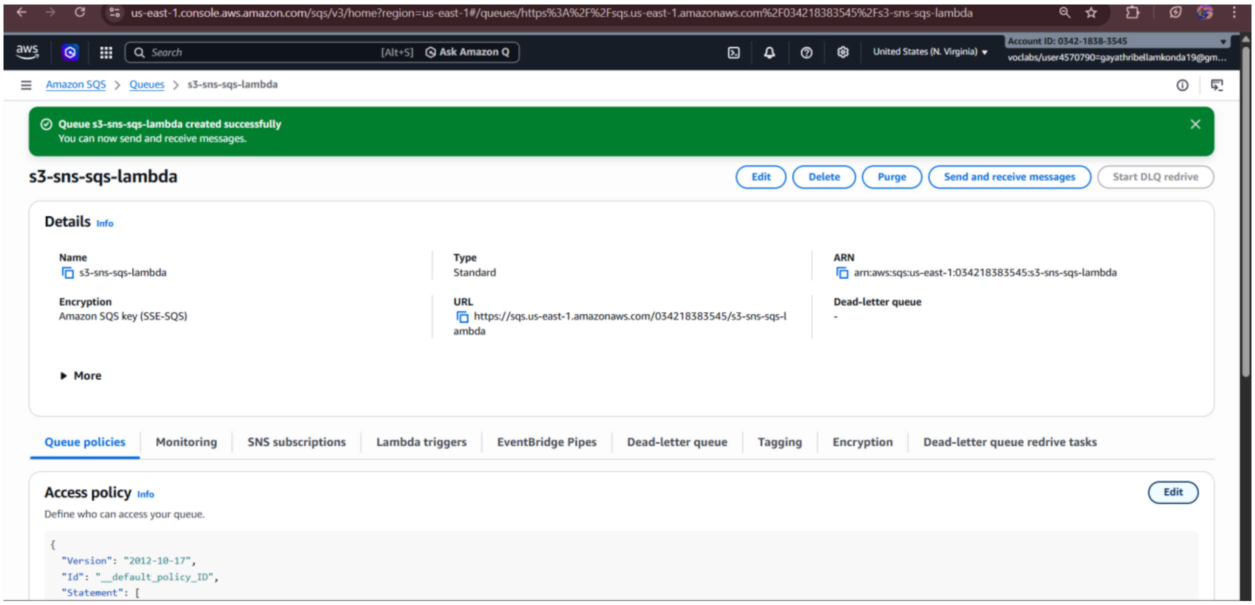
Configure:

- Protocol → **Amazon SQS**
- Endpoint → Select **MyS3Queue**

Click **Create subscription**

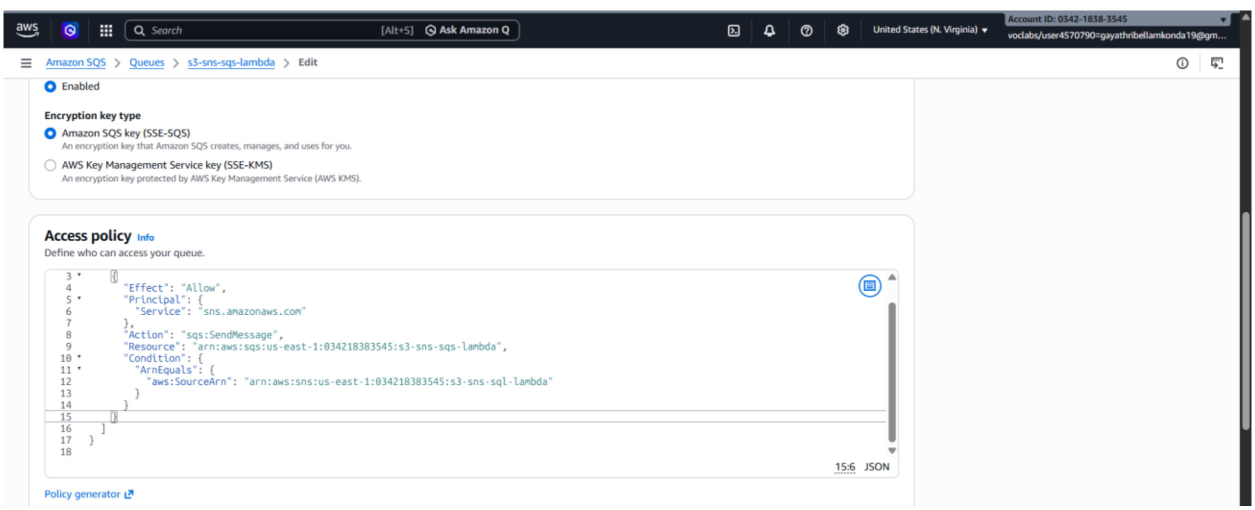


Open **MyS3Queue → Access Policy → Edit**

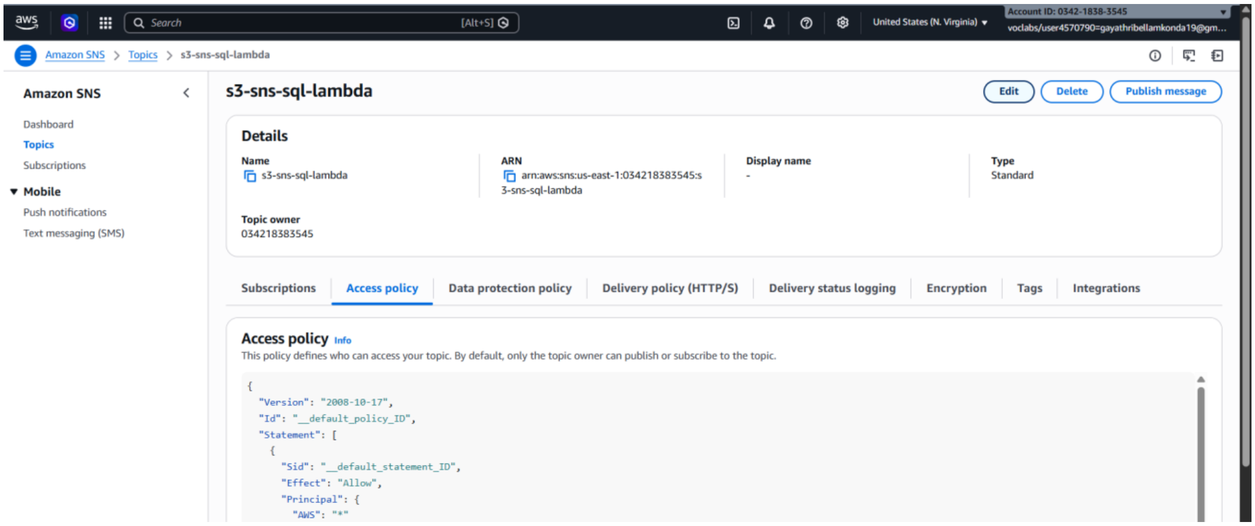


Replace:

- SQS ARN
- SNS ARN

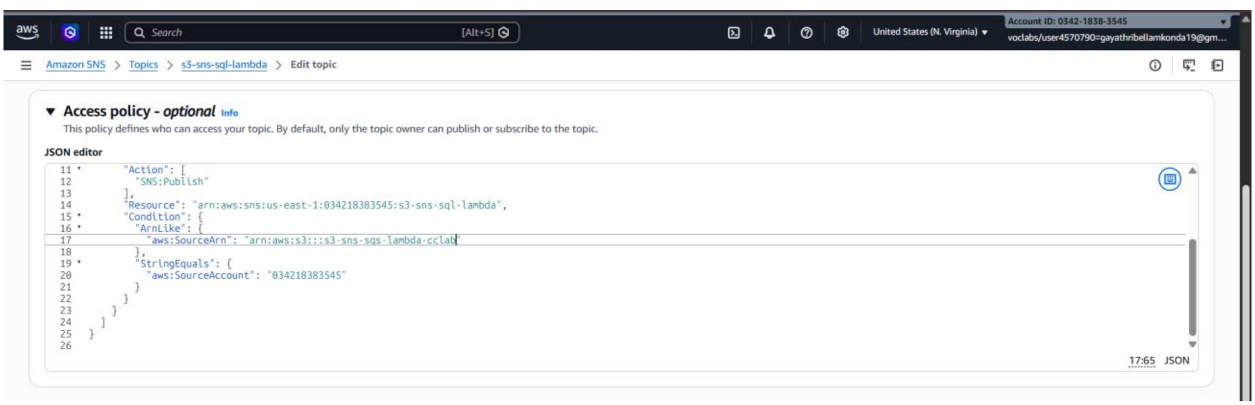


Open **SNS Topic → Edit access policy**



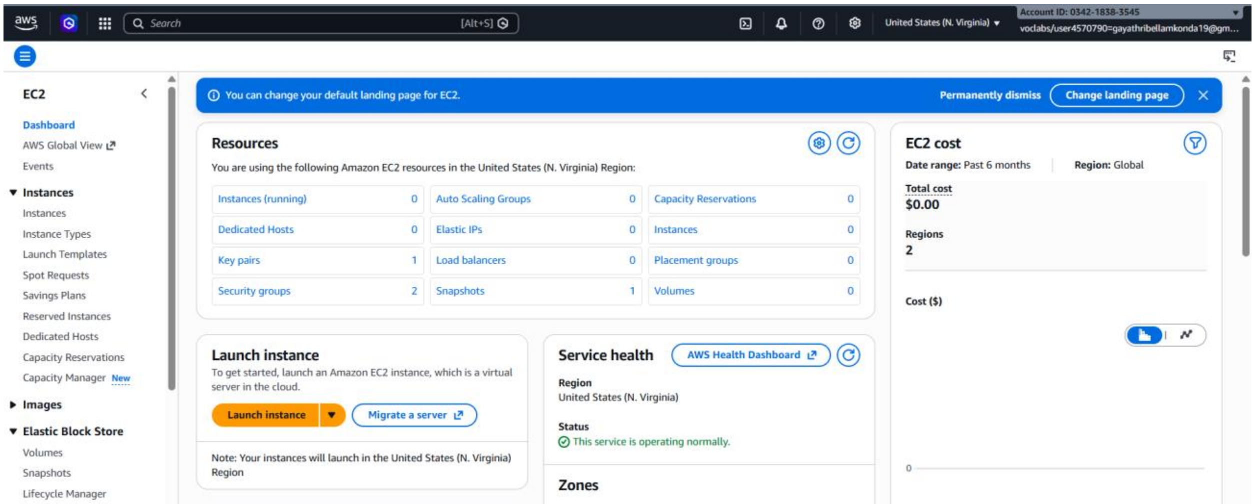
Replace:

- SNS topic ARN
- S3 bucket ARN
- Account ID

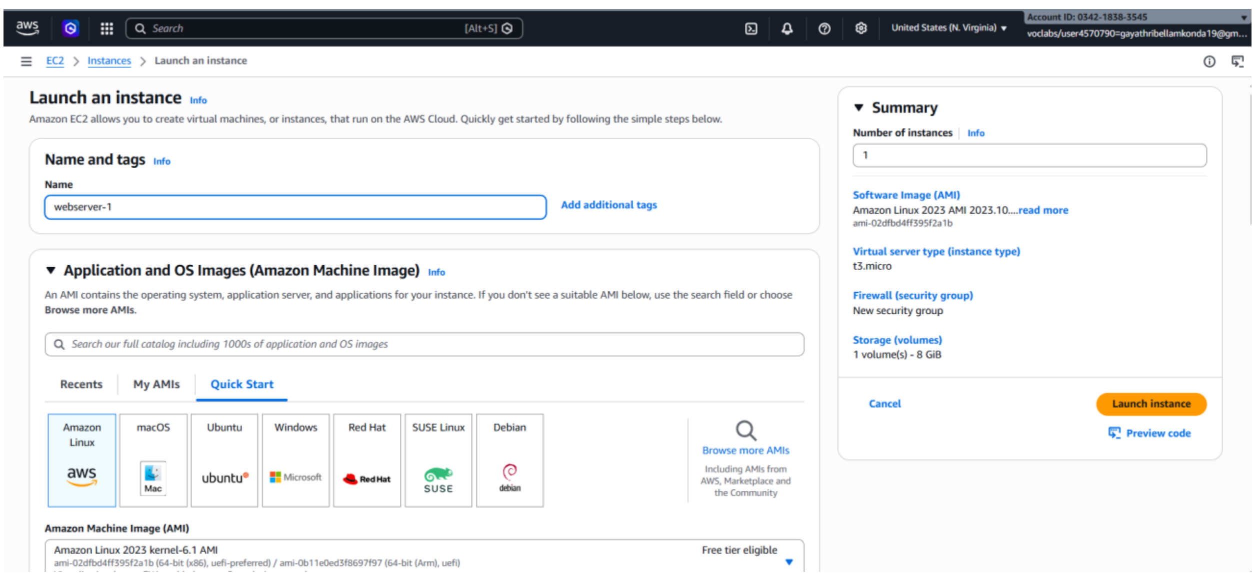


WEEK-9- Implementation of Elastic Load Balancer with Auto Scaling using AWS EC2 Instance

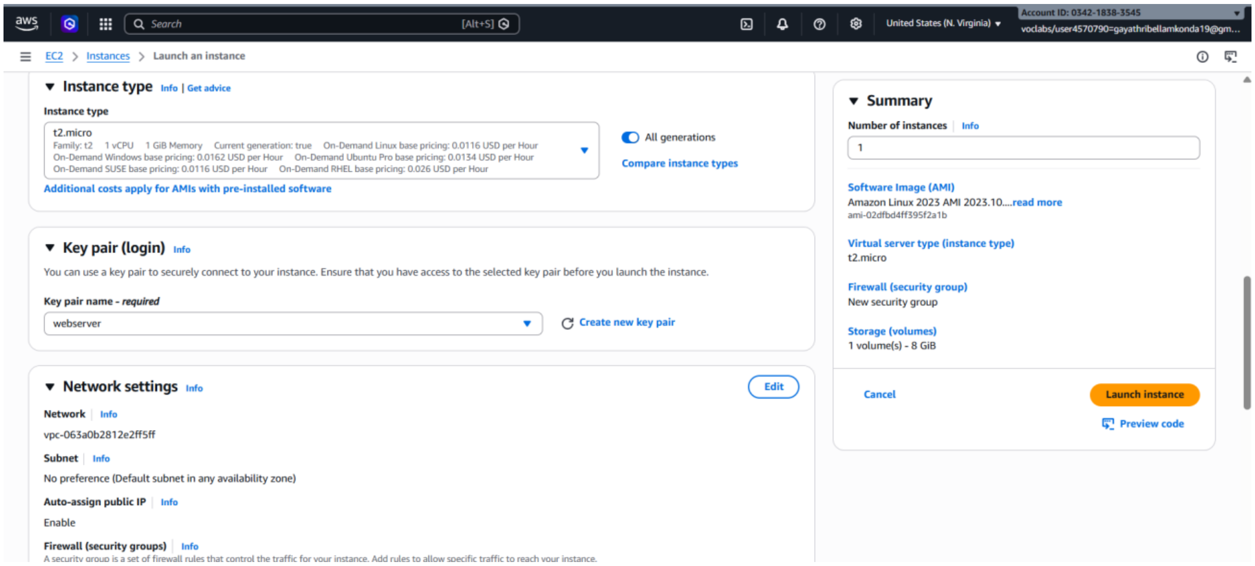
Go to **AWS EC2 Console →click on Launch Instance**.



Name: webserver-1 -> Choose an **AMI** (Amazon Linux 2)



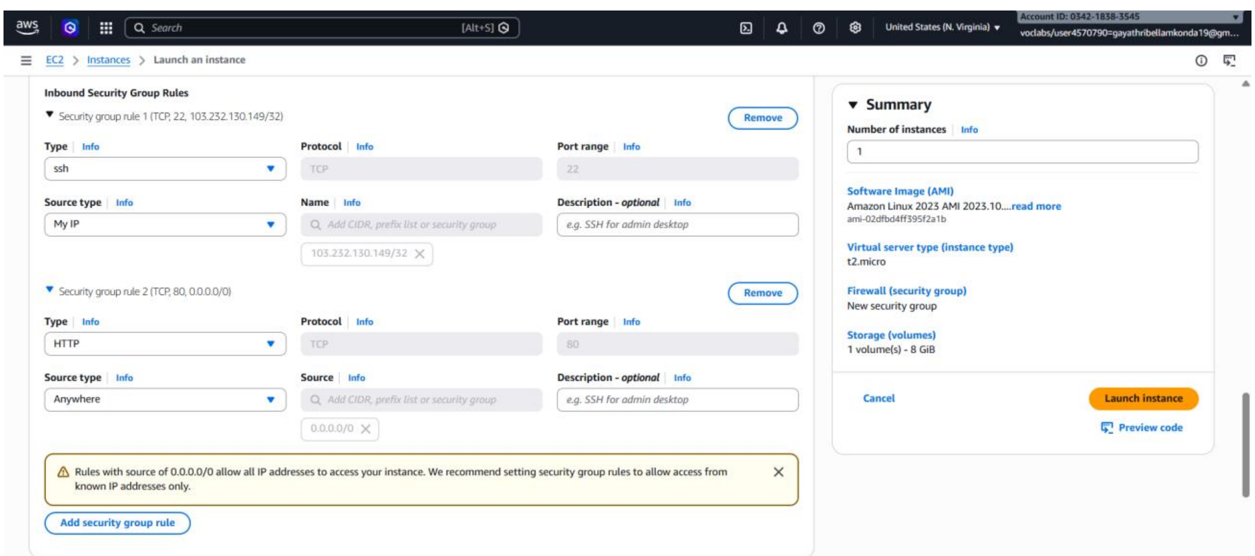
Select **Instance Type** (e.g., t2.micro) -> Create Key Pair -> click on edit button of network settings



Configure **Security Group**:

- Allow **SSH (22)** from your IP
- Allow **HTTP (80)** from anywhere (0.0.0.0/0)

Click on launch instance



Copy the ssh key of instance and paste in the powershell

Run sudo yum update –y command in the powershell

```
PS C:\Users\Admin\Downloads> ssh -i "webserver.pem" ec2-user@ec2-54-163-7-147.compute-1.amazonaws.com
The authenticity of host 'ec2-54-163-7-147.compute-1.amazonaws.com (54.163.7.147)' can't be established.
ED25519 key fingerprint is SHA256:MdoNefrKftF9F0gy1EYgs9ovkdrBs+928TSmb/EFlkA.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-54-163-7-147.compute-1.amazonaws.com' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-172-31-27-38 ~]$ sudo yum update -y
Amazon Linux 2023 Kernel Livepatch repository                              172 kB/s |  30 kB     00:00
Dependencies resolved.
Nothing to do.
Complete!
```

Run sudo yum install httpd –y command in the powershell

```
ec2-user@ip-172-31-27-38:~
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-27-38 ~]$ sudo yum install httpd -y
Last metadata expiration check: 0:00:21 ago on Fri Mar 13 15:52:55 2026.
Dependencies resolved.
========================================================================================================================
 Package                         Architecture       Version                                Repository           Size
========================================================================================================================
Installing:
 httpd                           x86_64             2.4.66-1.amzn2023.0.1                  amazonlinux          47 k
Installing dependencies:
 apr                             x86_64             1.7.5-1.amzn2023.0.4                   amazonlinux         129 k
 apr-util                        x86_64             1.6.3-1.amzn2023.0.2                   amazonlinux          97 k
 apr-util-lmdb                   x86_64             1.6.3-1.amzn2023.0.2                   amazonlinux          13 k
 generic-logos-httpd             noarch             18.0.0-12.amzn2023.0.3                 amazonlinux          19 k
 httpd-core                      x86_64             2.4.66-1.amzn2023.0.1                  amazonlinux         1.4 M
 httpd-filesystem                noarch             2.4.66-1.amzn2023.0.1                  amazonlinux          13 k
 httpd-tools                     x86_64             2.4.66-1.amzn2023.0.1                  amazonlinux          81 k
 libbrotli                       x86_64             1.0.9-4.amzn2023.0.2                   amazonlinux         315 k
 mailcap                         noarch             2.1.49-3.amzn2023.0.3                  amazonlinux          33 k
Installing weak dependencies:
```

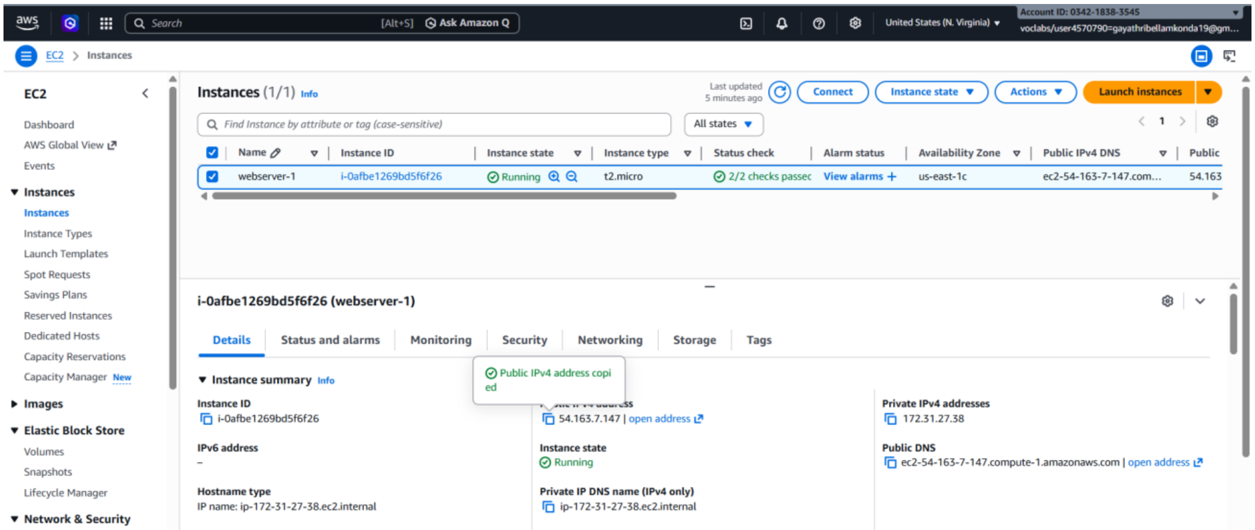
Run systemctl start httpd

systemctl enable httpd

echo "This is Server 1" > /var/www/html/index.html

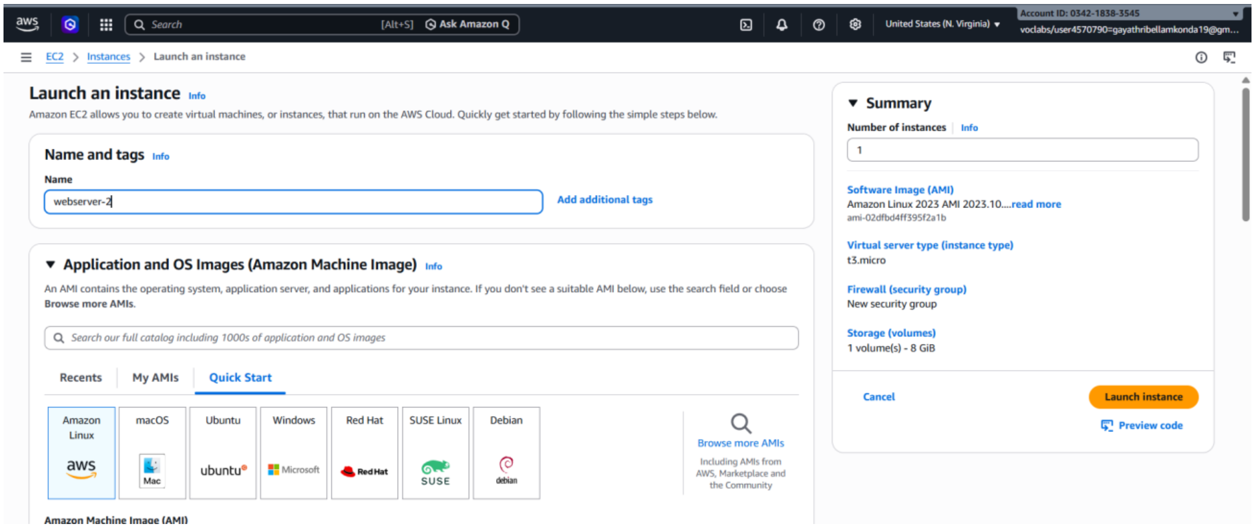
```
[ec2-user@ip-172-31-27-38 ~]$ sudo systemctl start httpd
[ec2-user@ip-172-31-27-38 ~]$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[ec2-user@ip-172-31-27-38 ~]$ echo "This is Server 1" | sudo tee /var/www/html/index.html
This is Server 1
[ec2-user@ip-172-31-27-38 ~]$
```

Select the instance and copy the public ip

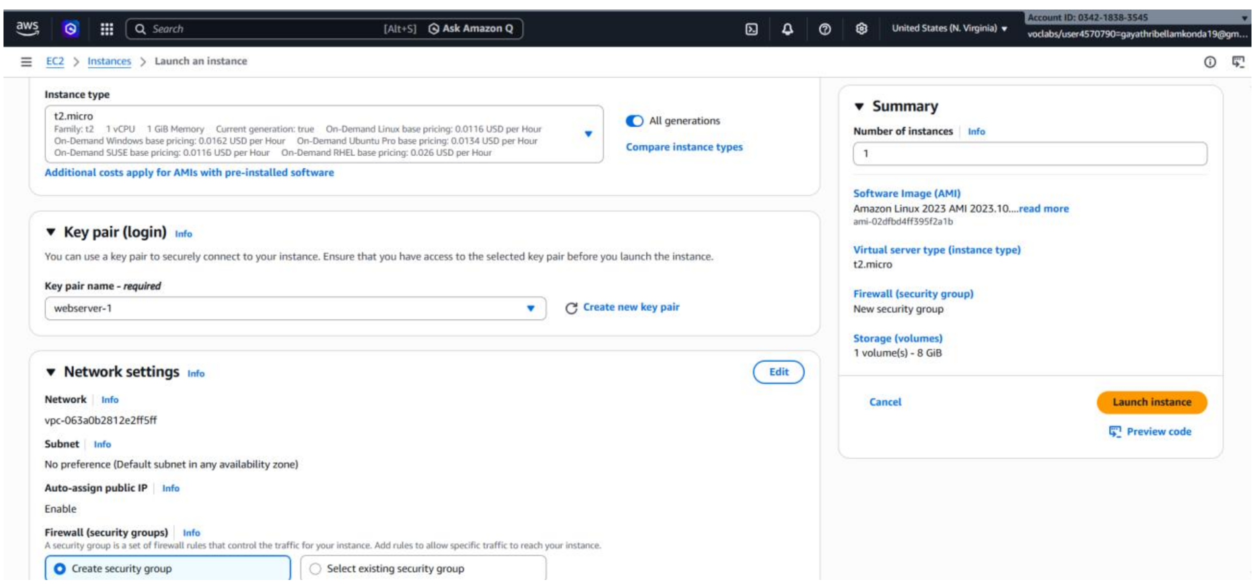


Paste the ip address in the incognito window -> the output will be (This is server 1)

Create another instance -> Name : webserver-2 -> select AMI as Amazon Linux



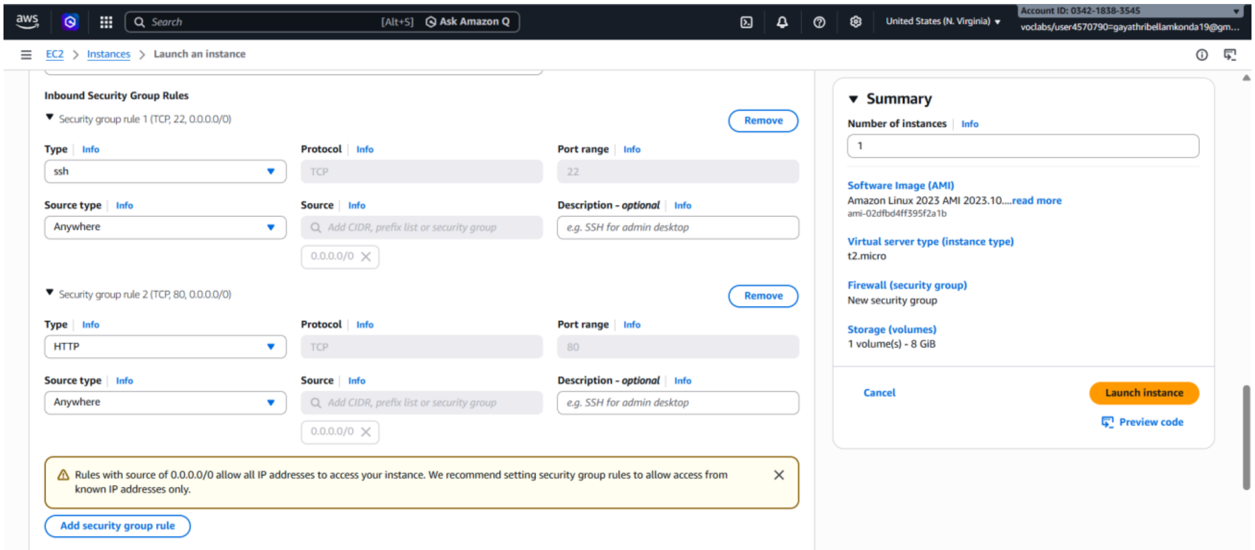
Select t2.micro as instance type -> create a key pair -> edit the network settings



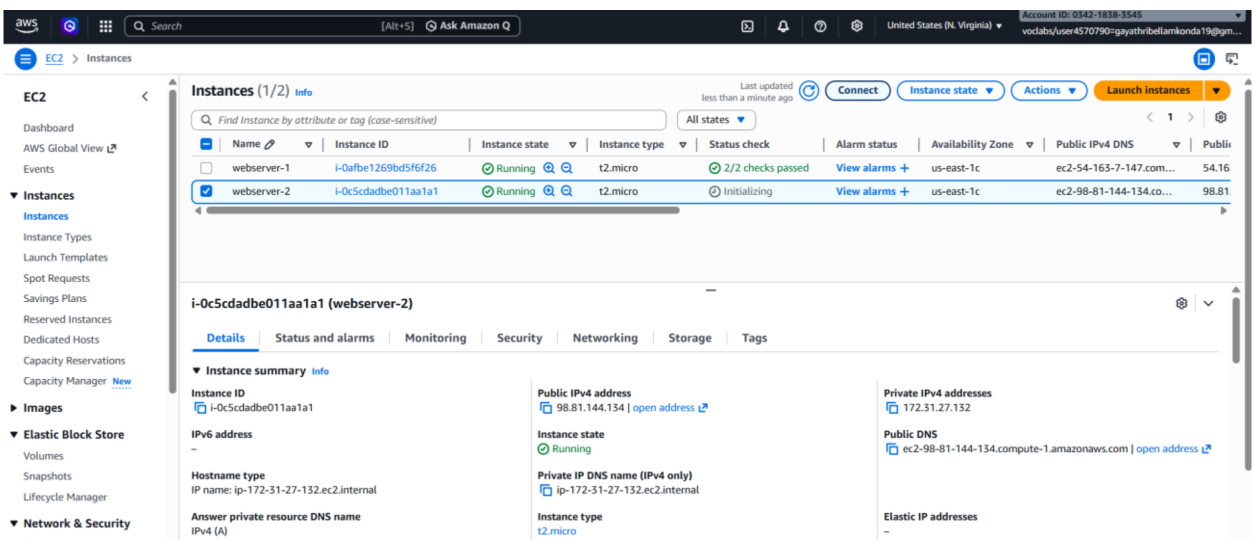
Configure **Security Group**:

- Allow **SSH (22)** from anywhere
- Allow **HTTP (80)** from anywhere (0.0.0.0/0)

Click on launch instance

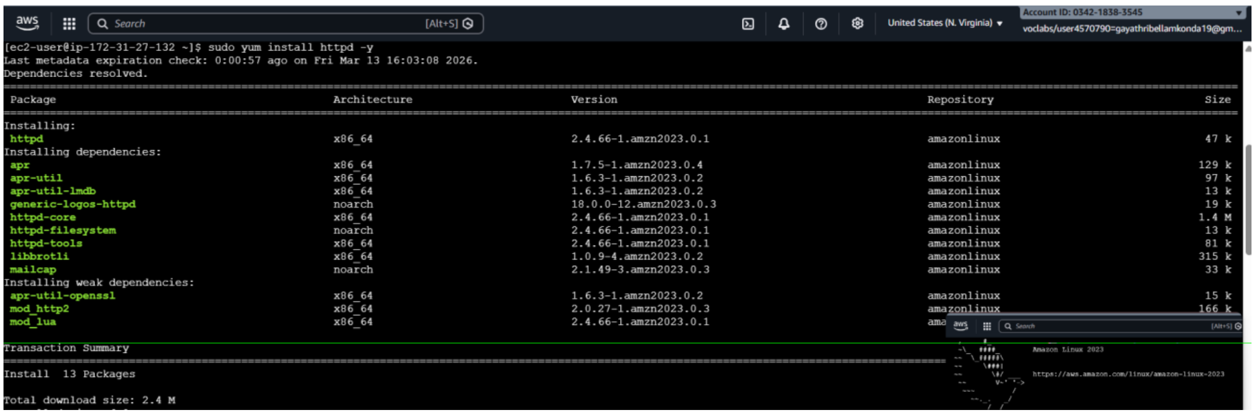


Select the instance and click on connect



Run sudo yum update –y

Run sudo yum install httpd –y

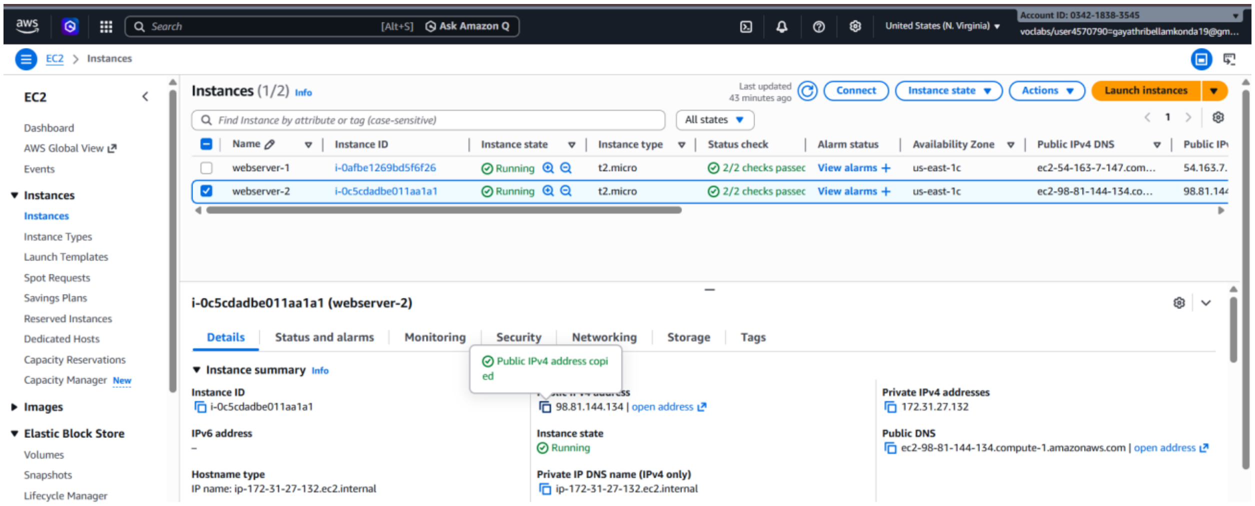


Run systemctl start httpd

systemctl enable httpd

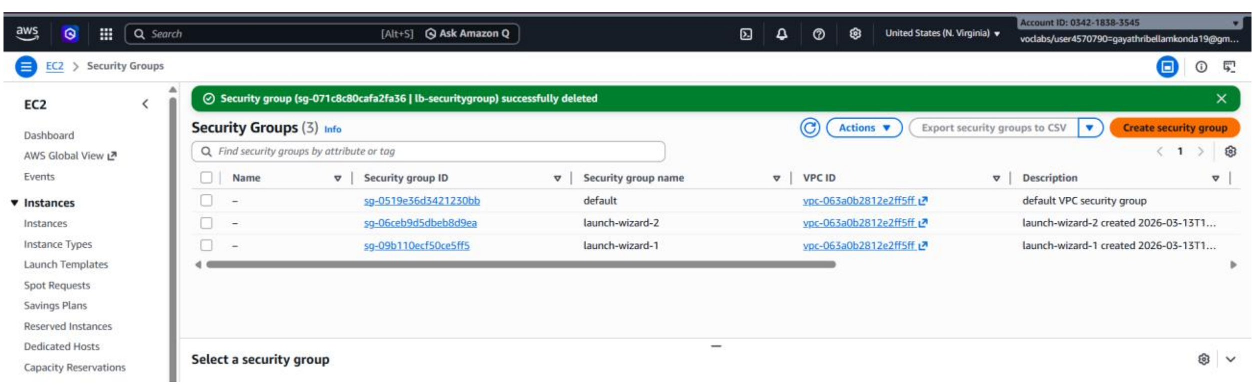
echo "This is Server 1" > /var/www/html/index.html

Select the instance and copy the public ip address

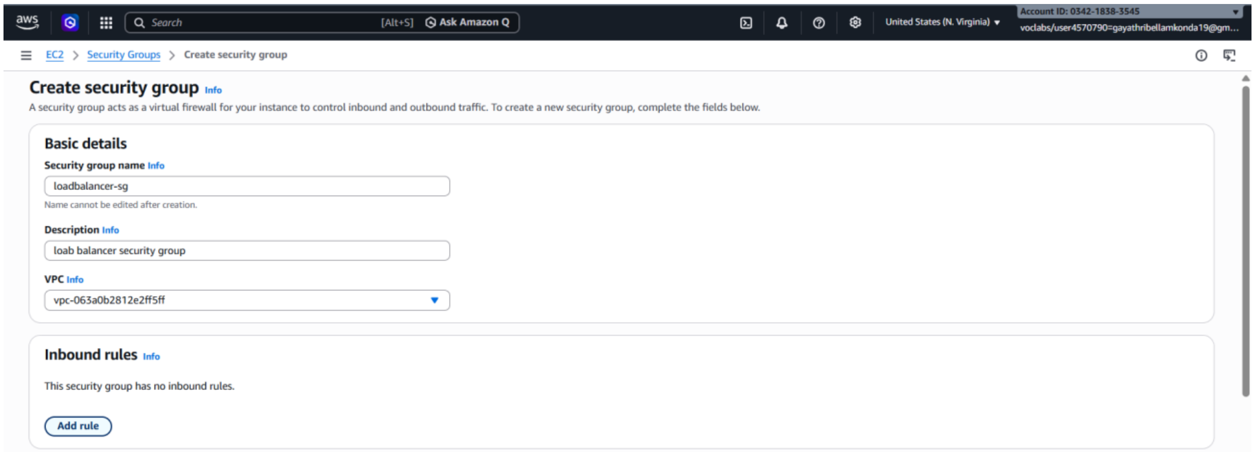


Paste the ip address in the incognito window -> the output will be (This is server 2)

Go to **EC2 → Security Groups → Click on Create Security Group**



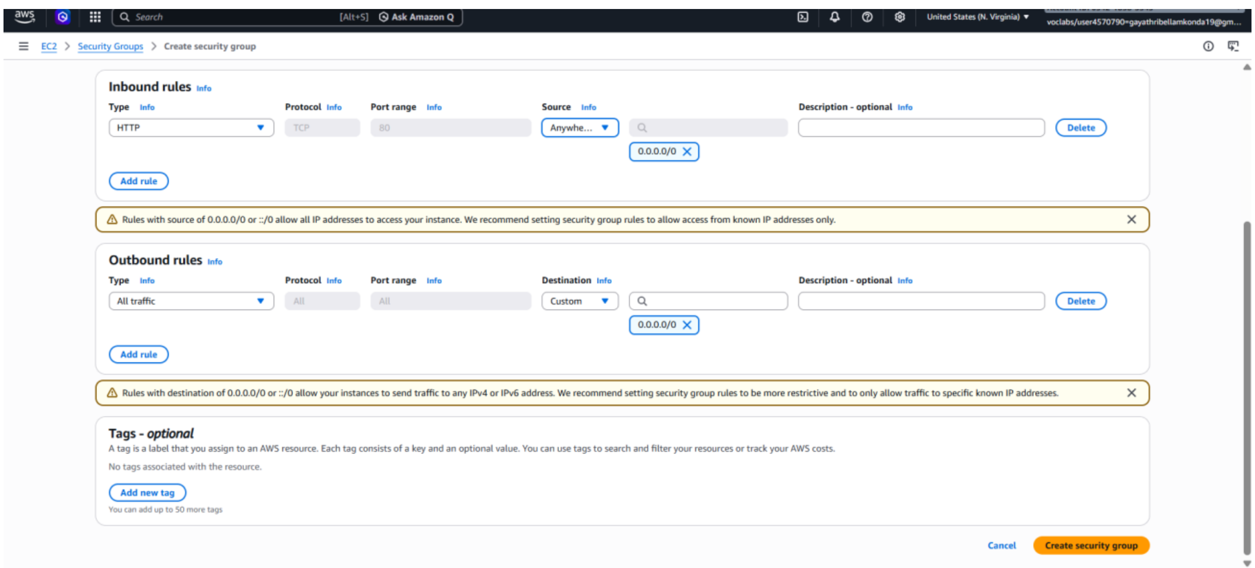
Name : loadbalancer-sg -> give description



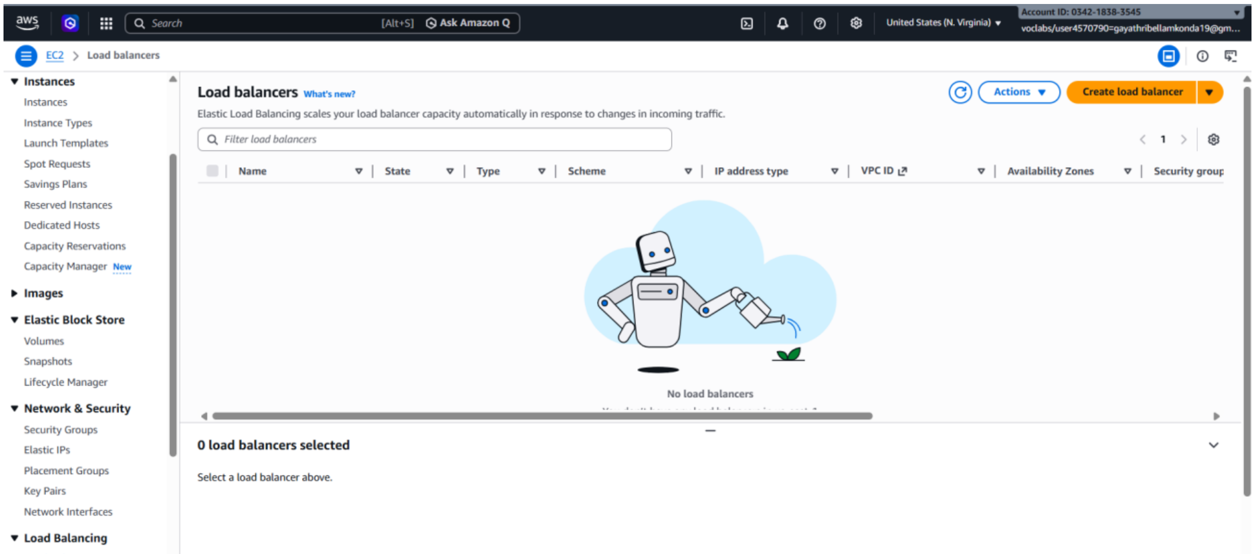
click on add rule of inbound rule -> Allow **HTTP (80)** from anywhere

Leave the outbound rules as default

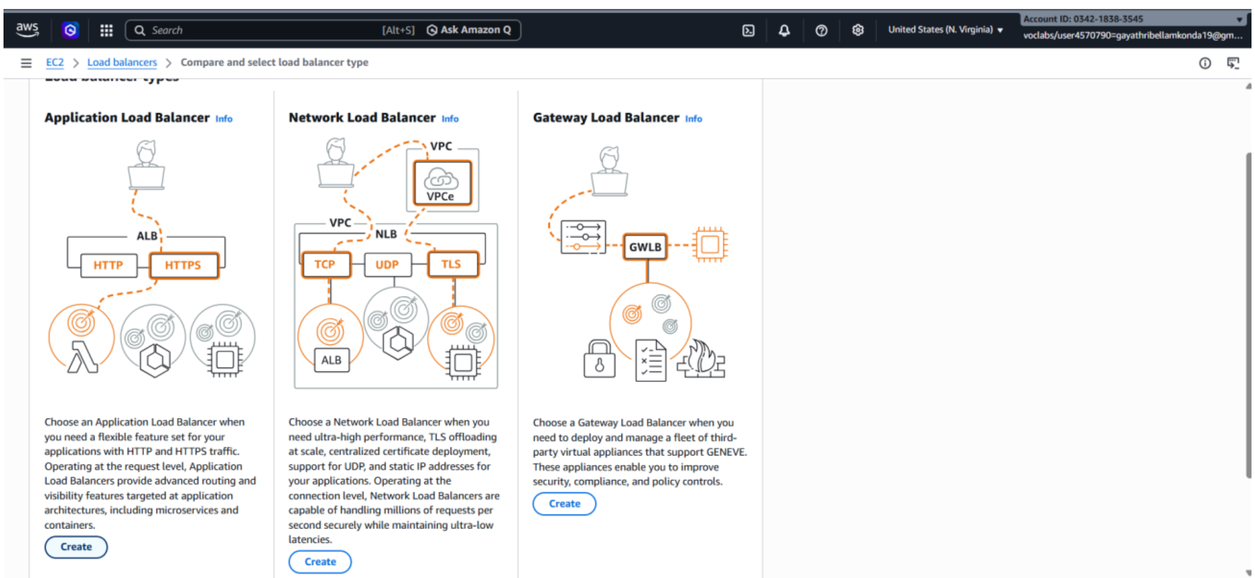
Click on create security group



Go to **EC2 → Load Balancers → Click on Create Load Balancer**

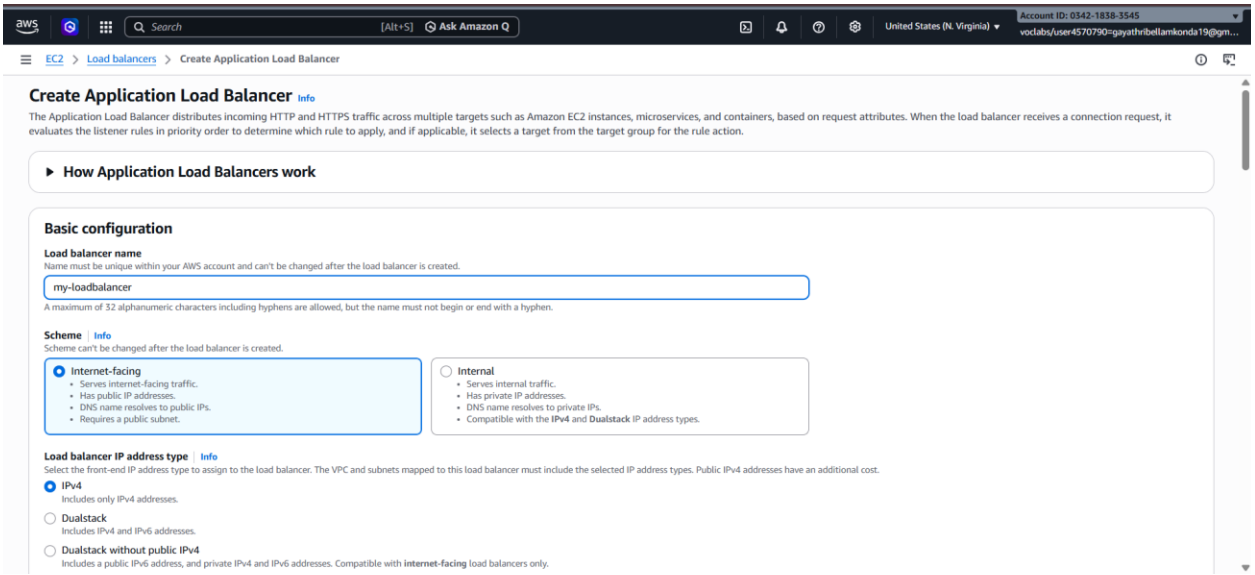


Click on create of **Application Load Balancer**



Name: my-loadbalancer -> Scheme: Internet-facing

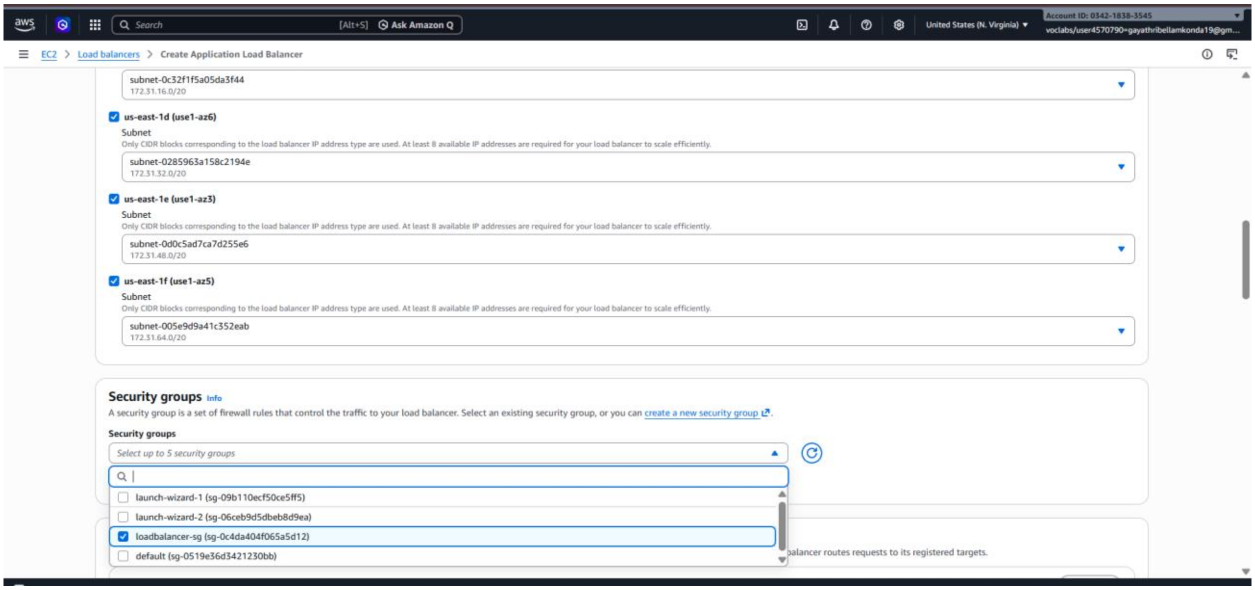
IP address type: IPv4



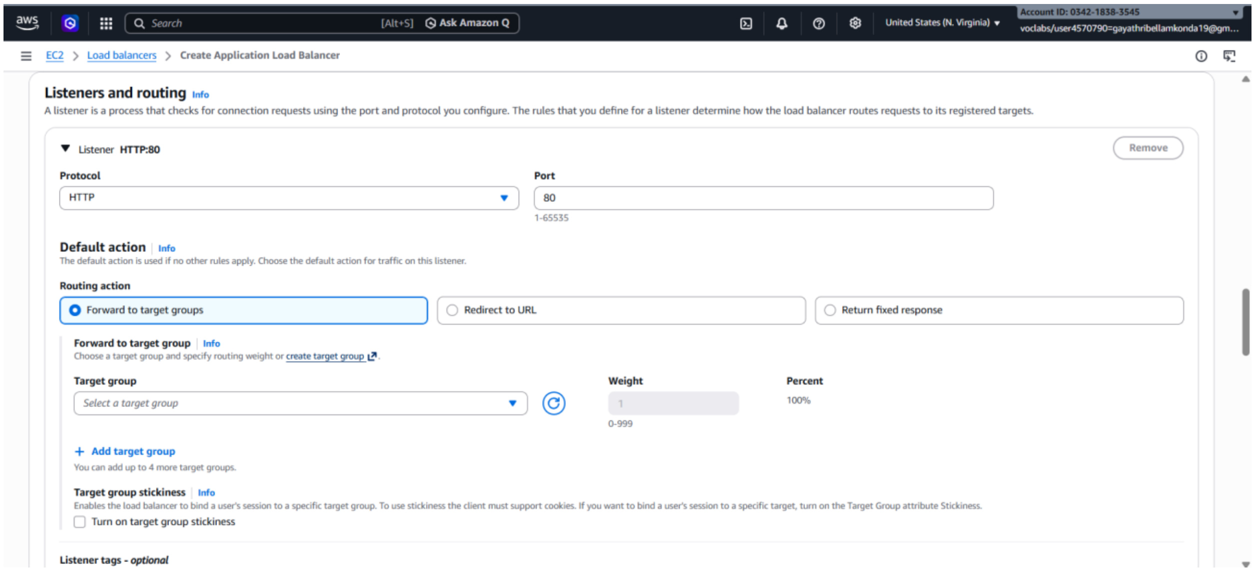
Select the deafault **VPC** where your EC2 instances exist

Select **all subnets** (one for each instance if possible

Choose the **loadbalancer-sg** security group created earlier

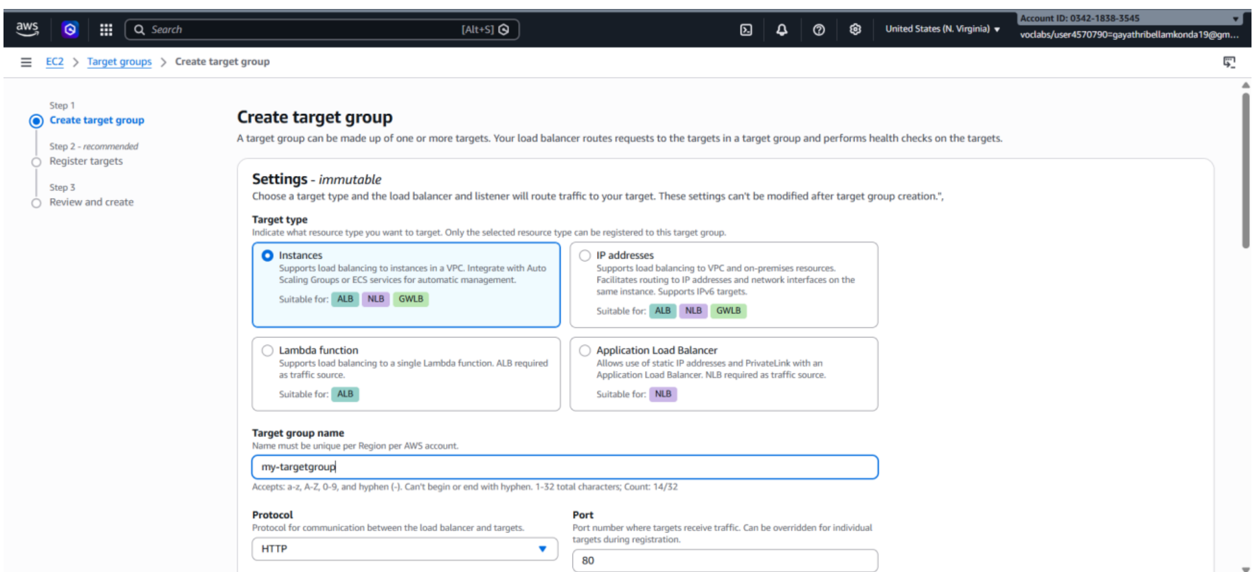


Click on create target group

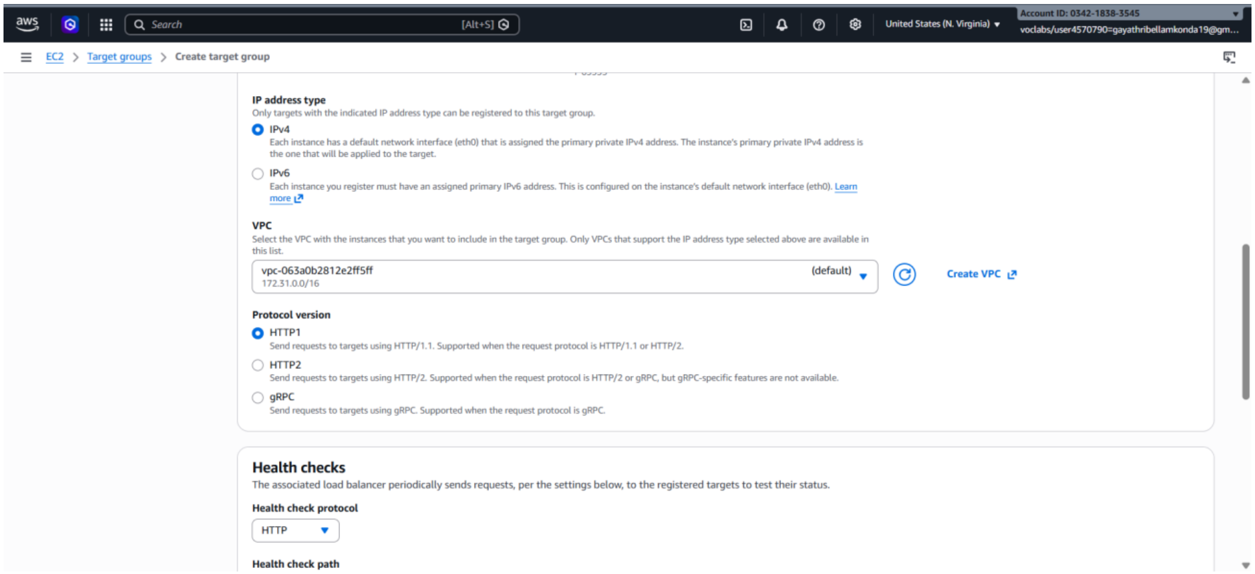


Name: my-targetgroup -> Target type: Instance

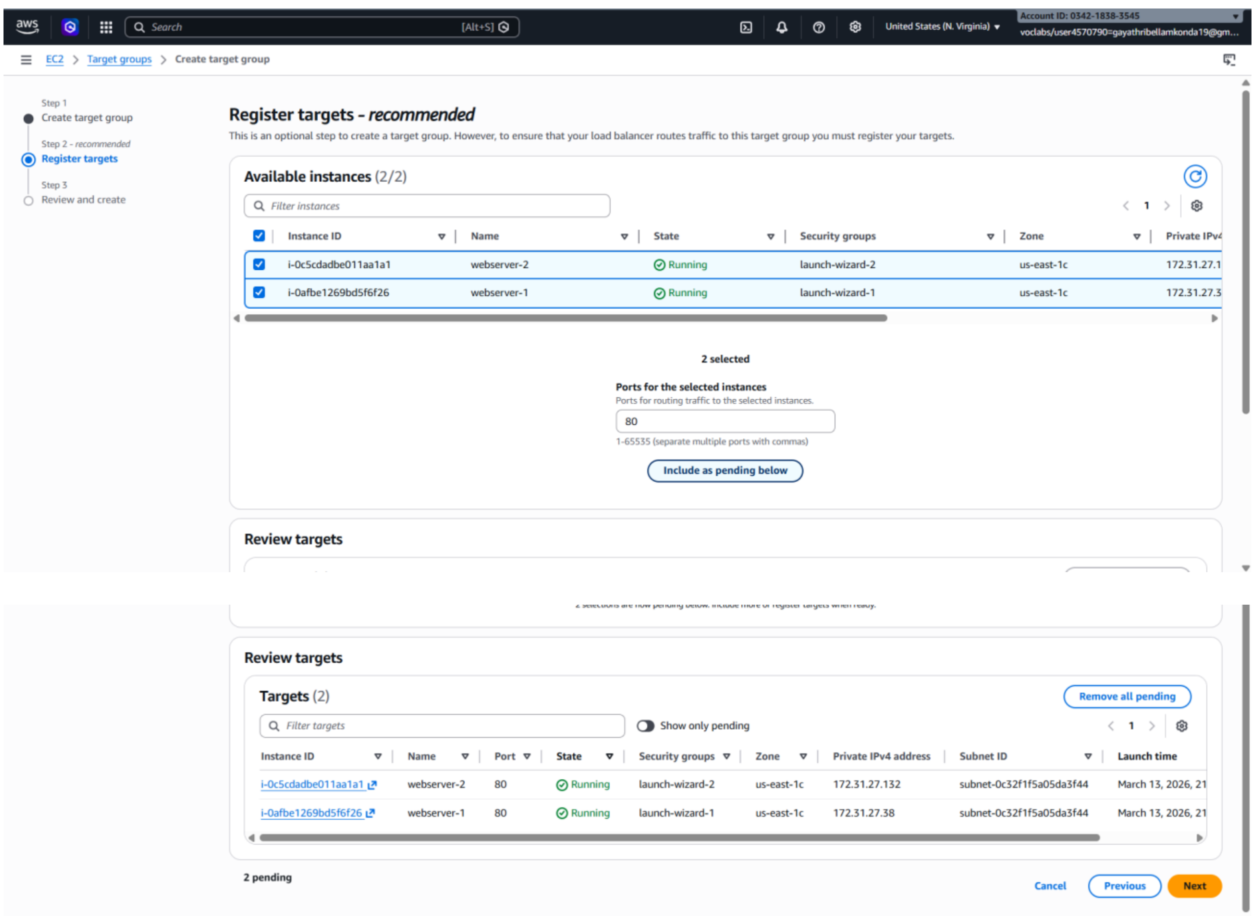
Protocol: HTTP -> Port: 80



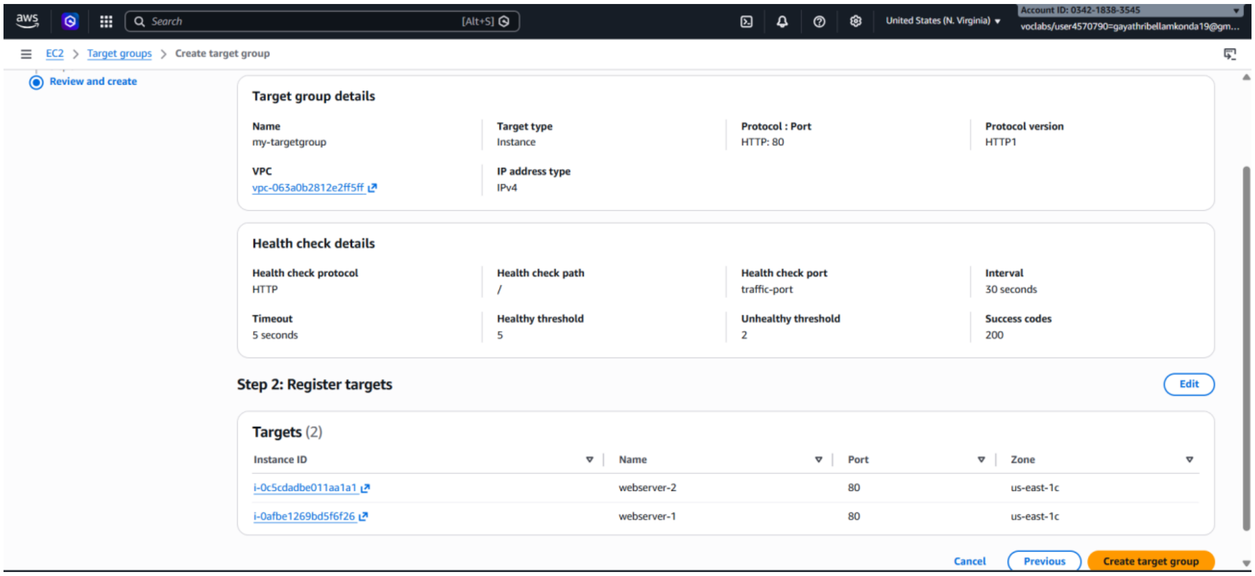
Select ipv4 -> select the default vpc -> click on next



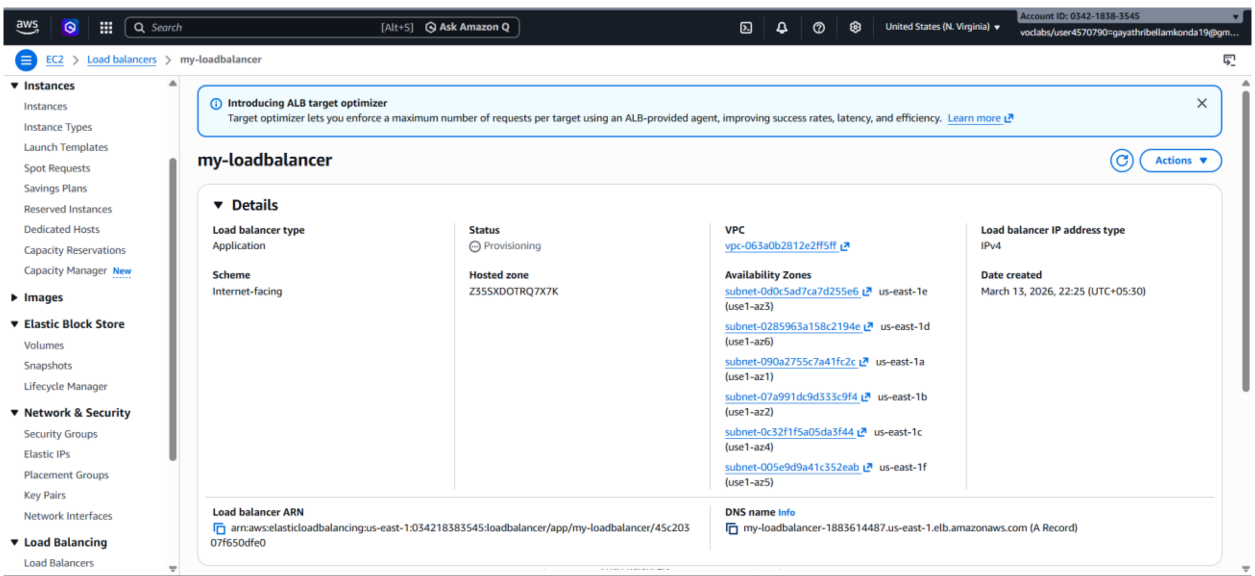
Select all the instances -> Click **Include as pending** → **Click on next**

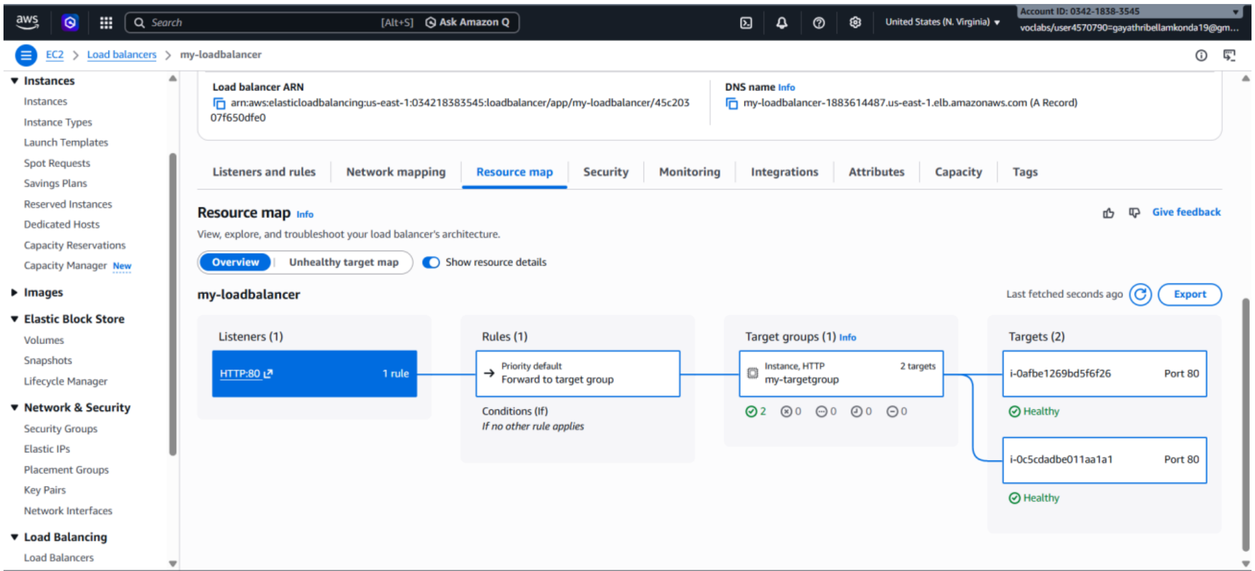


Review all settings → Click **Create target group**



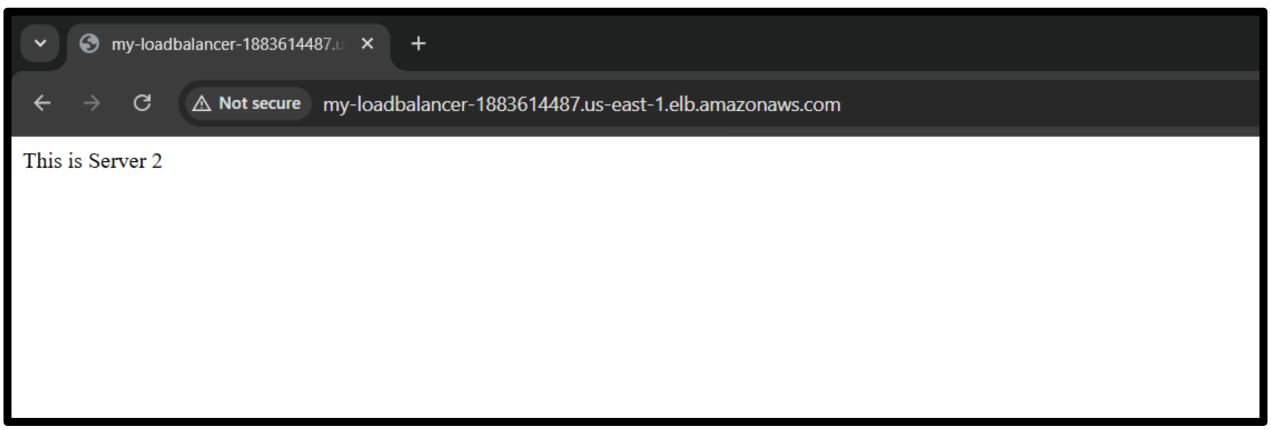
Go to **Load Balancers → Select the loadbalancer → Description → copy the DNS name**





Paste it in a browser → You should see either **WebServer-1** or **WebServer-2** page.

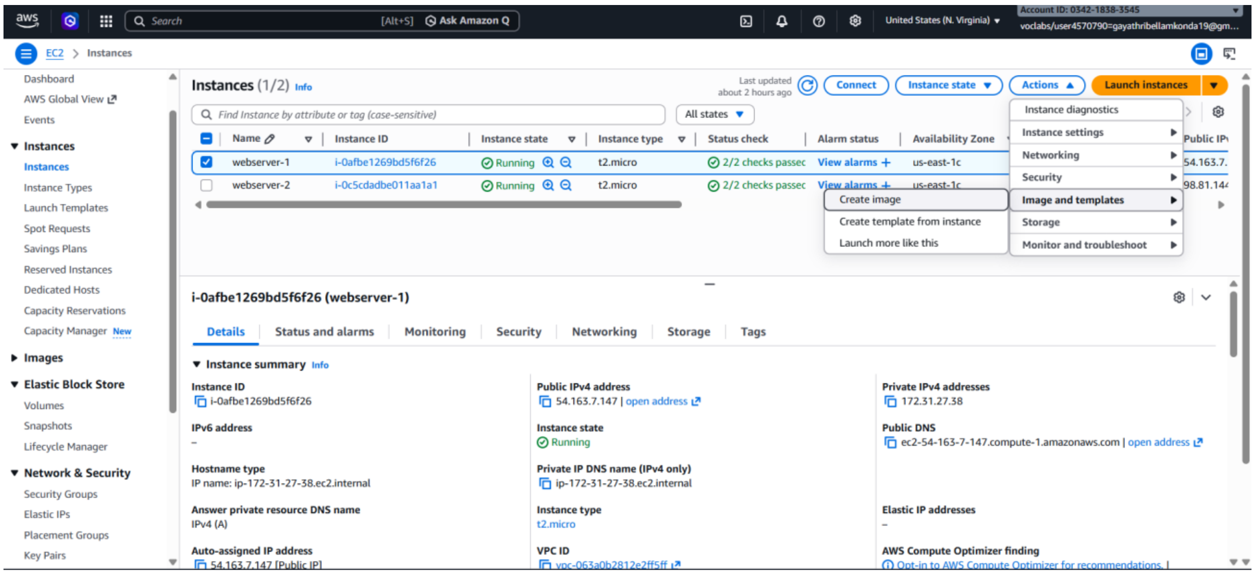
Refresh multiple times → Load balancer distributes traffic between the two instances



Auto Scaling Group with Load Balancer

Go to **EC2 Console → Instances ->** Select the EC2 instance

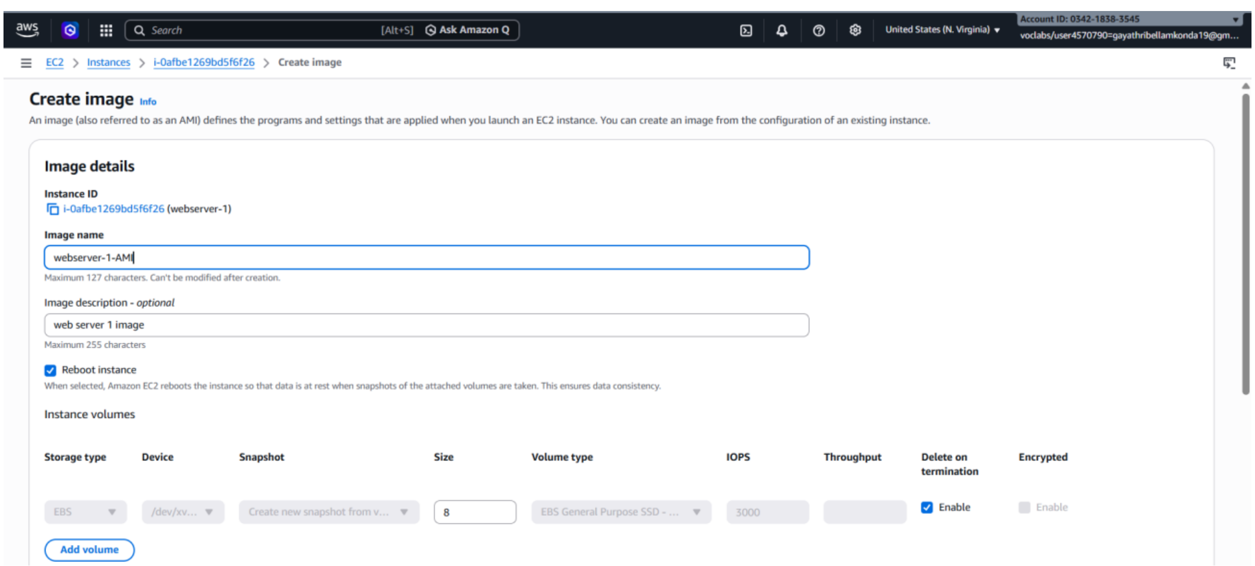
Click **Actions → Image and Templates → Create Image**



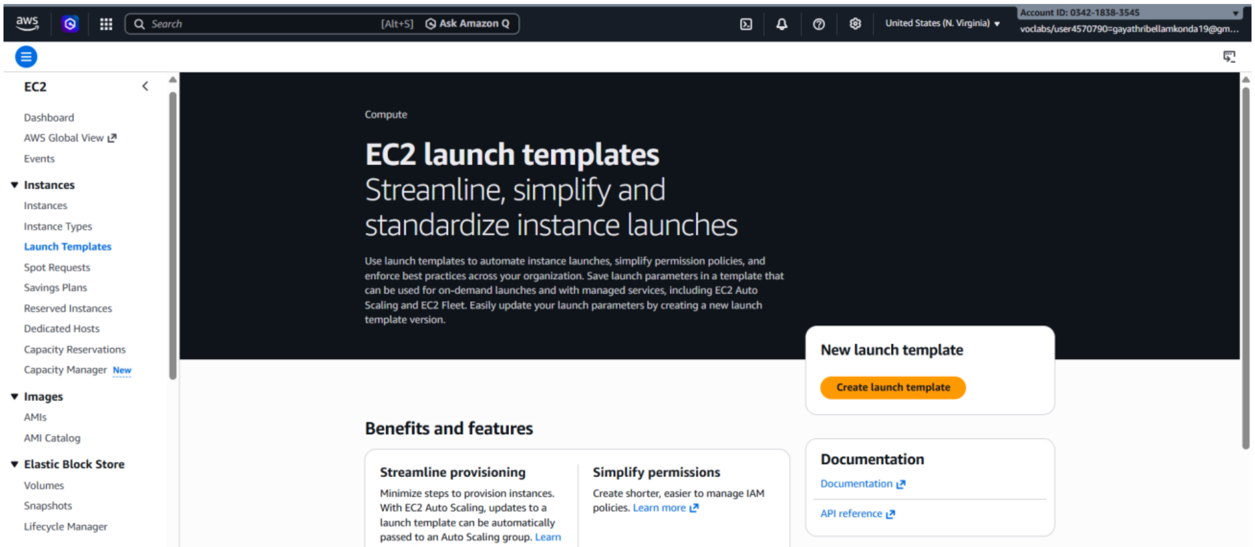
Provide Image Name: e.g., my-webserver-AMI

Optional: Add description

Click **Create Image**

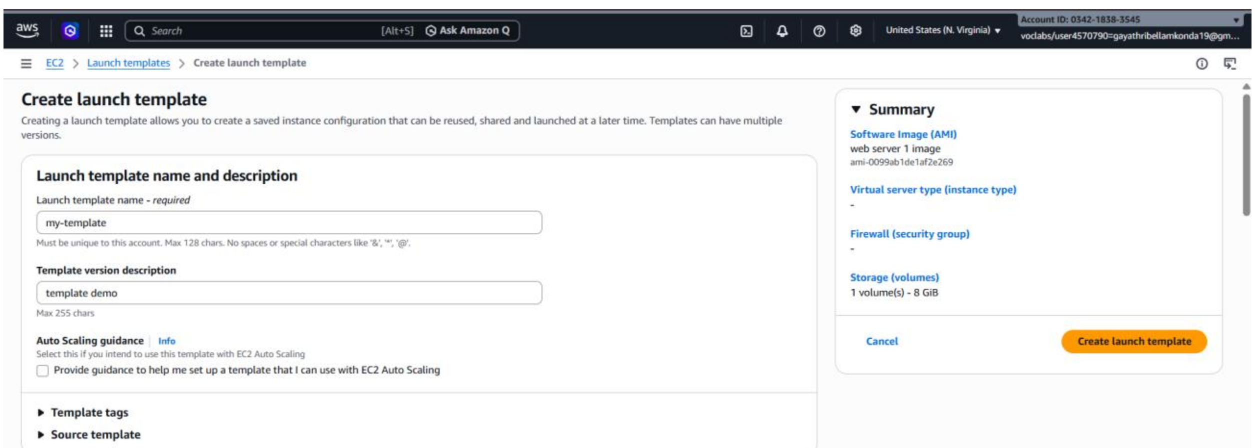


Go to **EC2 → instances->Launch Templates → Create Launch Template**



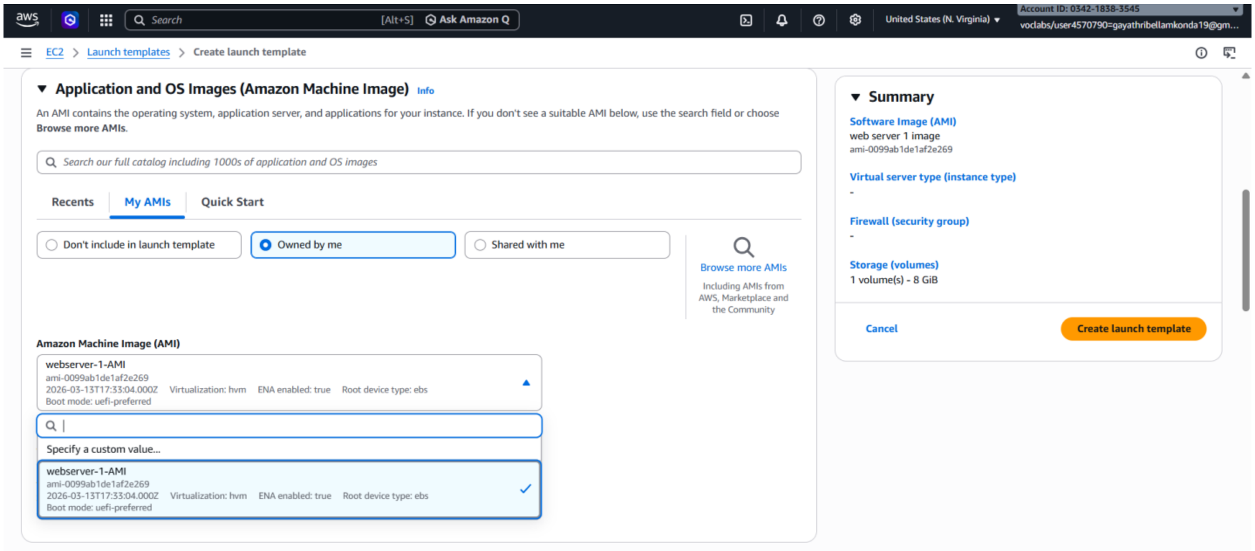
Launch Template Name: my –template

Add description



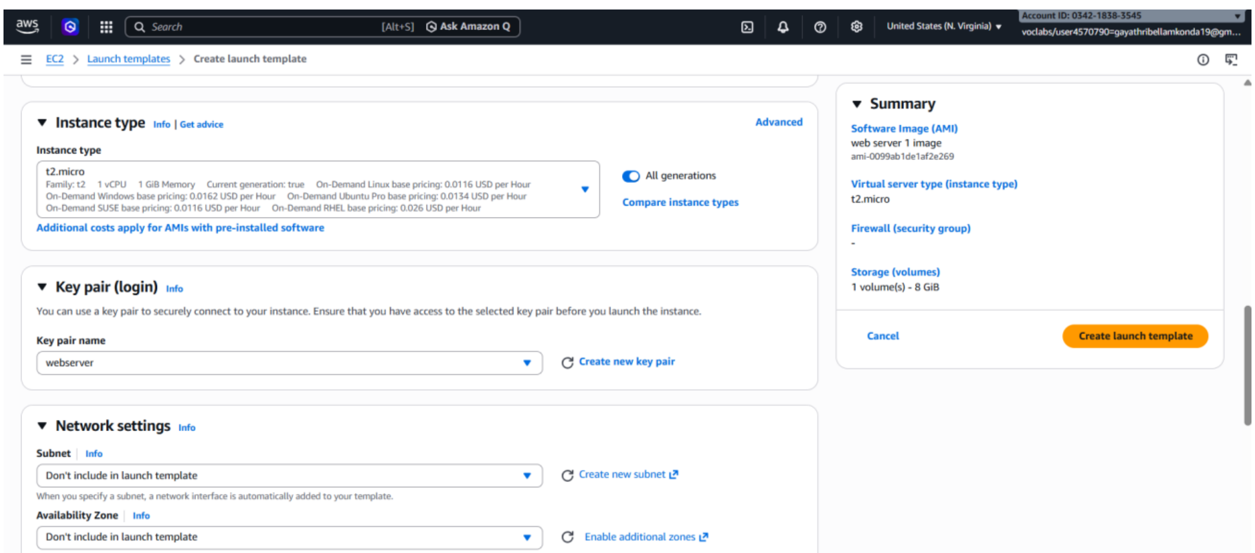
Select My AMIs -> select owned by me

AMI: Select the AMI created in Step 1 (webserver-1-AMI)



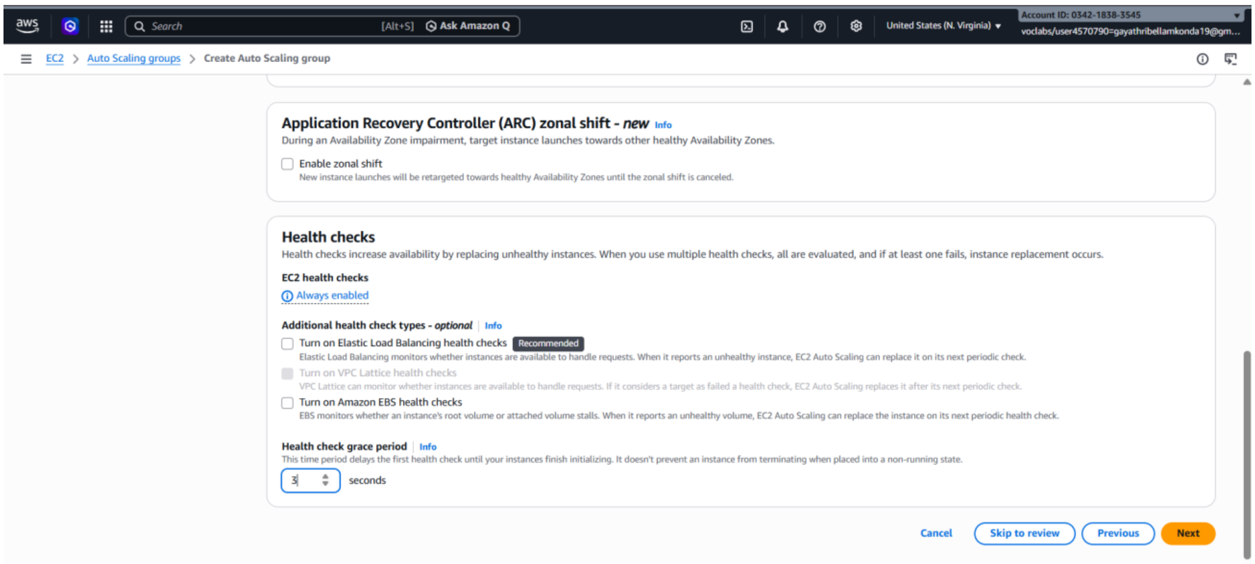
Instance Type: t2.micro

Key Pair: Select your existing key pair



Set **Health Check Grace Period**: 3 seconds

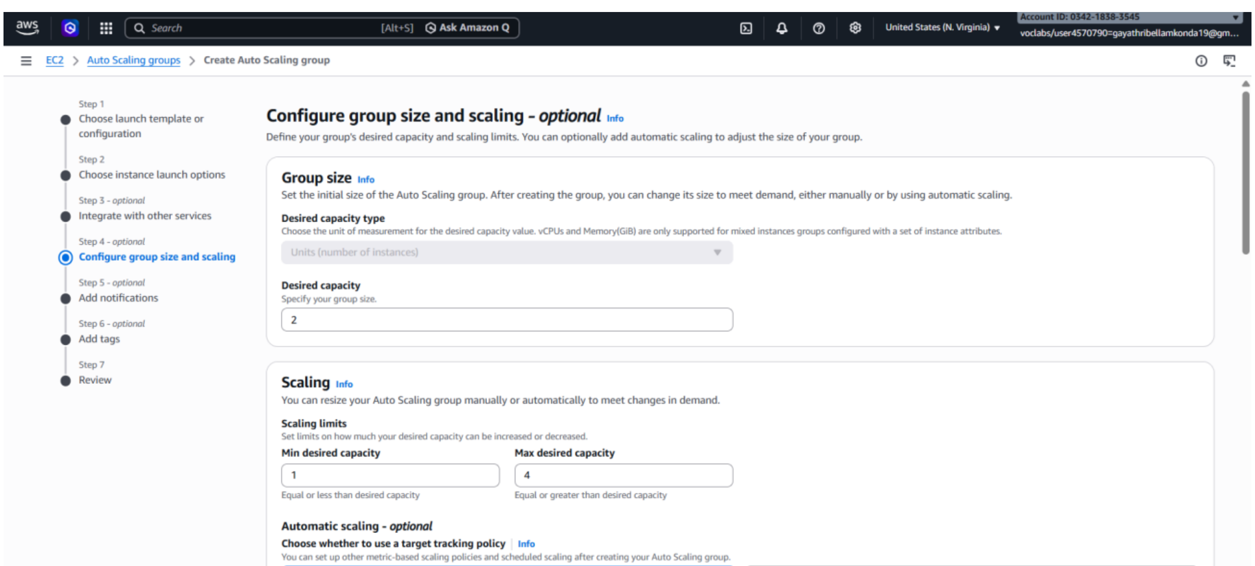
Click **Next**



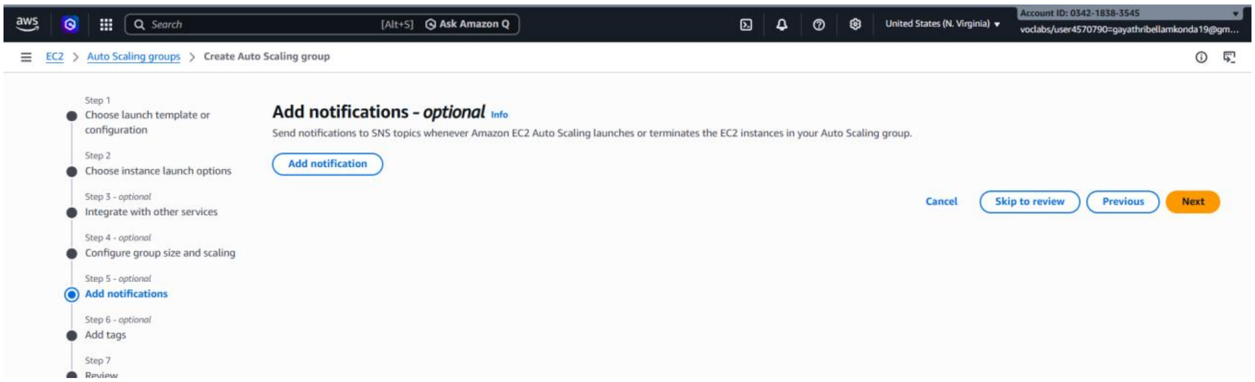
Desired Capacity: 2 (initial instances)

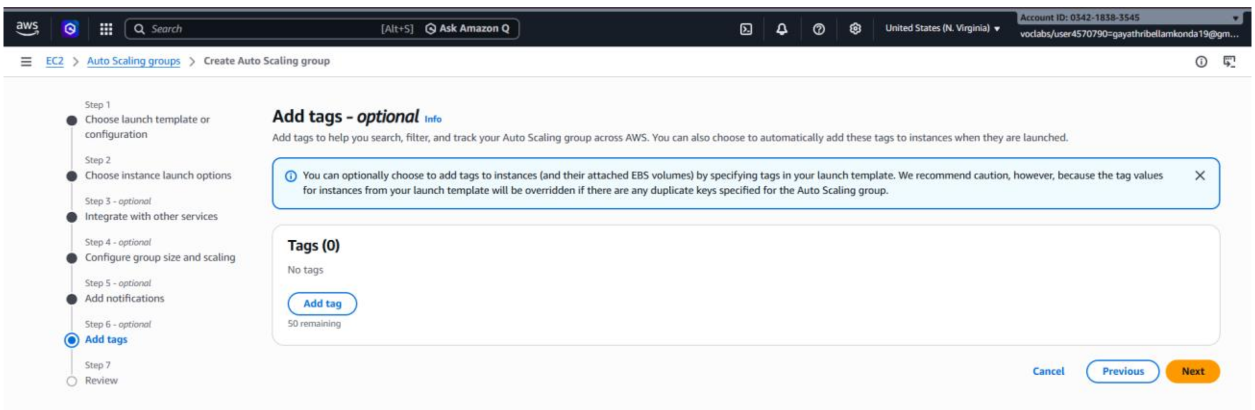
Minimum Capacity: 1 -> **Maximum Capacity**: 4

Click **Next**



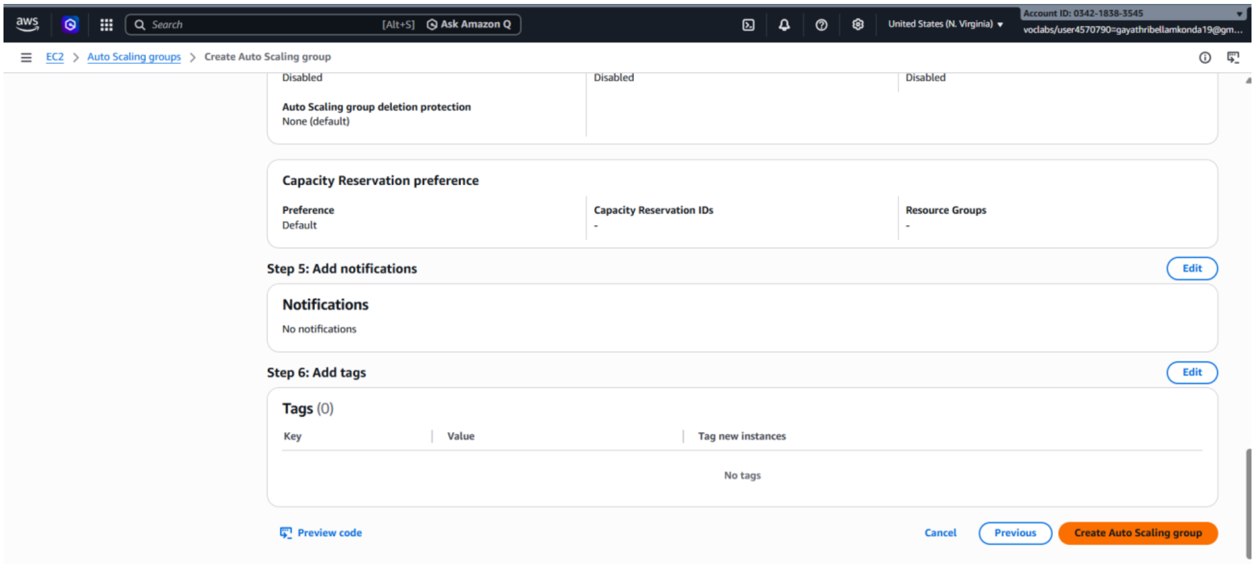
Leave notifications and tags as default -> click next





Review all settings

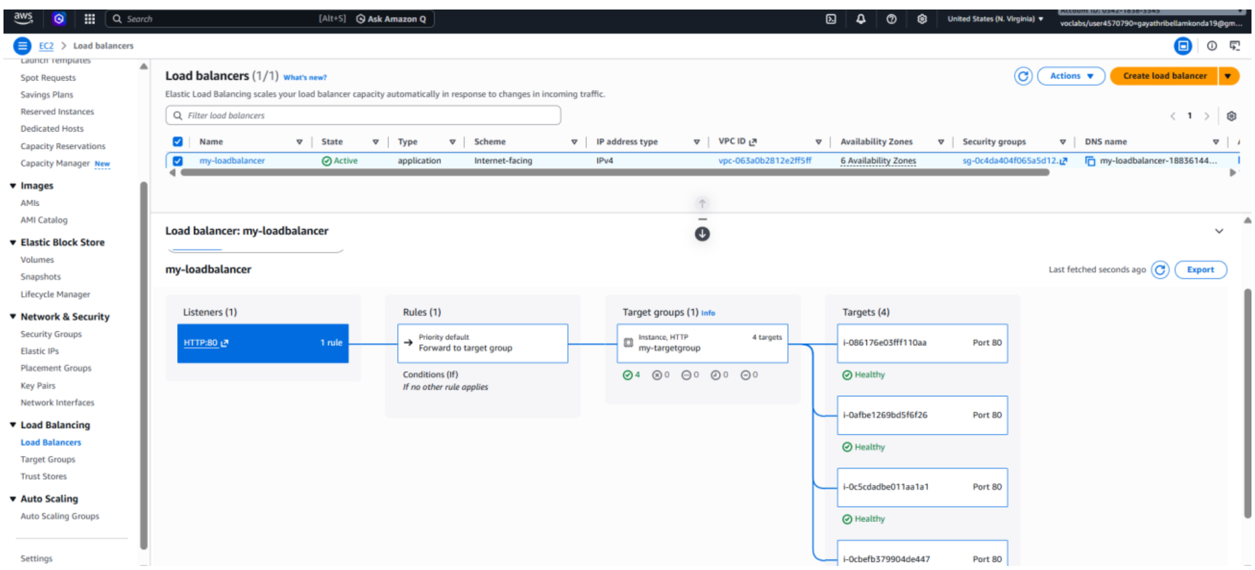
Click **Create Auto Scaling Group**



Go to **Load Balancers → Select your Load Balancer**

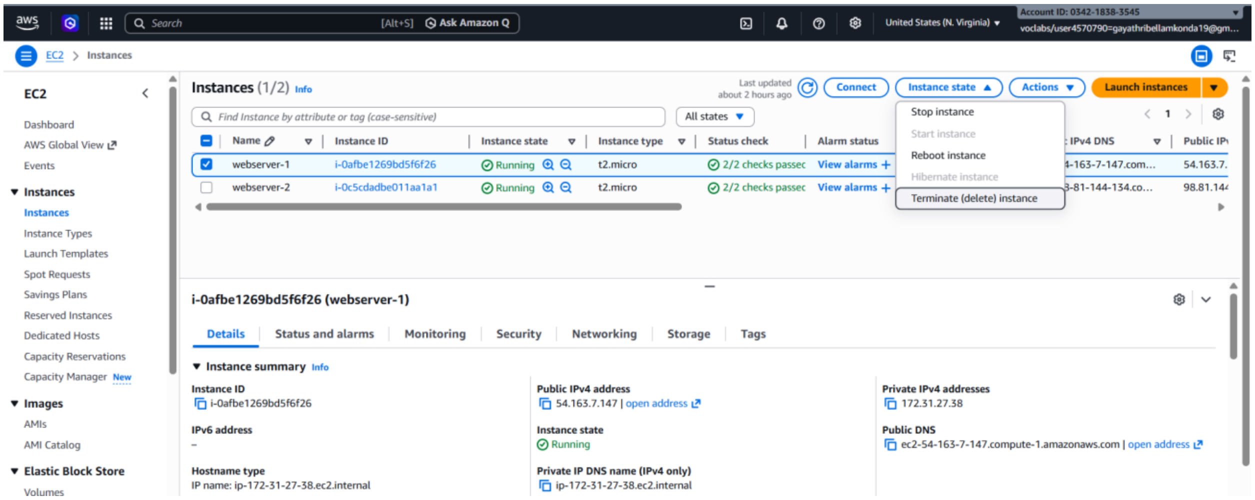
Check **Auto Scaling Group → Instances**

- You may see up to **4 instances** if scaling triggers

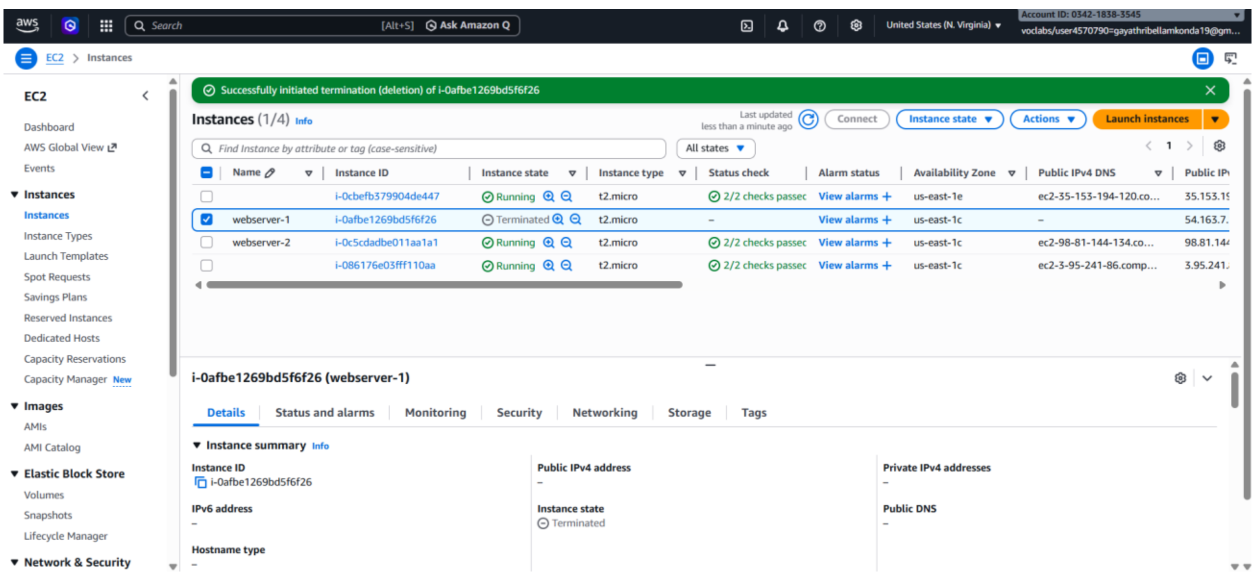


Test **auto recovery:**

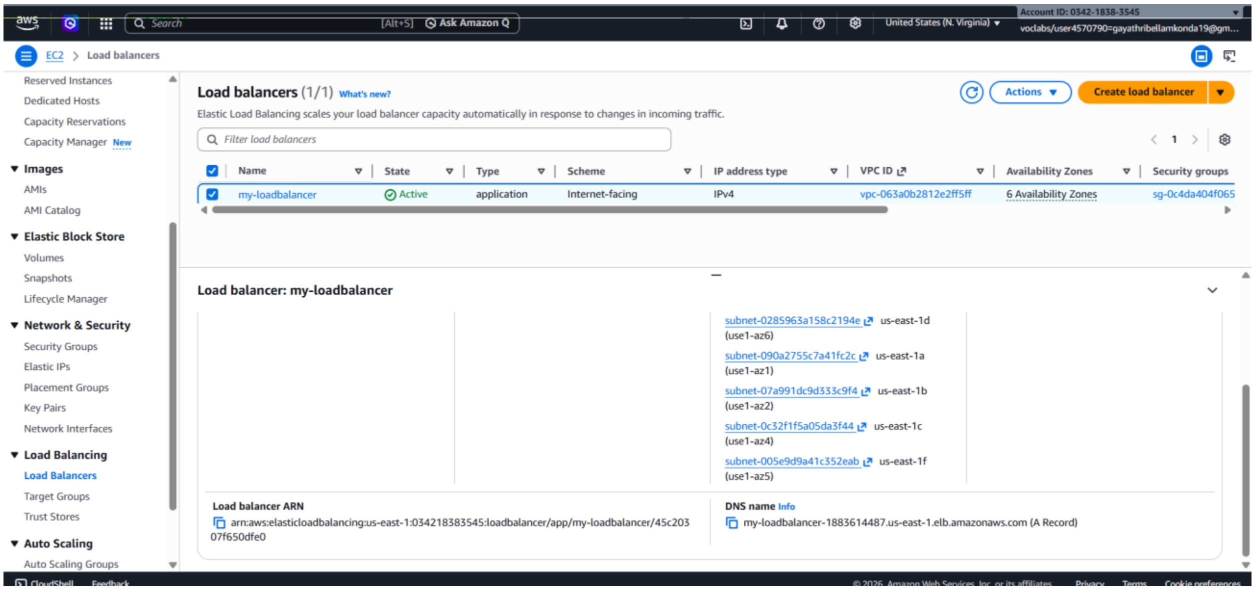
Select the instance -> Terminate the instance



2 instances will be created automatically by the auto scaling

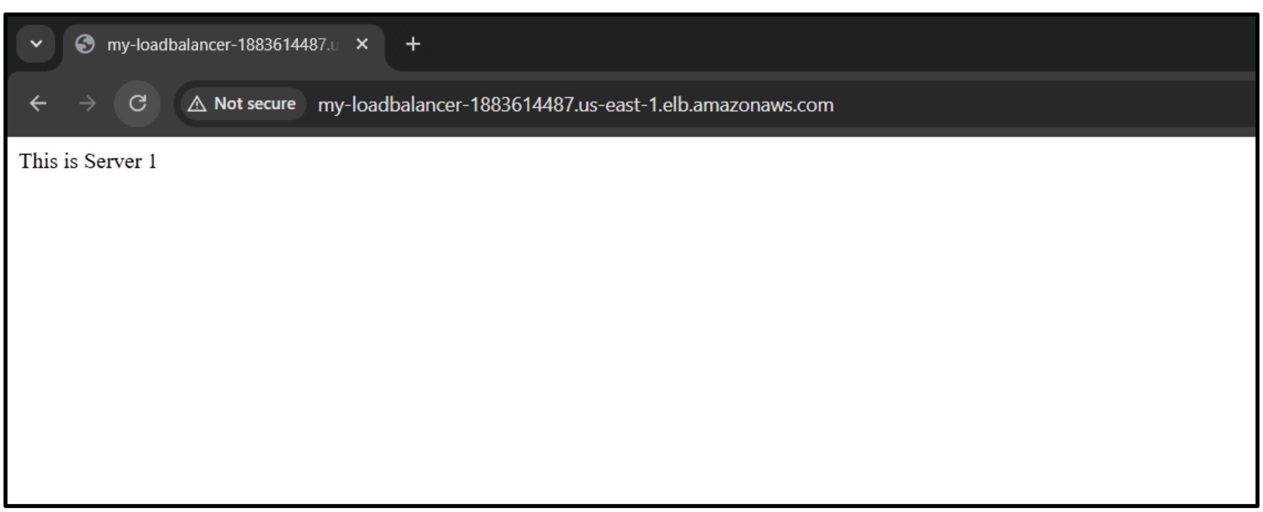


Select the load balancer -> copy the DNS name



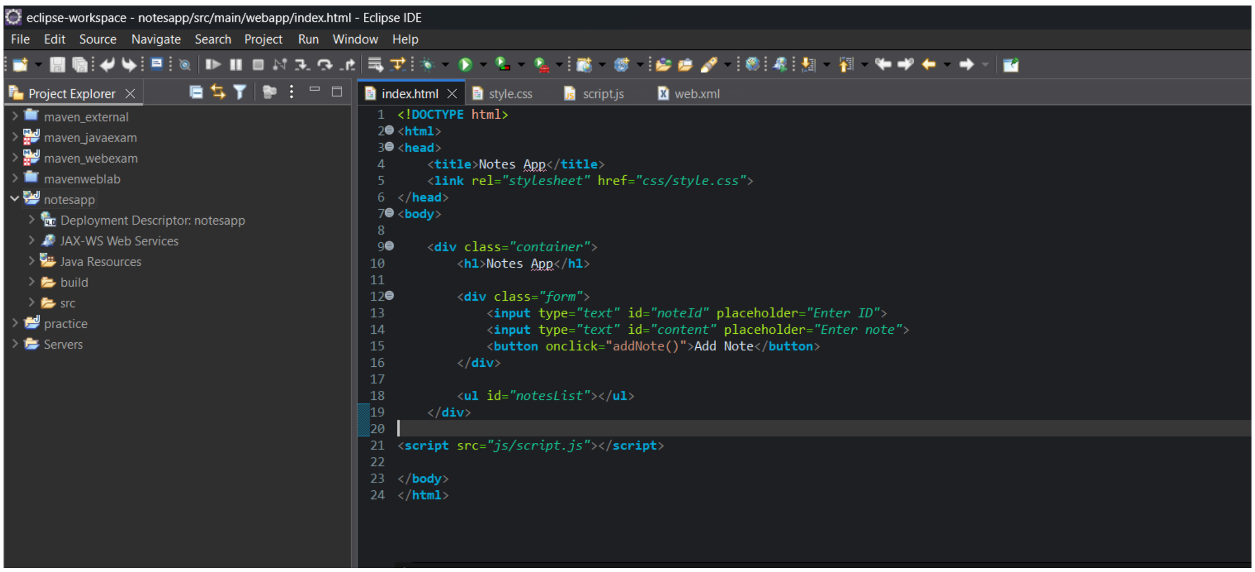
Paste the DNS in the incognito mode

You should see your web page from one of the instances -> Refresh multiple times → traffic alternates between instances

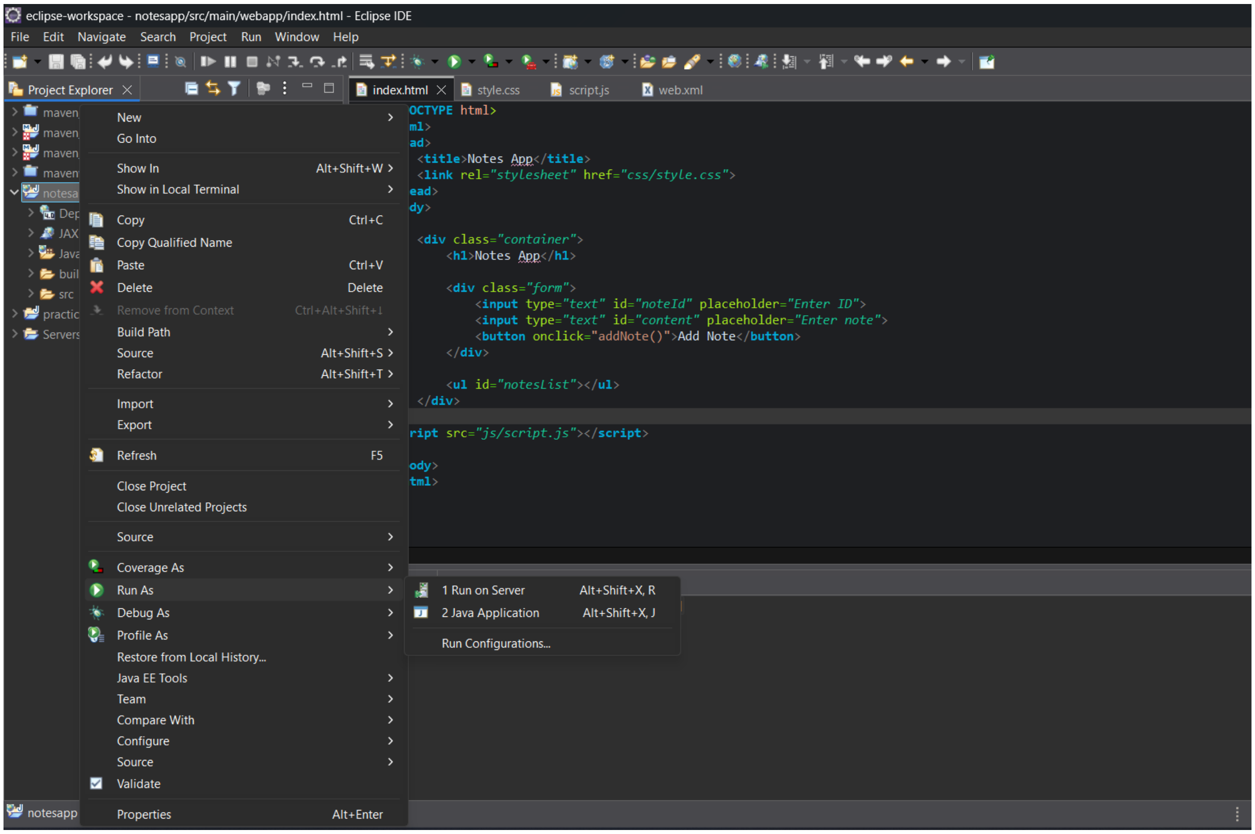


WEEK -10 : ELASTIC BEANSTALK

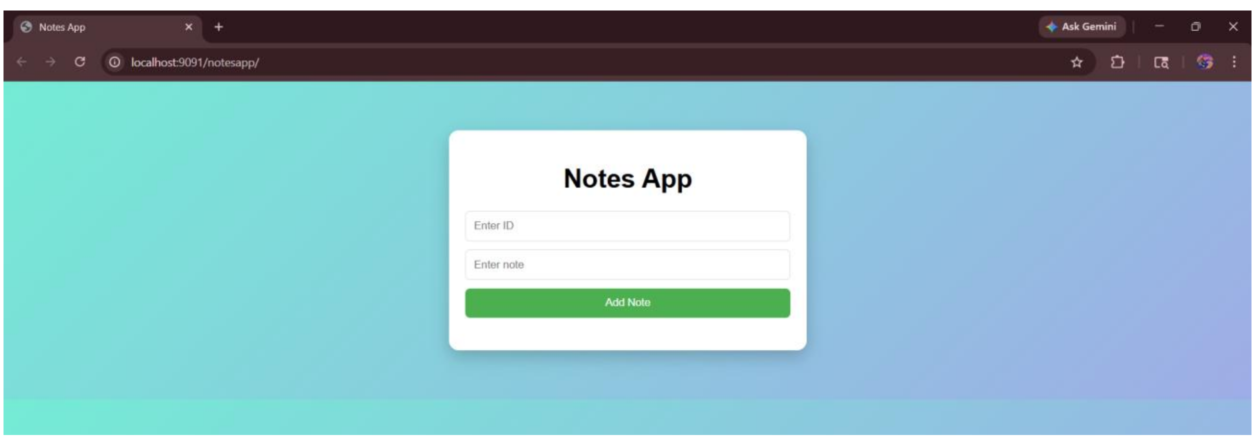
- Open Eclipse
- Go to: **File → New → Dynamic Web Project**
    - Project Name: `NotesApp`
    - Target Runtime: (Select Apache Tomcat)
- Click **Finish**



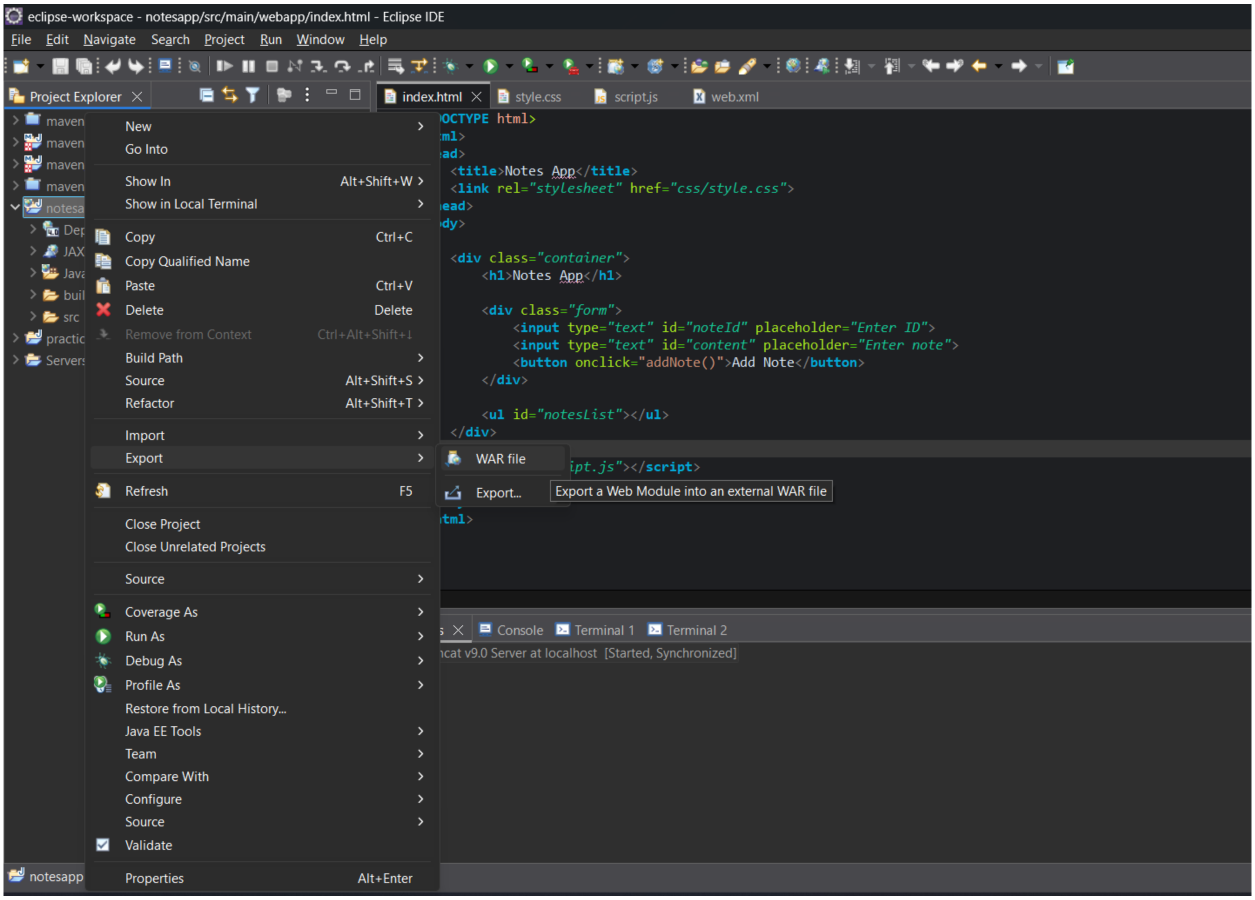
- Right-click project
- **Run As → Run on Server**
- Choose **Apache Tomcat**



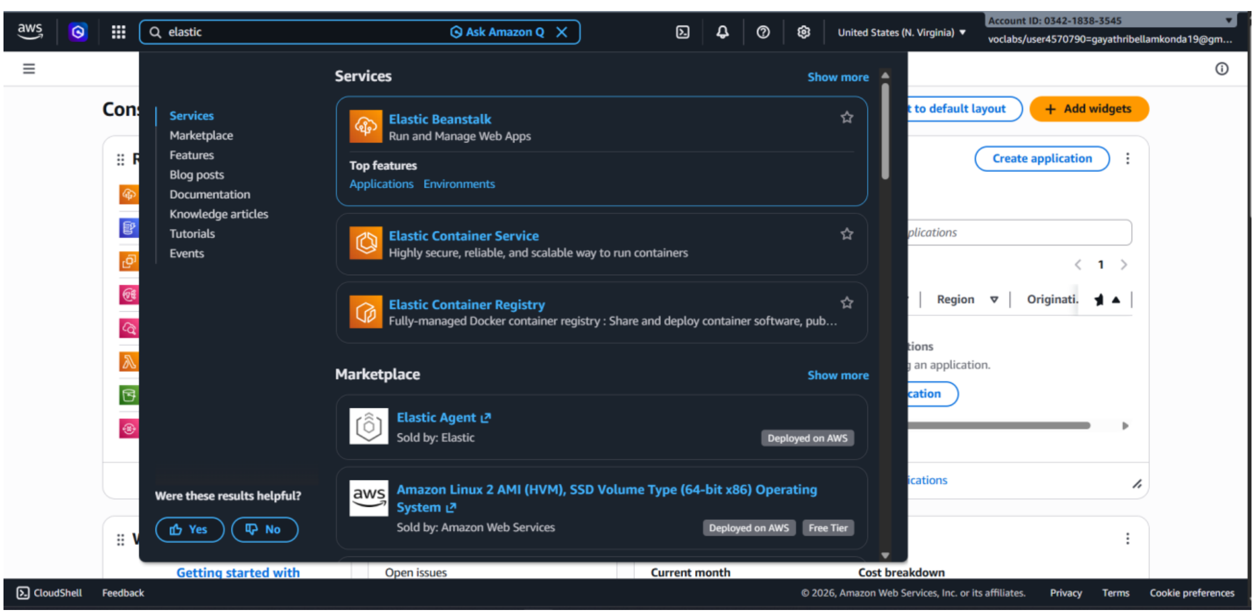
You should see your Notes App running locally.



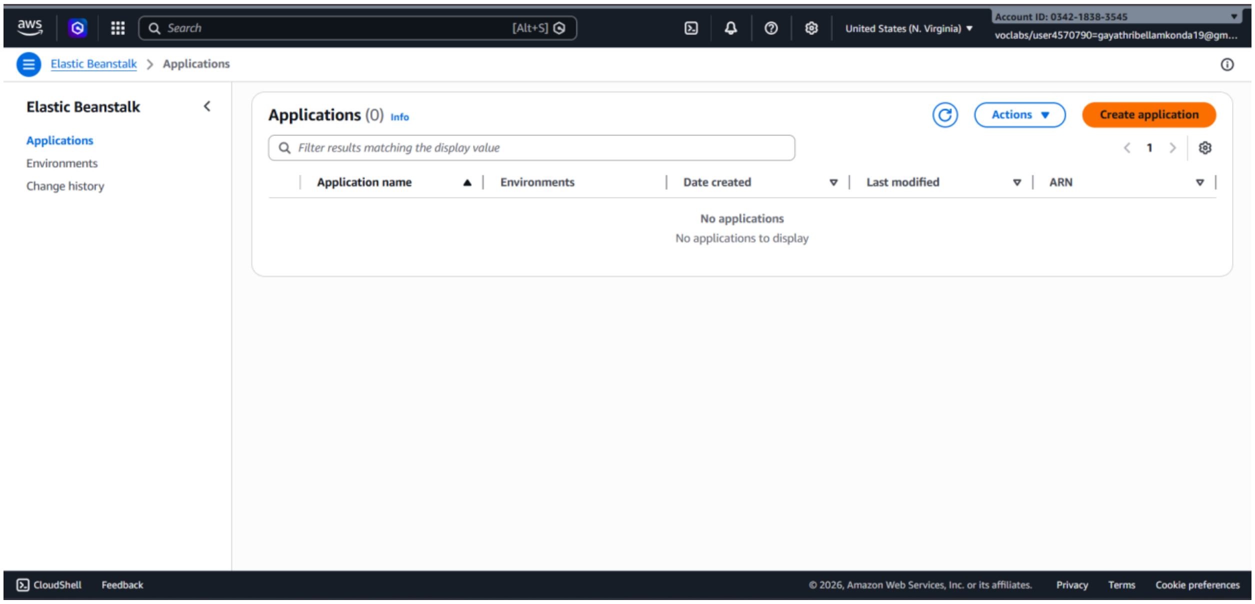
- Right-click project
- **Export → WAR file**
- Choose destination:



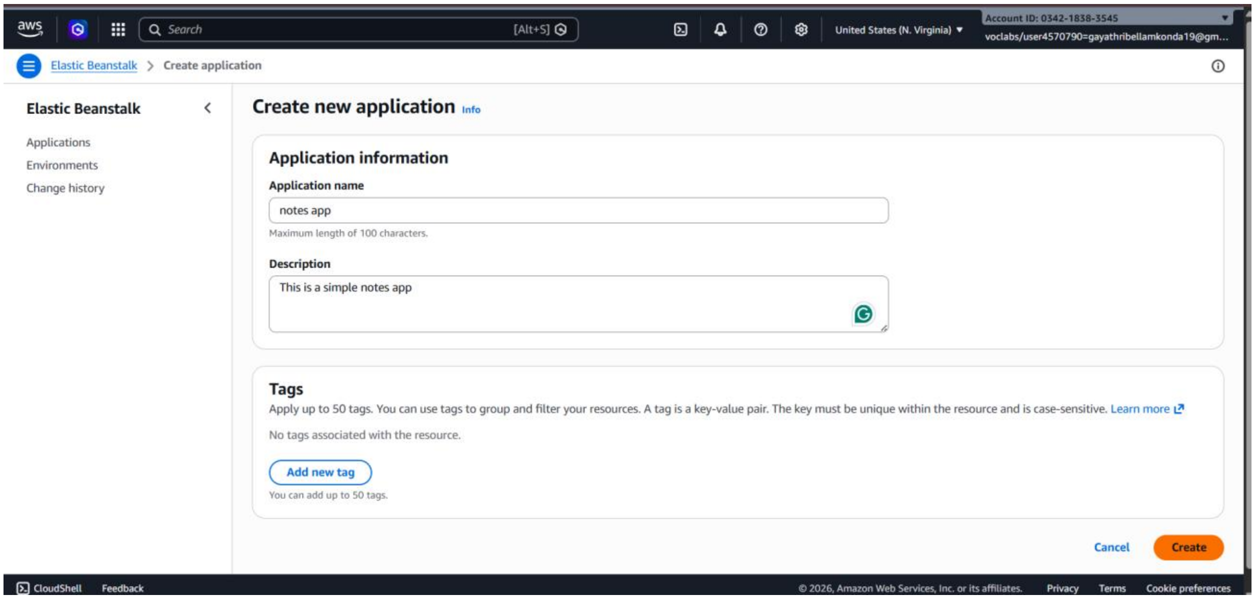
Open AWS console and search for Elastic Beanstalk



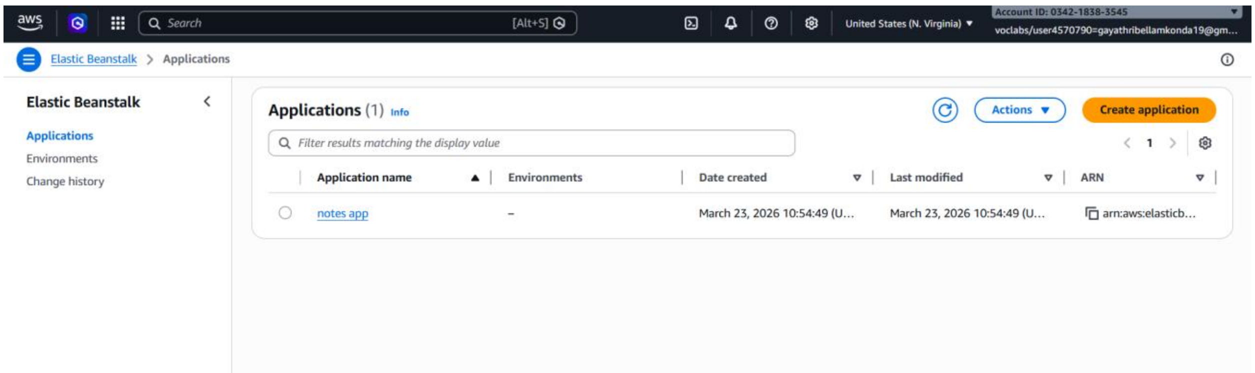
Click on create application



- Go to Elastic Beanstalk , Create a new application name it “Notes app”
- Add Description
- Click on create

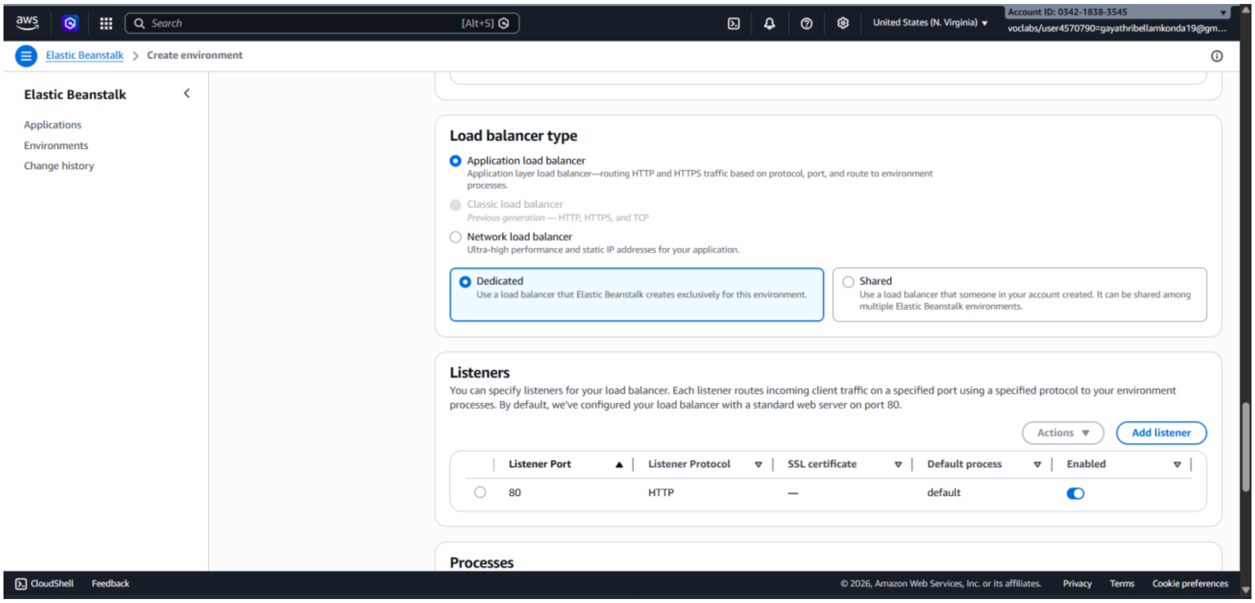


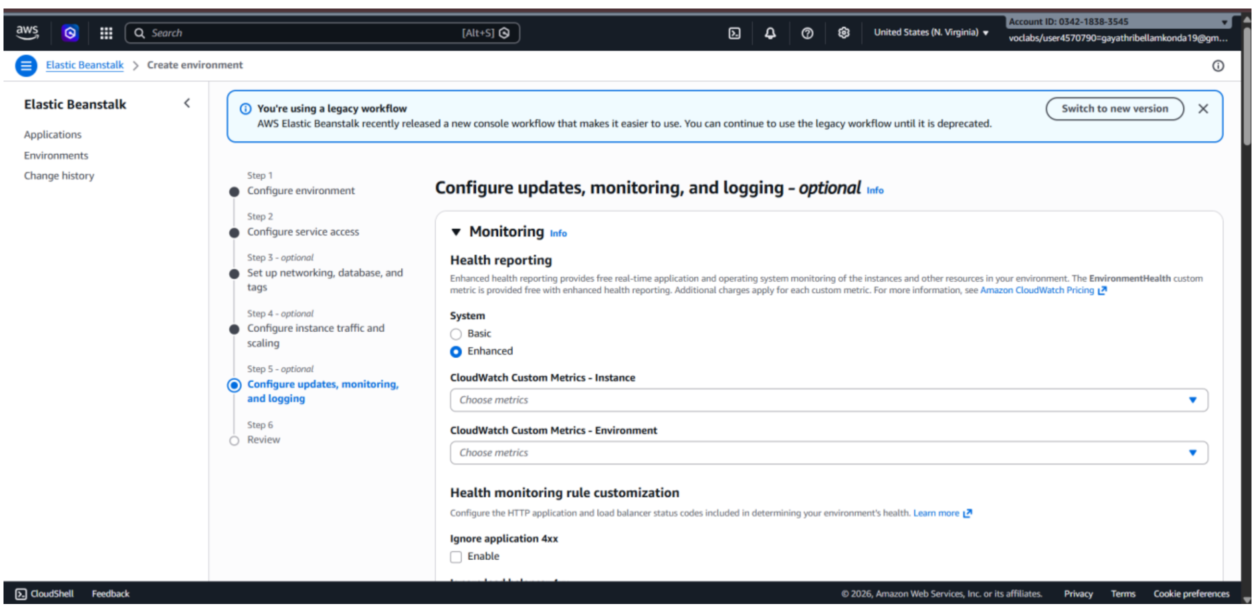
Application will be created



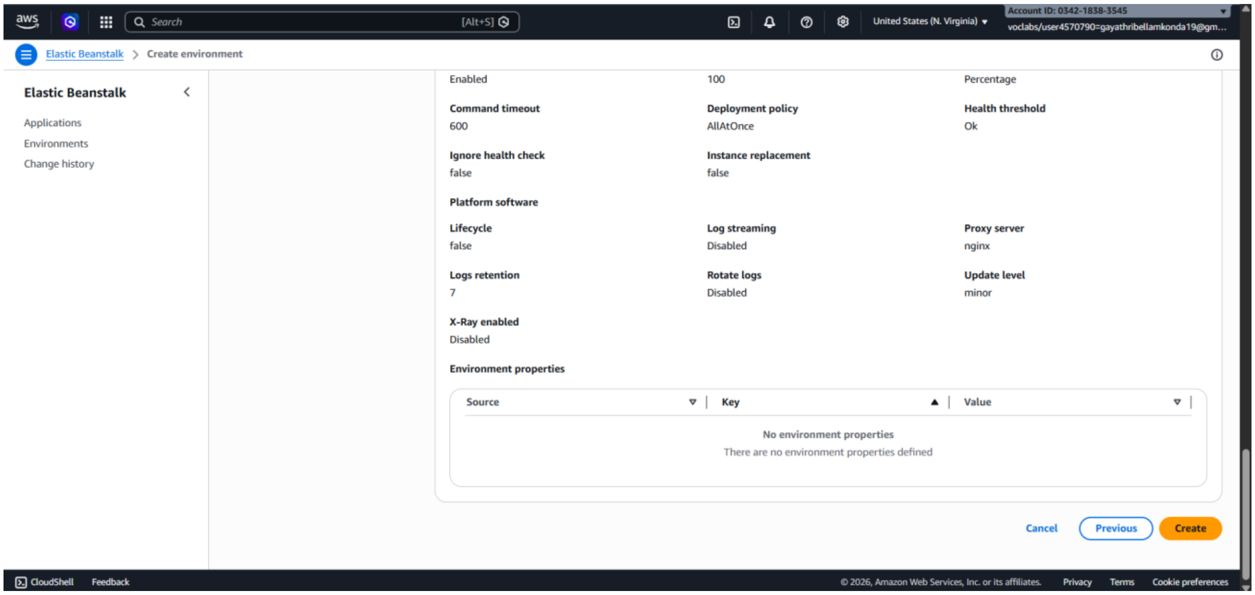
Click on the application -> click on create new environment

Keep every thing as default and click on next



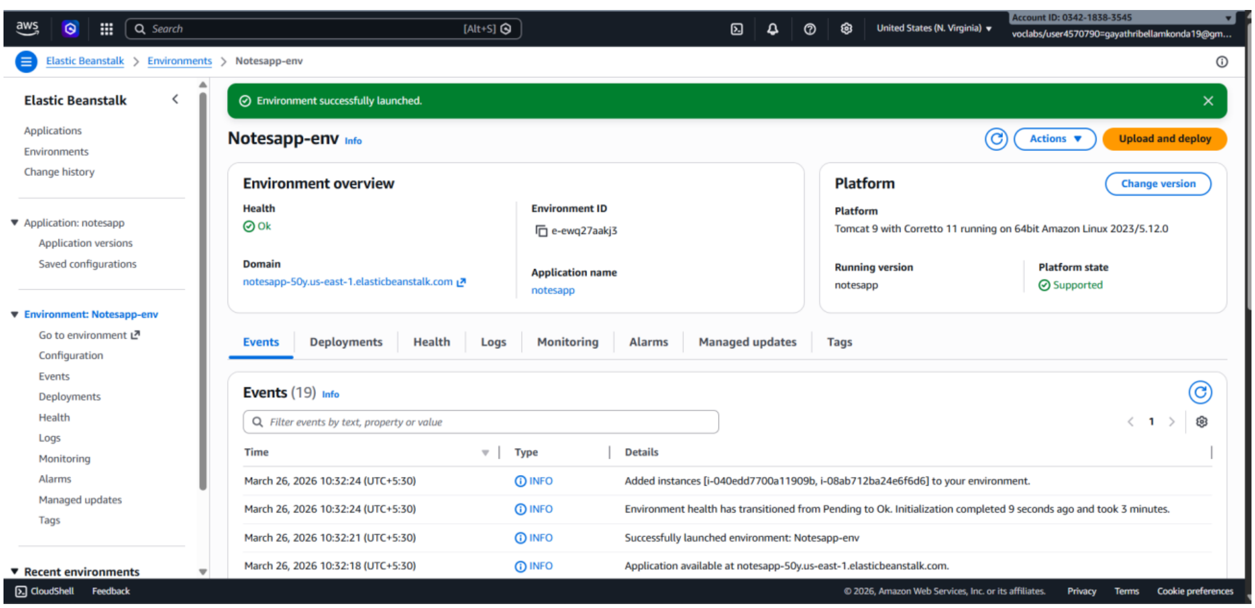


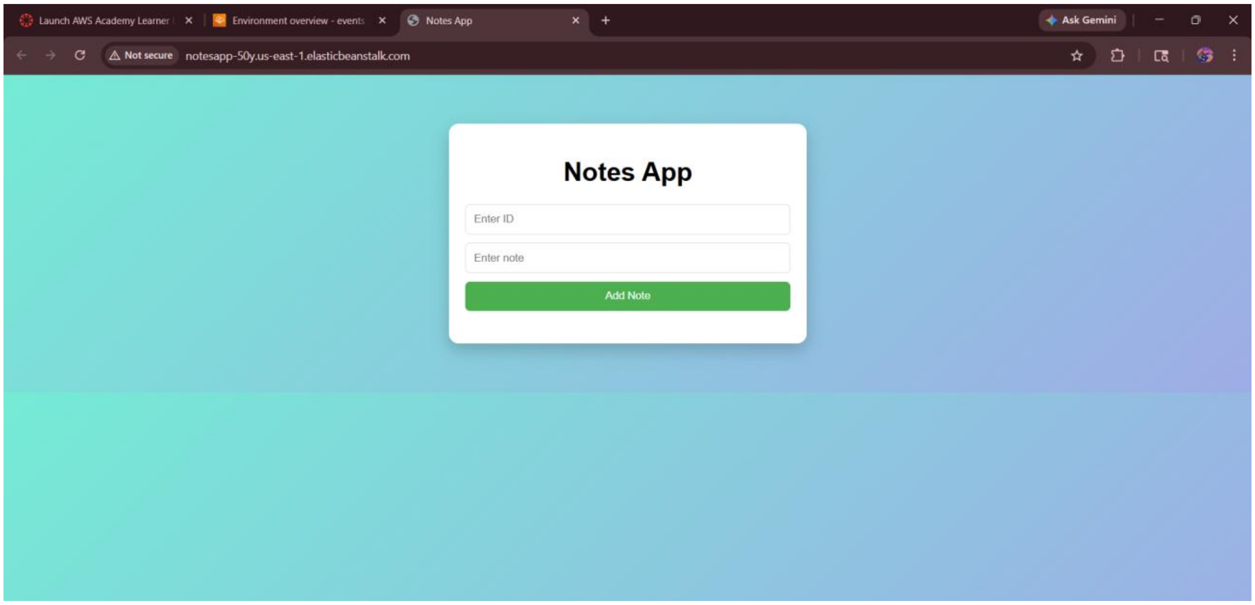
Review the environment and click on create



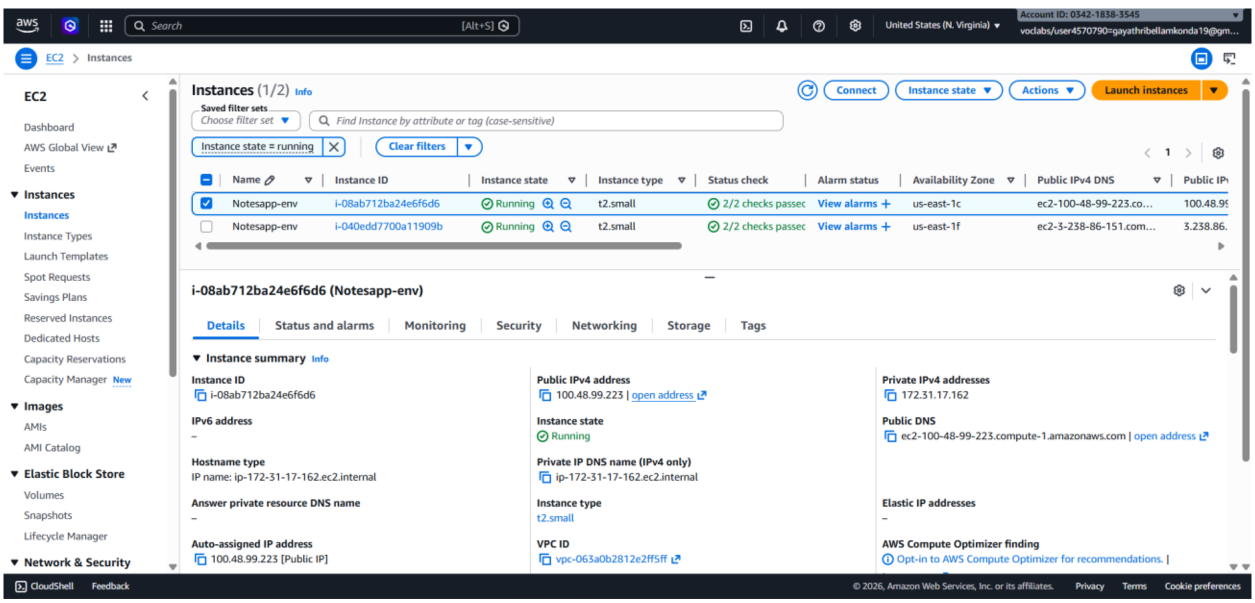
Environment will be successfully created after some time

Click on domain link it will navigate to the application output



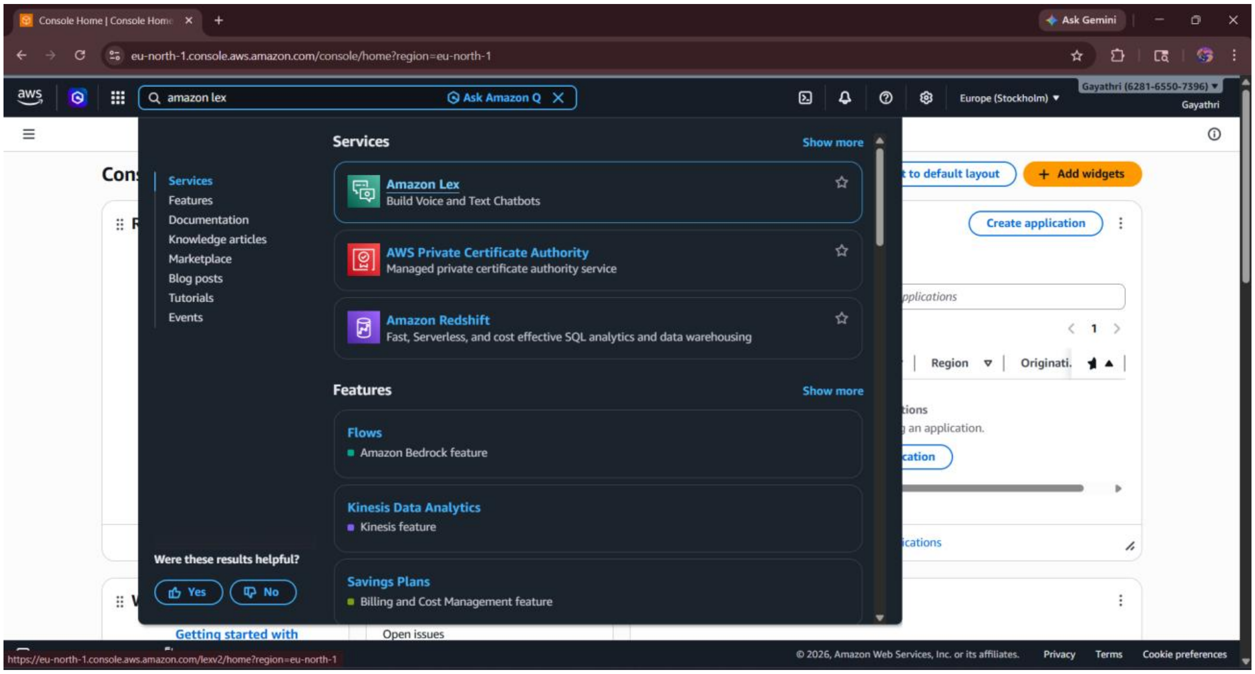


Instances will be automatically gets created

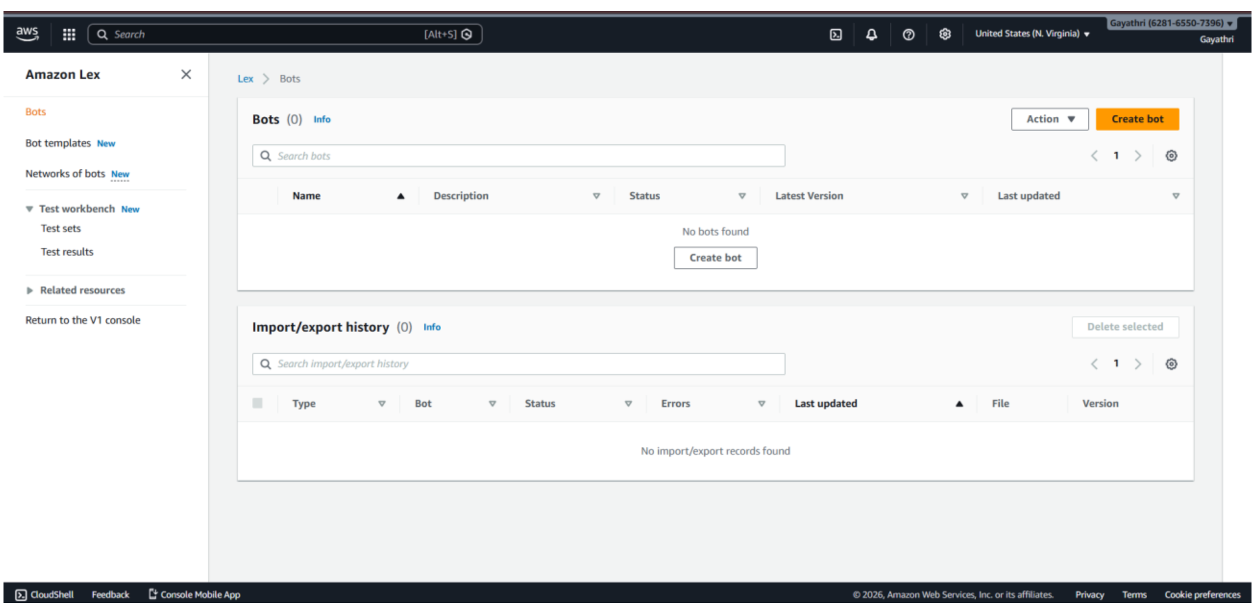


WEEK-11—Amazon LEX

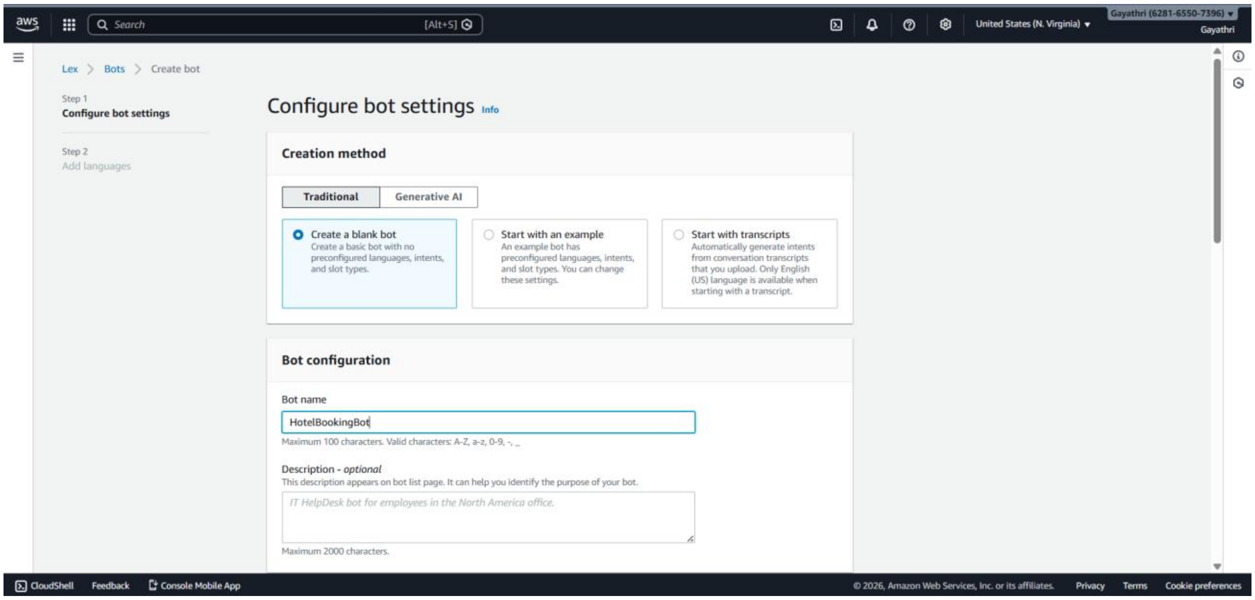
Login to AWS Console -> Search **Amazon Lex**



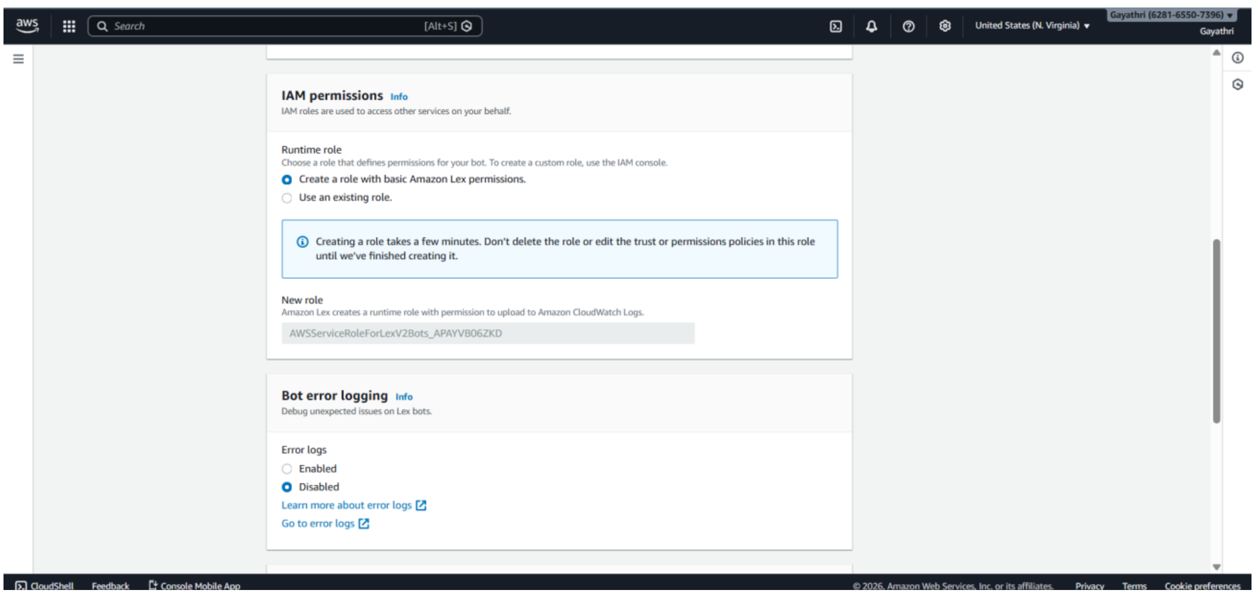
Choose: **Create a blank bot**



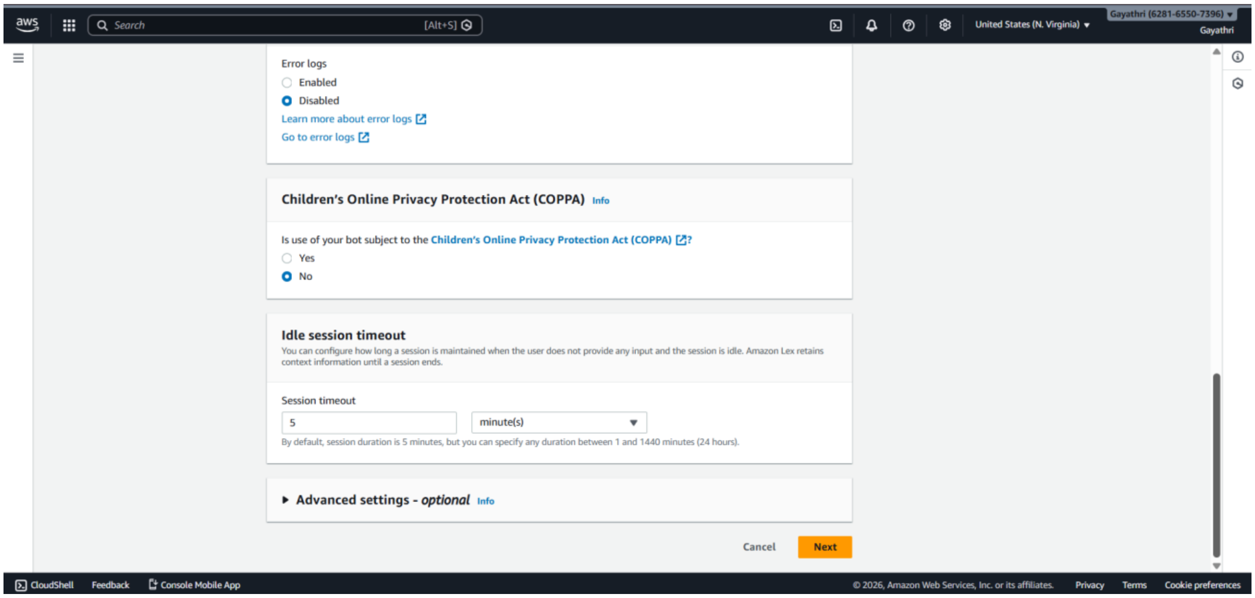
Enter: Bot name: HotelBookingBot -> Description is optinal



IAM role → Create new role

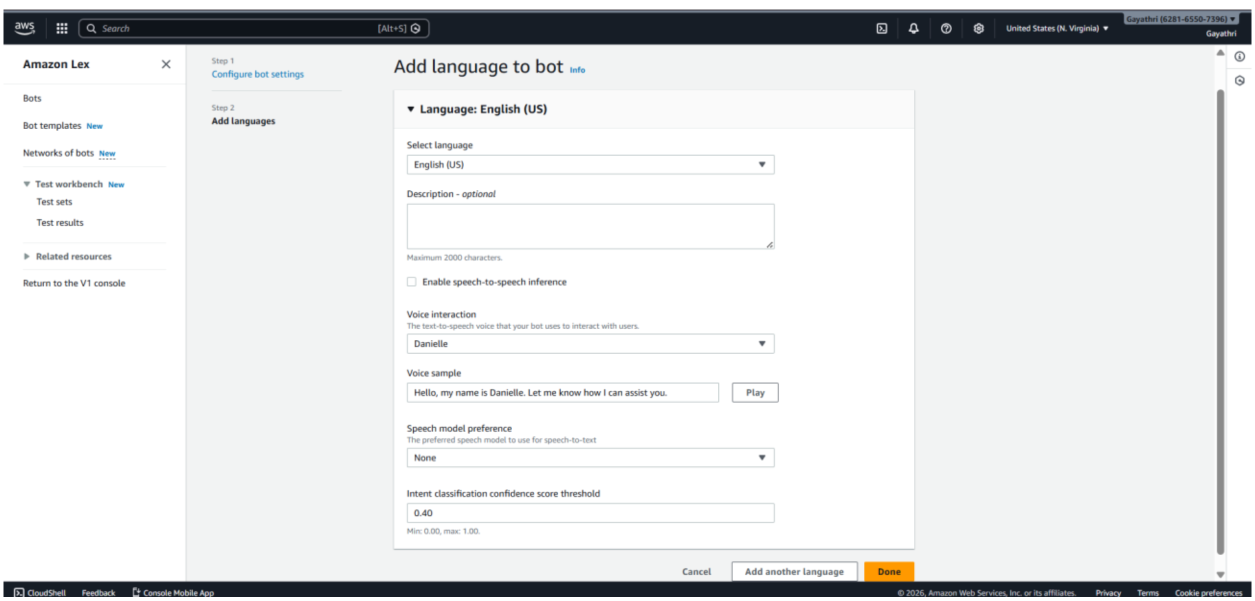


Remaining default→ Click next



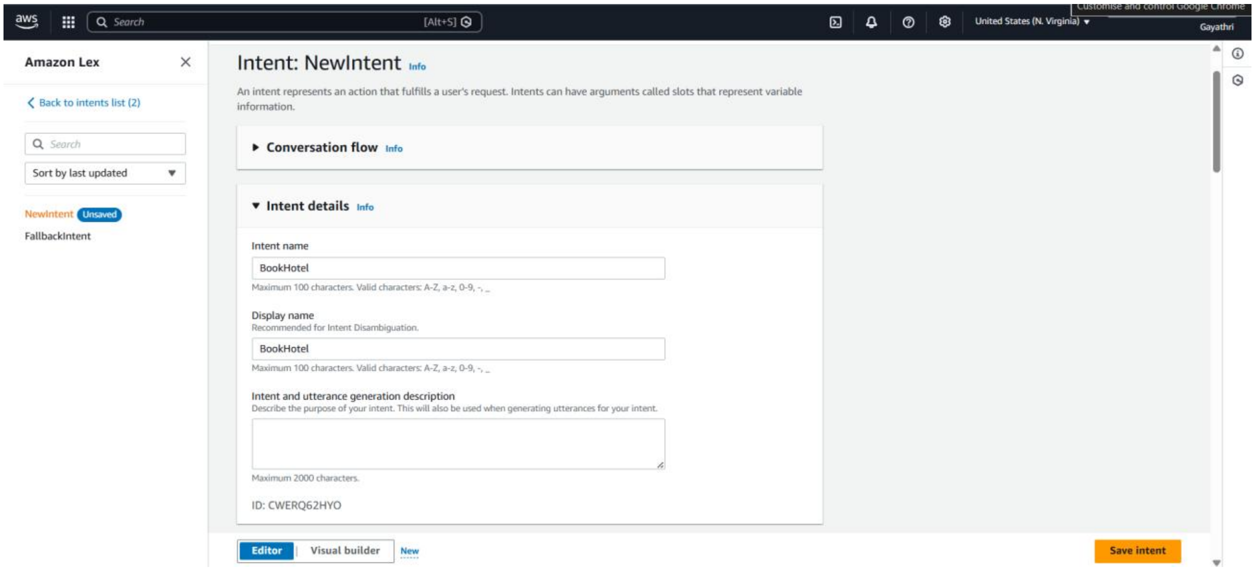
Language: English -> Voice (optional)

Click **Done**



Go to **Intents** **->** Name: BookHotel

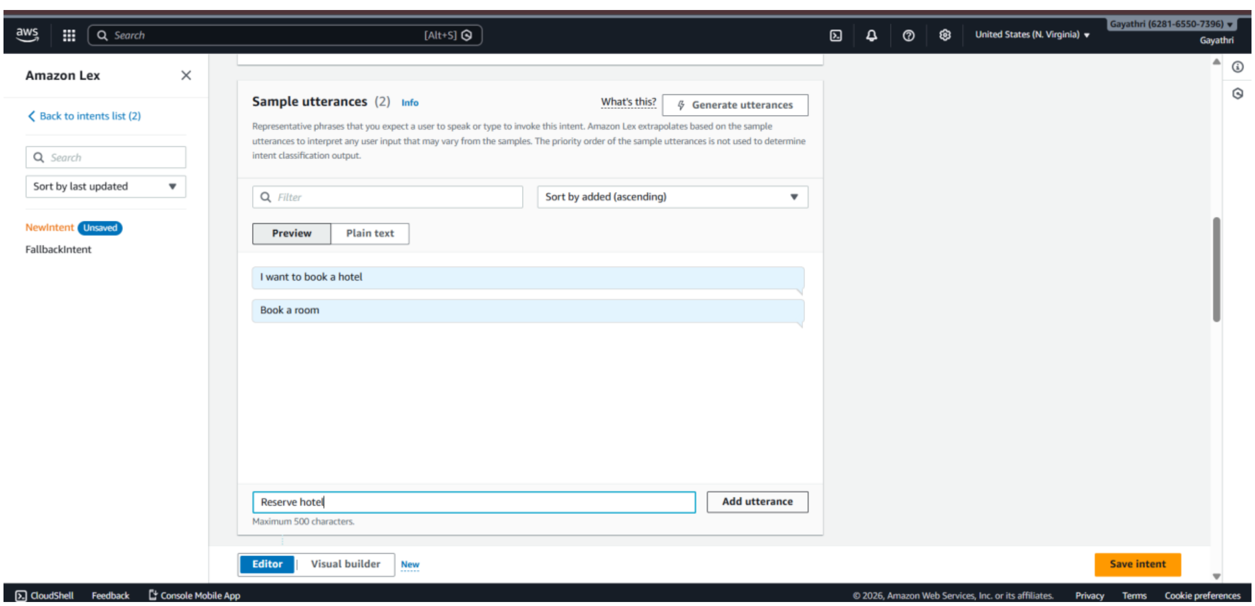
Click **Create**

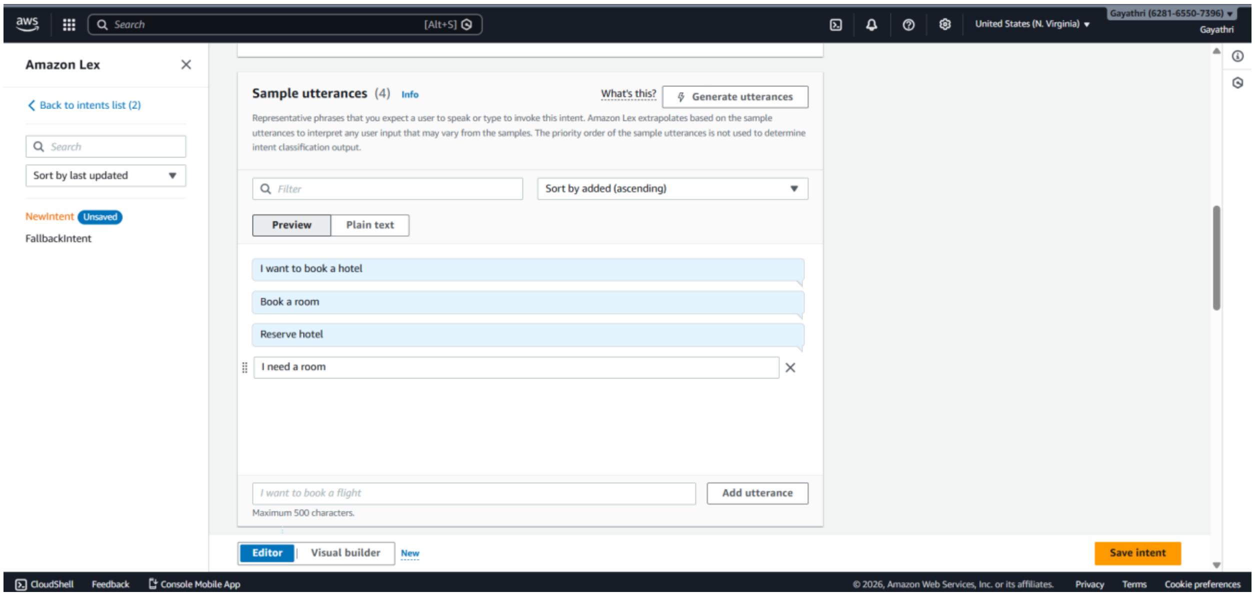


Add user sentences (how user talks):

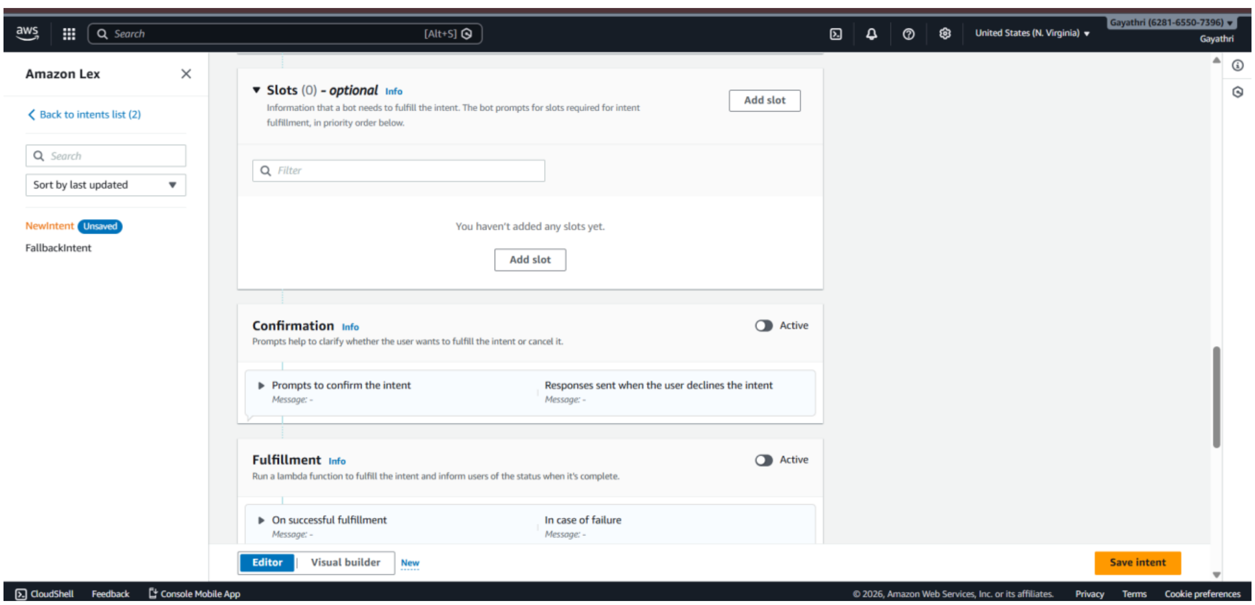
I want to book a hotel ,Book a room ,Reserve hotel , I need a room

These are **Utterances**



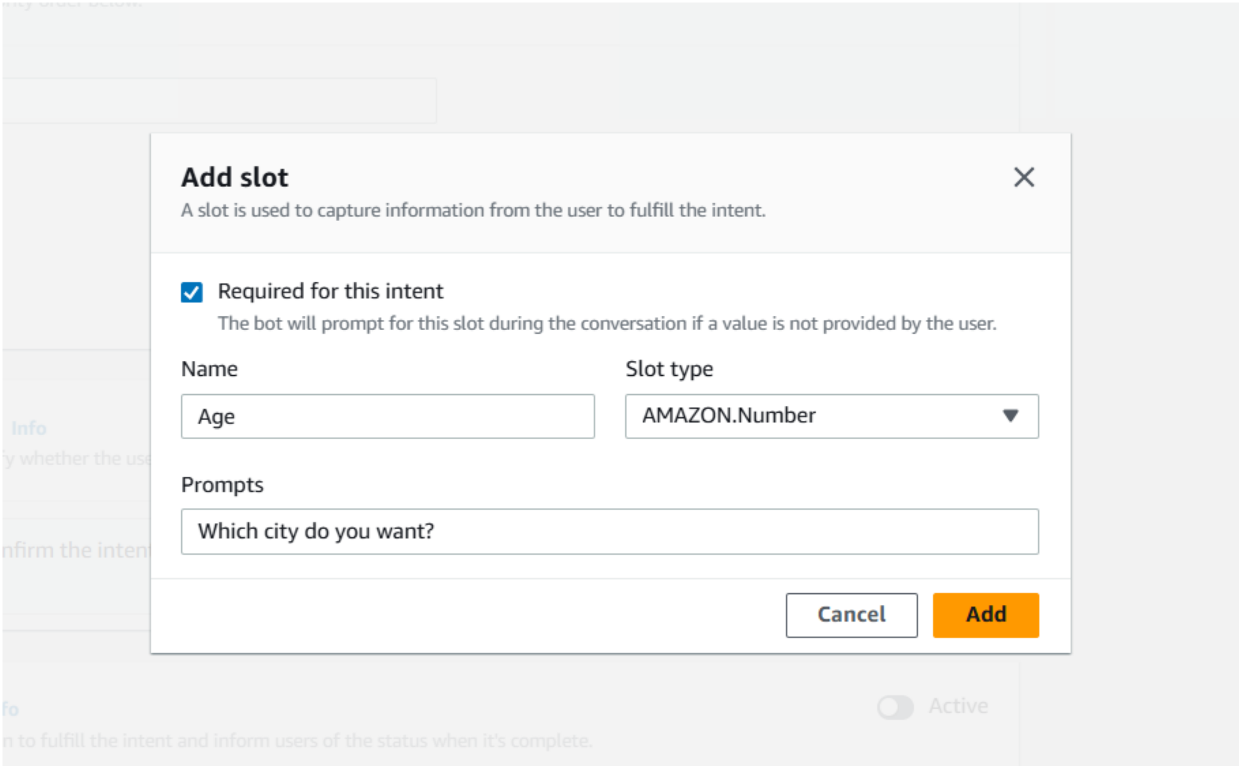


Click **Add slot**

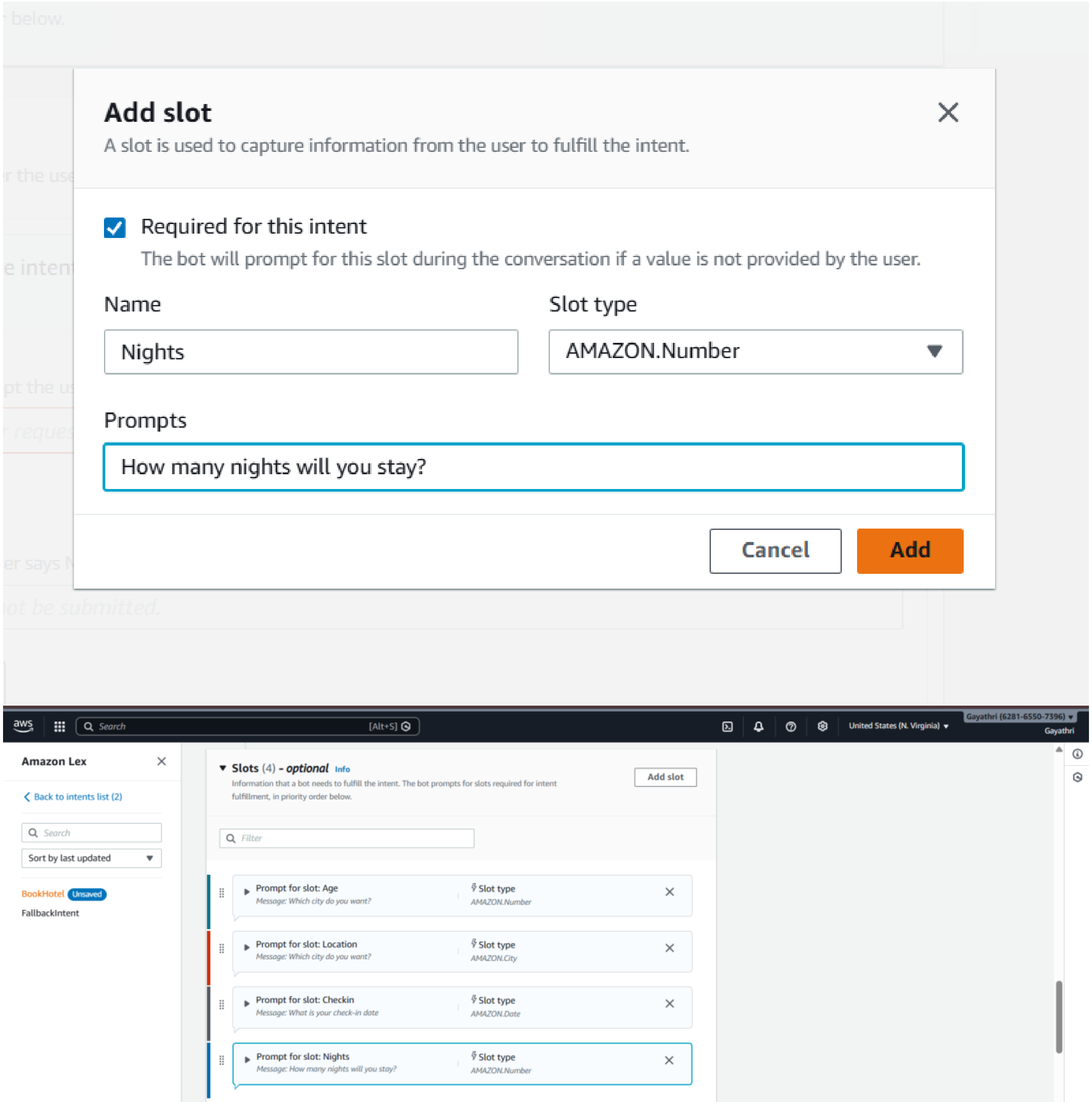


Fill: Slot name: age , Slot type: AMAZON.Number , Prompt: *Which city do you want?*

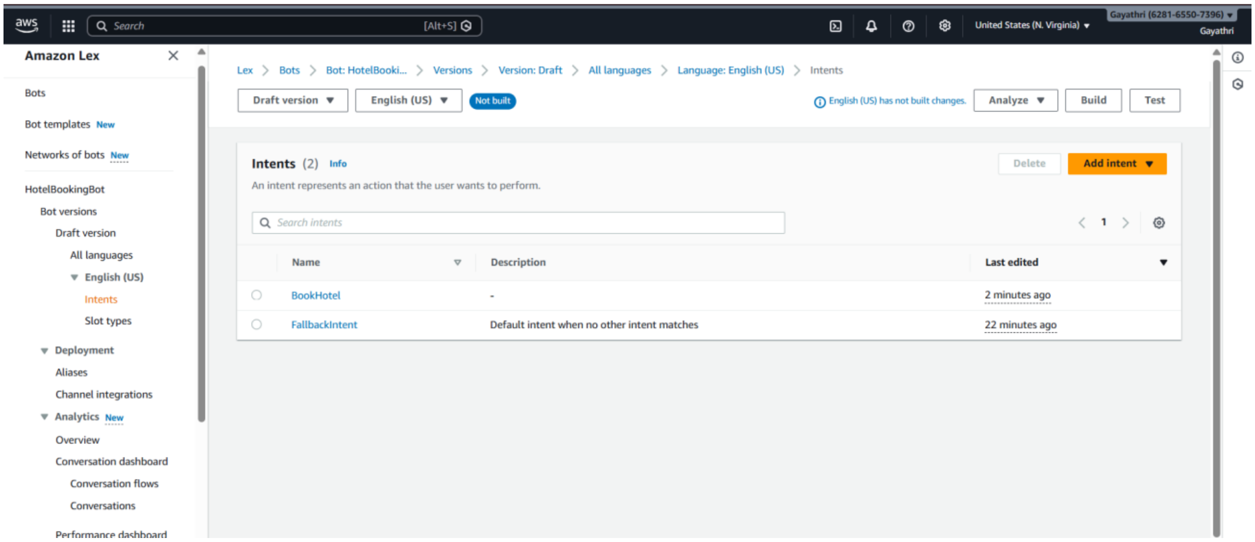
Mark as **Required -> click on Add**


Add slot
A slot is used to capture information from the user to fulfill the intent.
Required for this intent
The bot will prompt for this slot during the conversation if a value is not provided by the user.
Name
Age
Slot type
AMAZON.Number
Prompts
Which city do you want?
Cancel
Add
Active

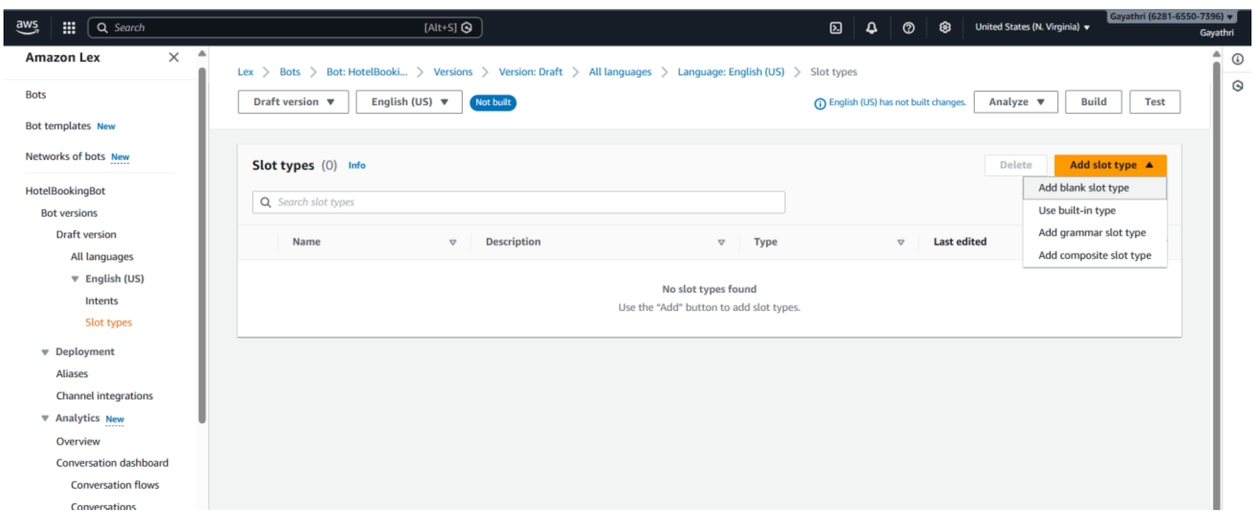
Click on Age slot



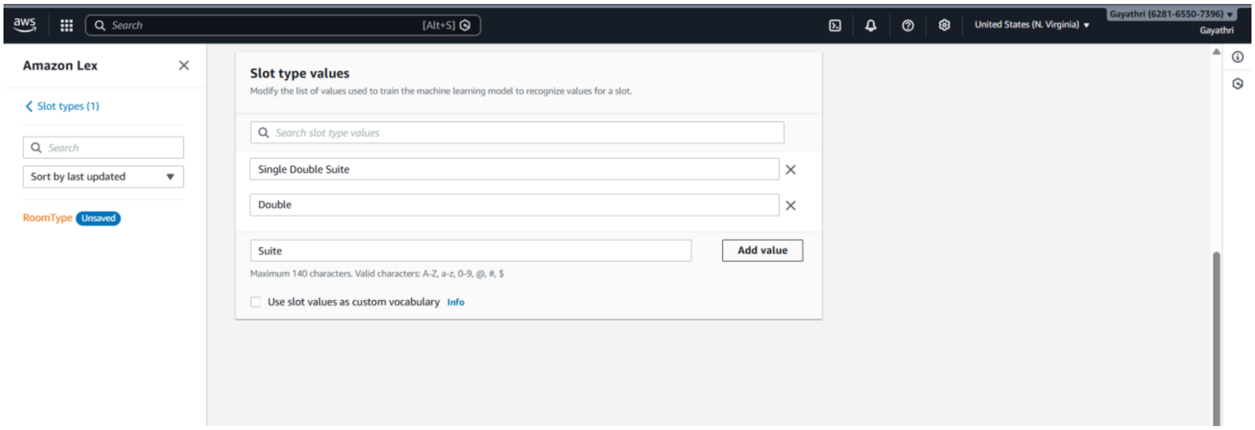
Click on back to instance list



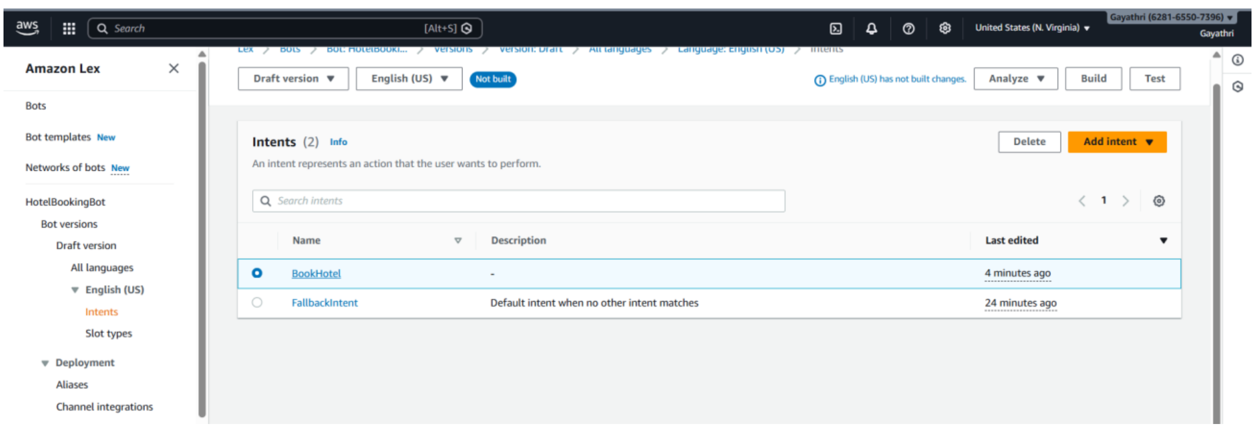
Click on slot types -> click on Add slot type -> click on Add blank slot type



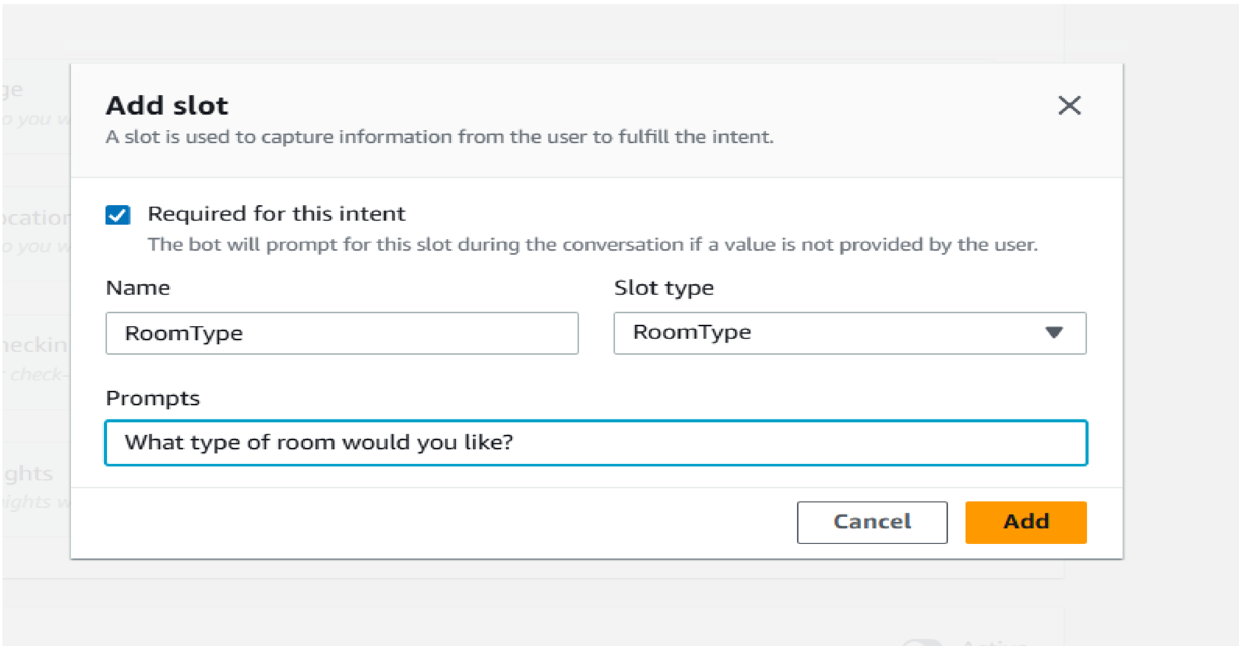
Click on Add values and add “single” ,”double”, “suite” values



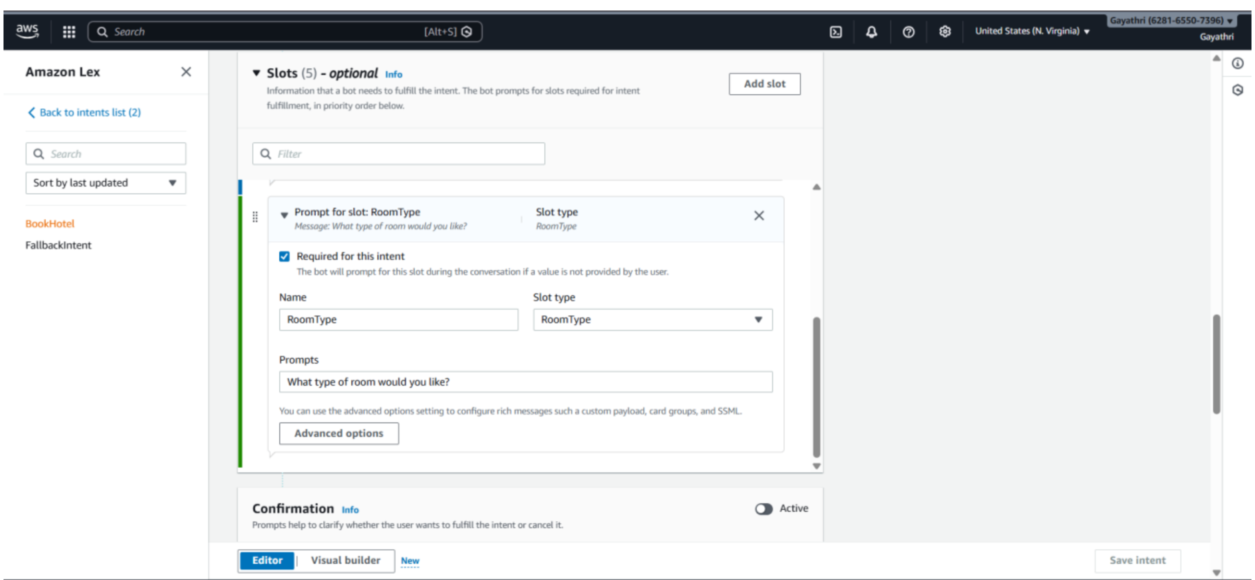
Click on bookhotel



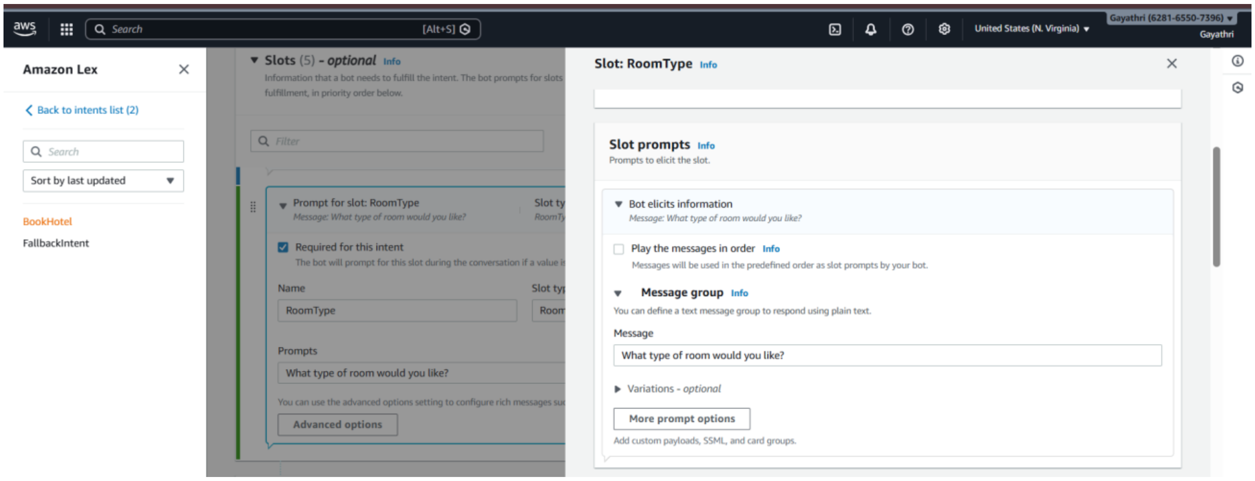
- Go to **Slot -> click on add slot**
- Slot name: RoomType
- Slot type: RoomType



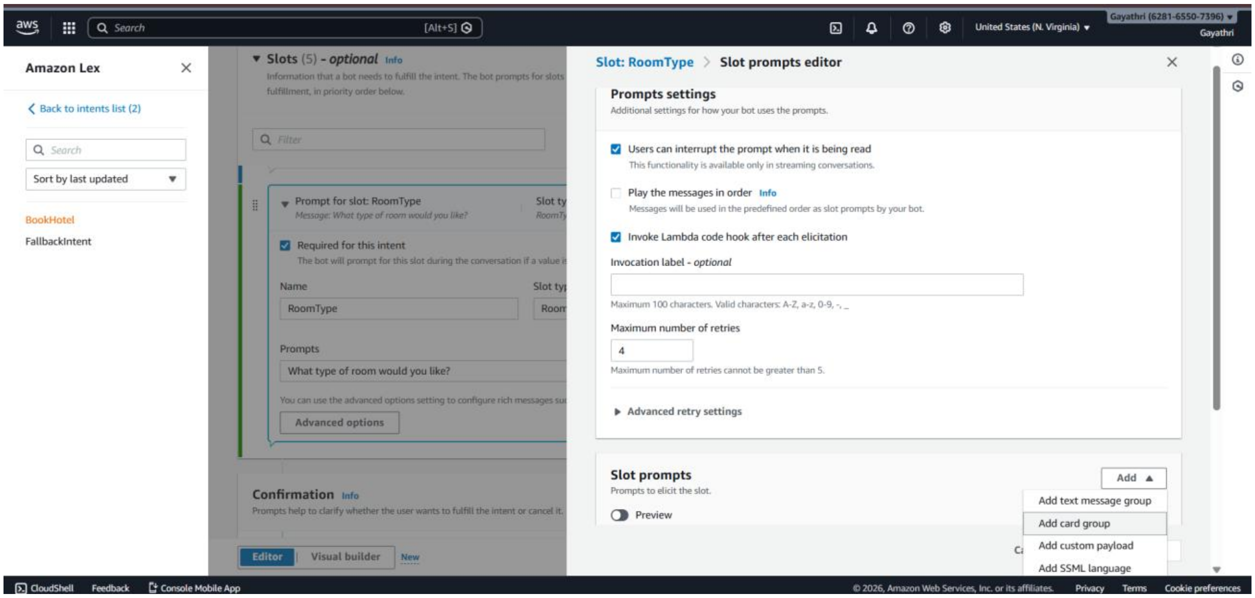
Click on advance options



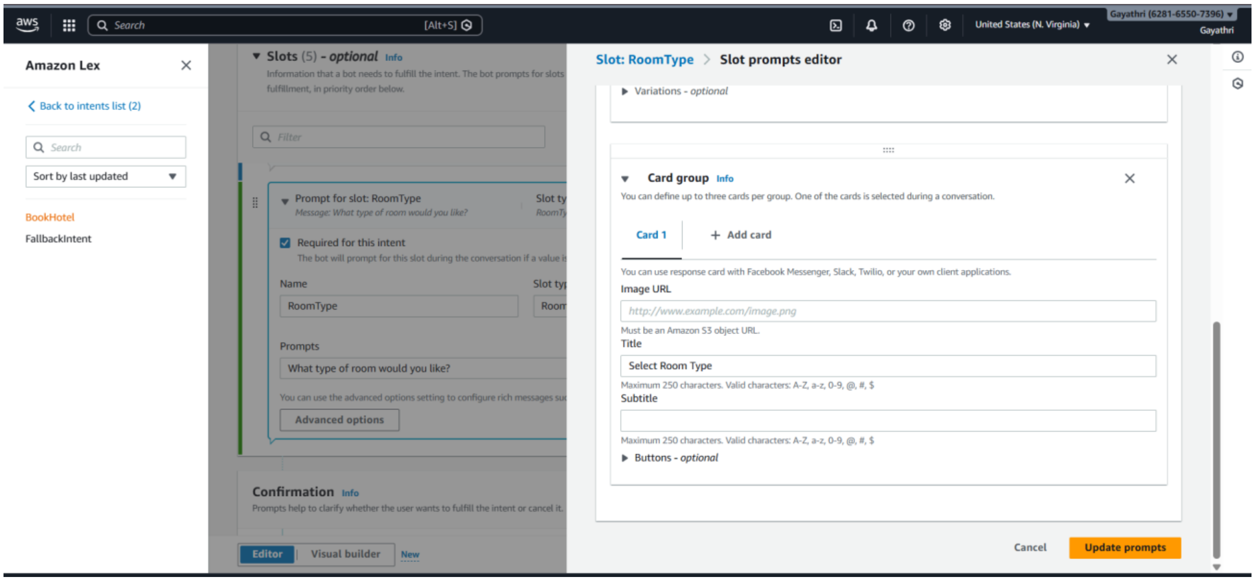
Go to **slot prompts -> more prompt options**



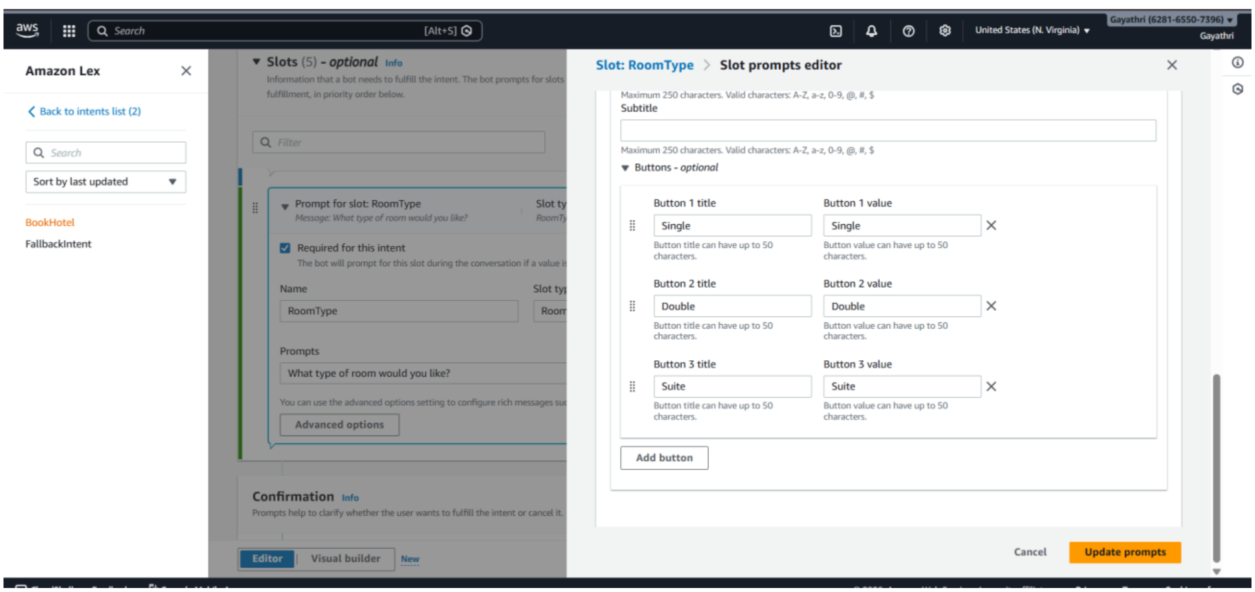
Click **Add -> add card groups**



Add title -> click on buttons

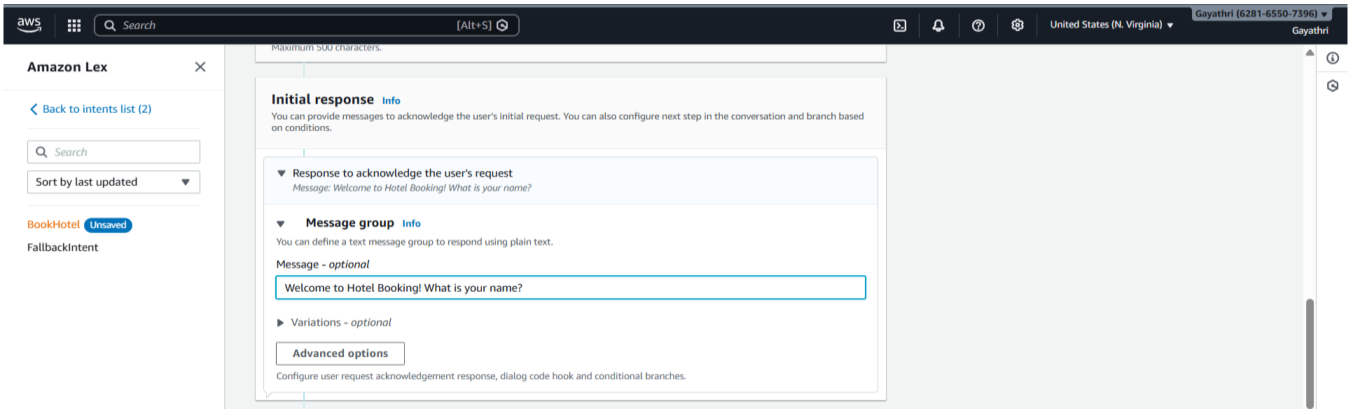


Add titles and values -> update prompts



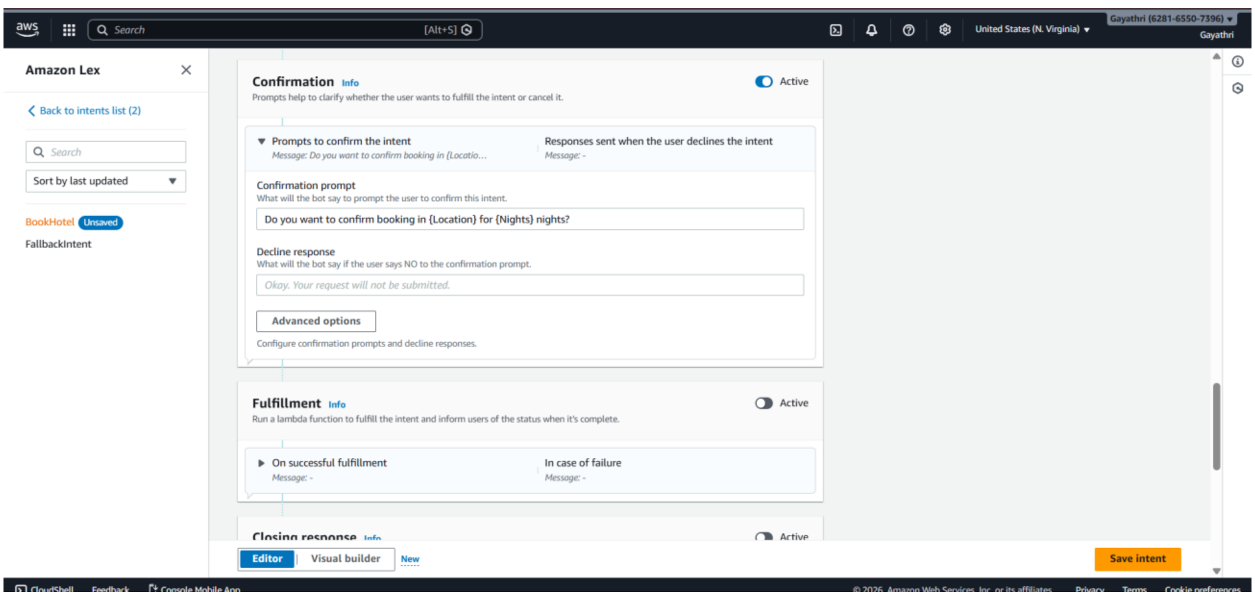
Initial Response:

Welcome to Hotel Booking! What is your name?

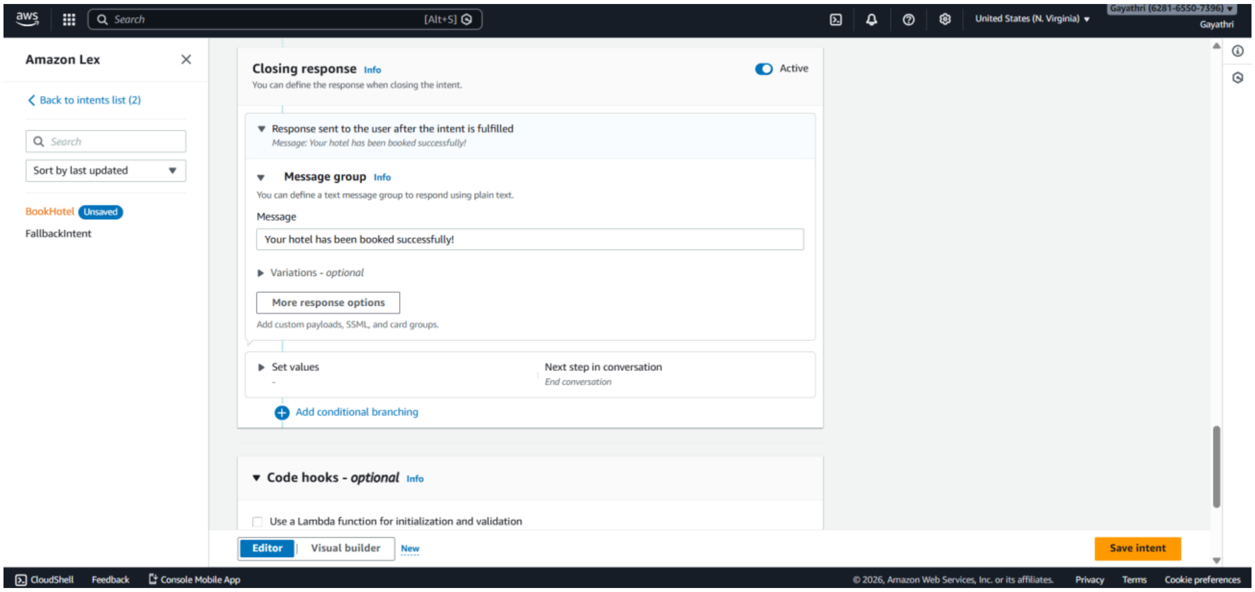


Confirmation:

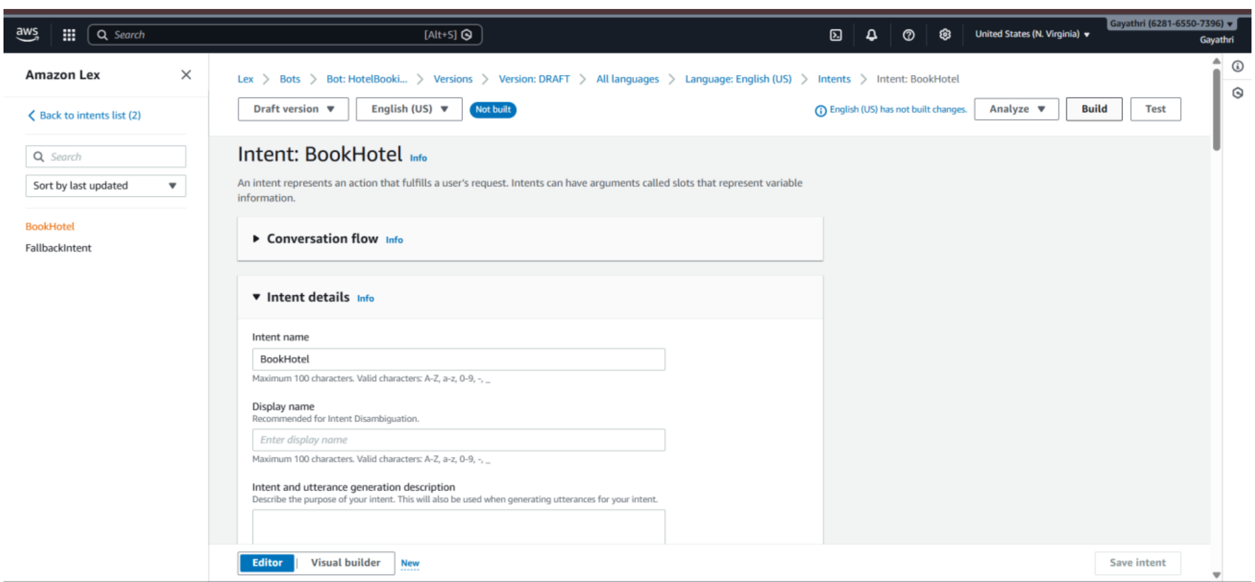
Do you want to confirm booking in {location} for {nights} nights?



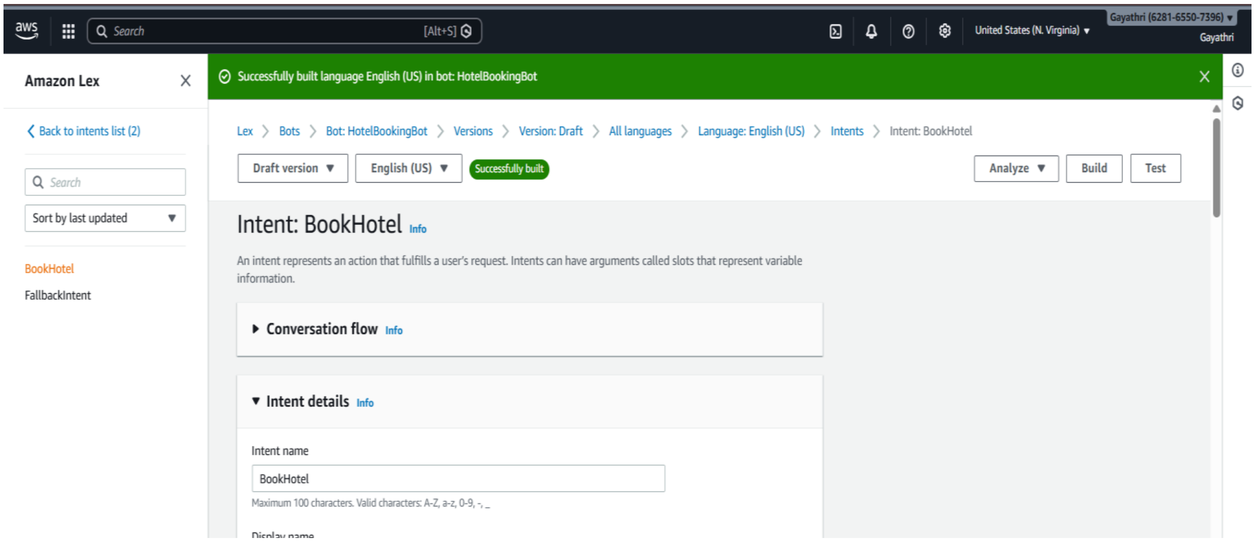
Add closing response (optional) -> click on save intent



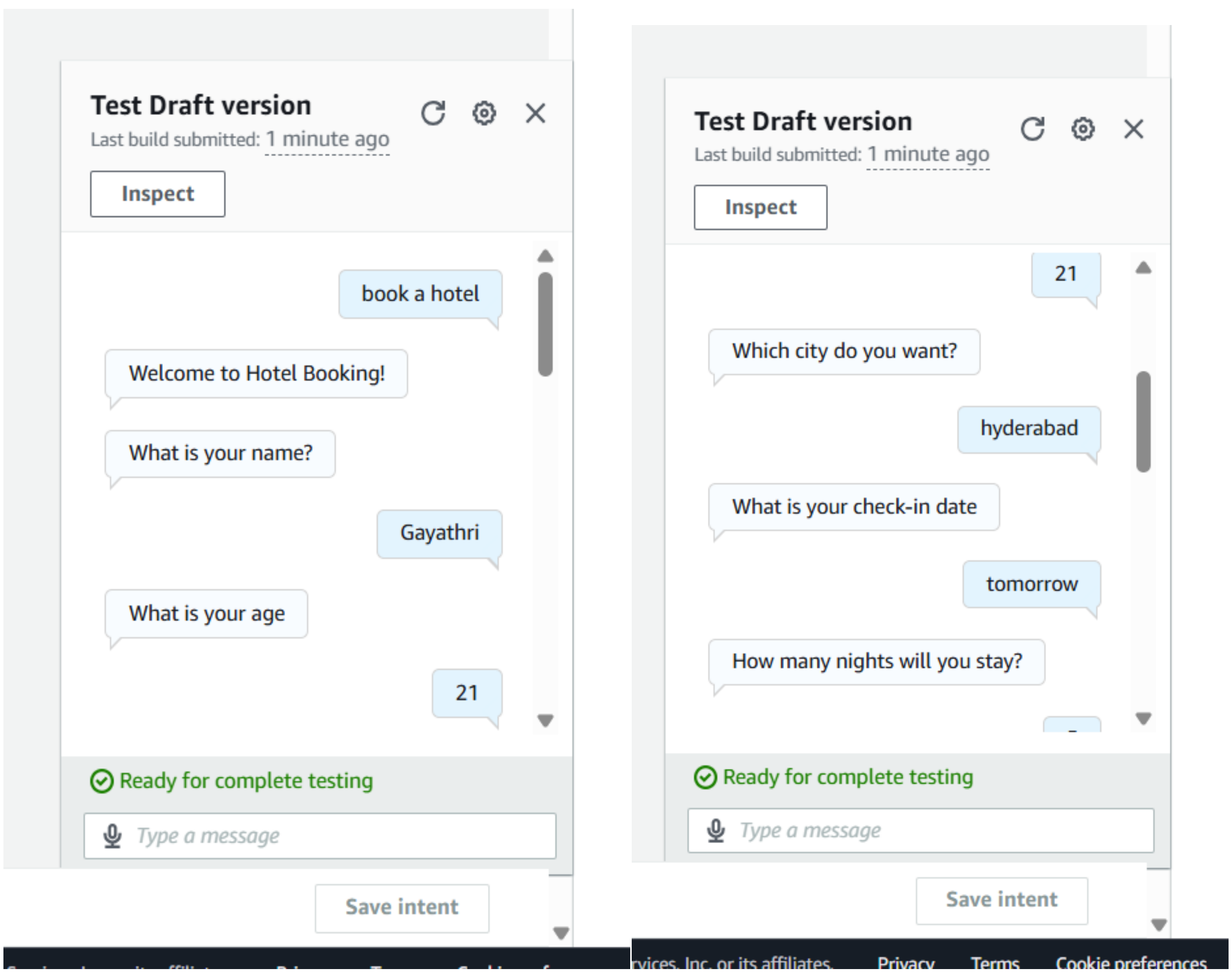
Click on build

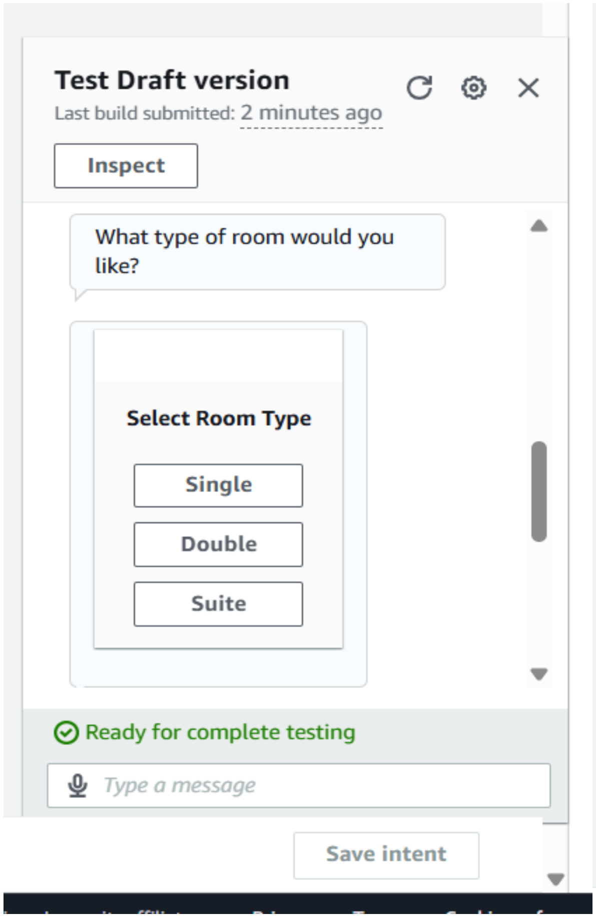
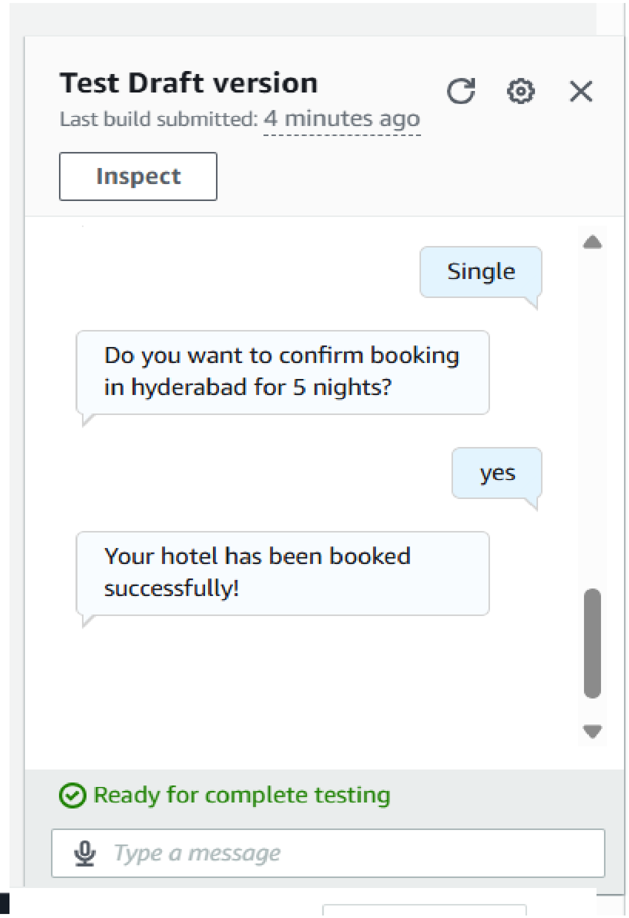


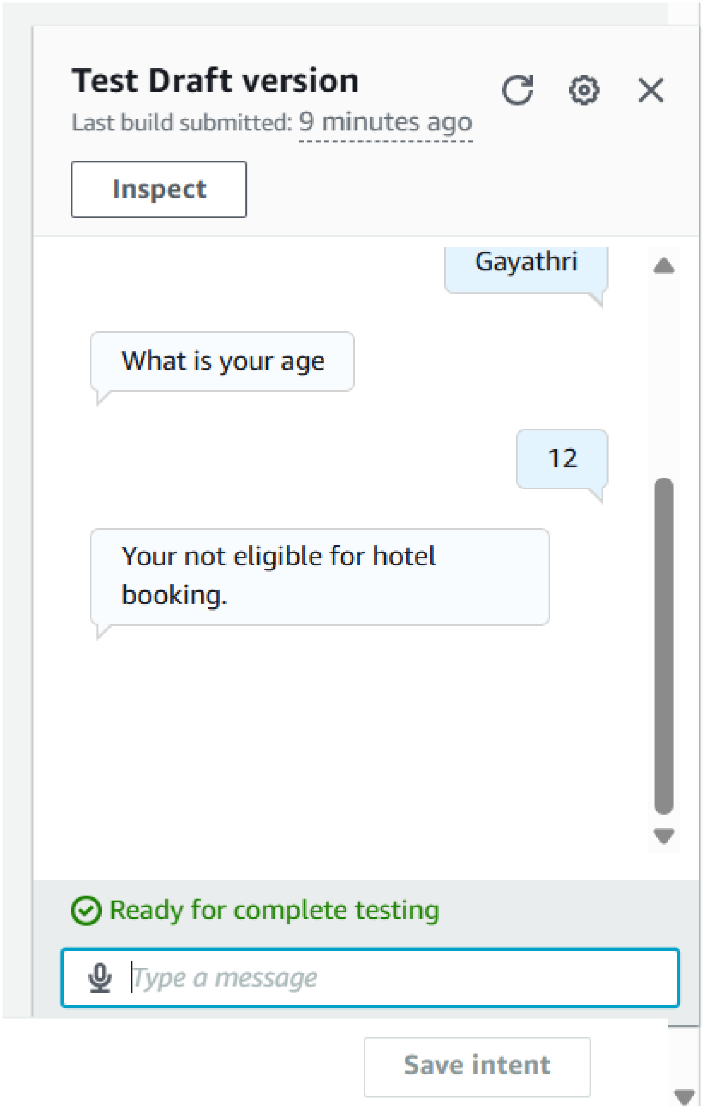
After the build is successful -> click on test



Test the chatbot



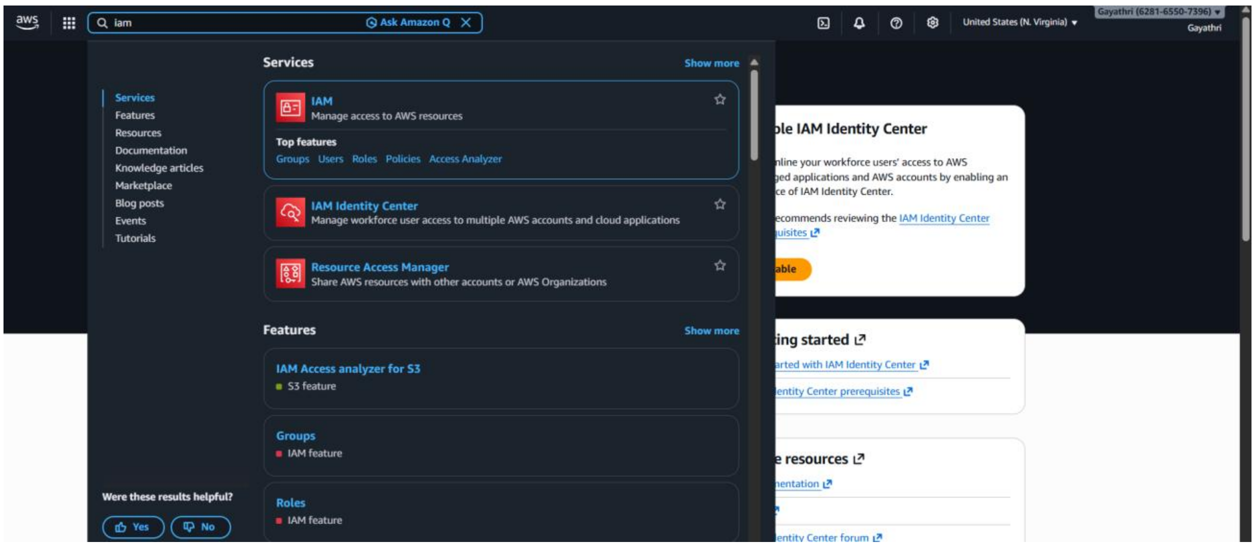

Test Draft version
Last build submitted: 2 minutes ago
Inspect
What type of room would you like?
Select Room Type
Single
Double
Suite
Ready for complete testing
Type a message
Save intent

Test Draft version
Last build submitted: 4 minutes ago
Inspect
Single
Do you want to confirm booking in hyderabad for 5 nights?
yes
Your hotel has been booked successfully!
Ready for complete testing
Type a message


Test Draft version
Last build submitted: 9 minutes ago
Inspect
Gayathri
What is your age
12
Your not eligible for hotel booking.
Ready for complete testing
Type a message
Save intent

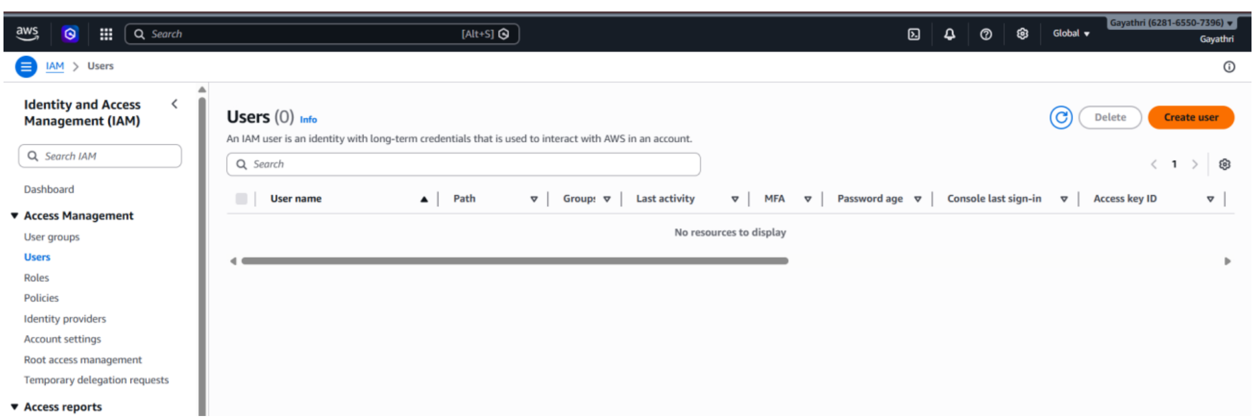
WEEK-12 –Creating IAM USER with permission Access it with GUI and CLI

Log in to the AWS Management Console as the **Root User**.

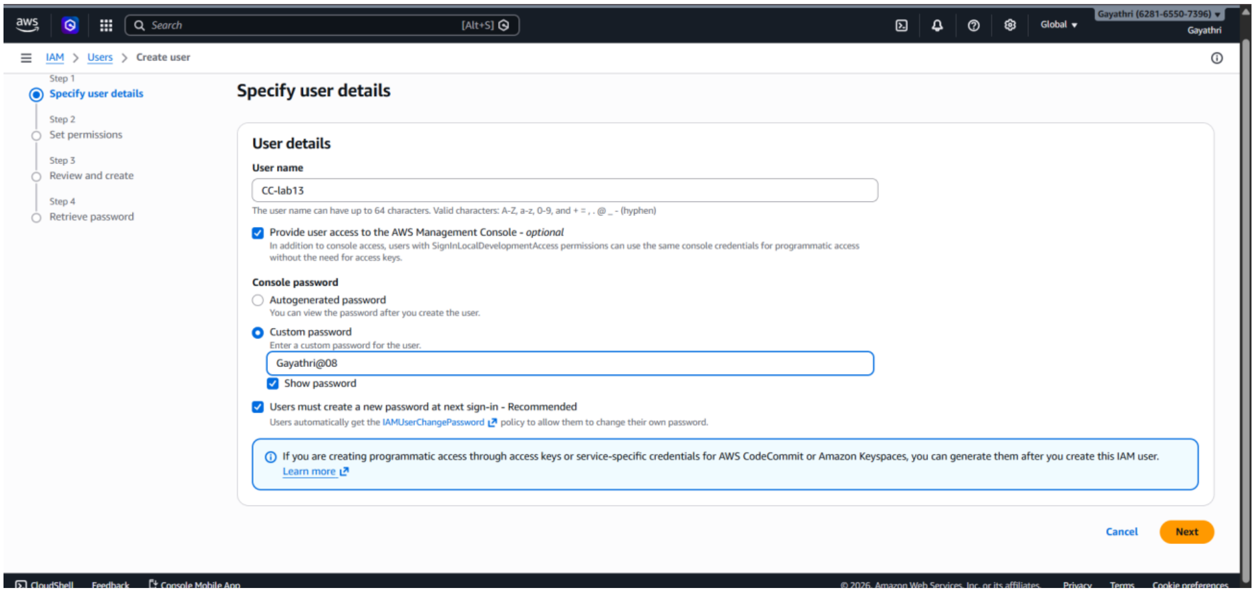
Search for **IAM** in search bar.



Navigate to Users and click Create user.

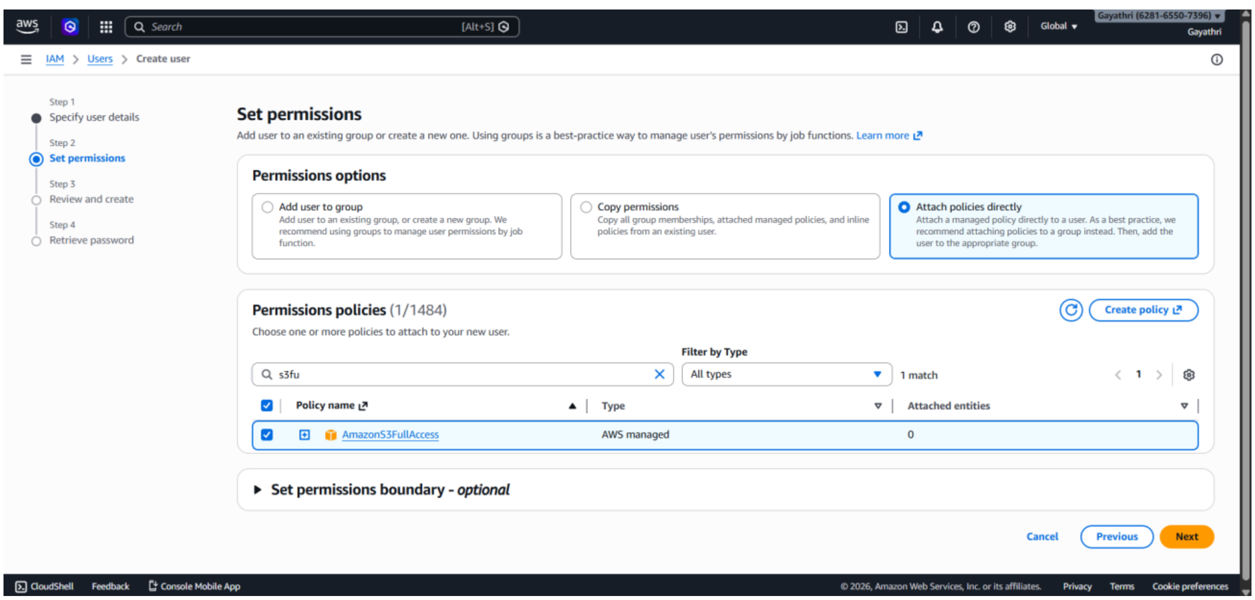


- User Details: Enter a name .Check the box for "Provide user access to the AWS Management Console." – custom password.
- Click next

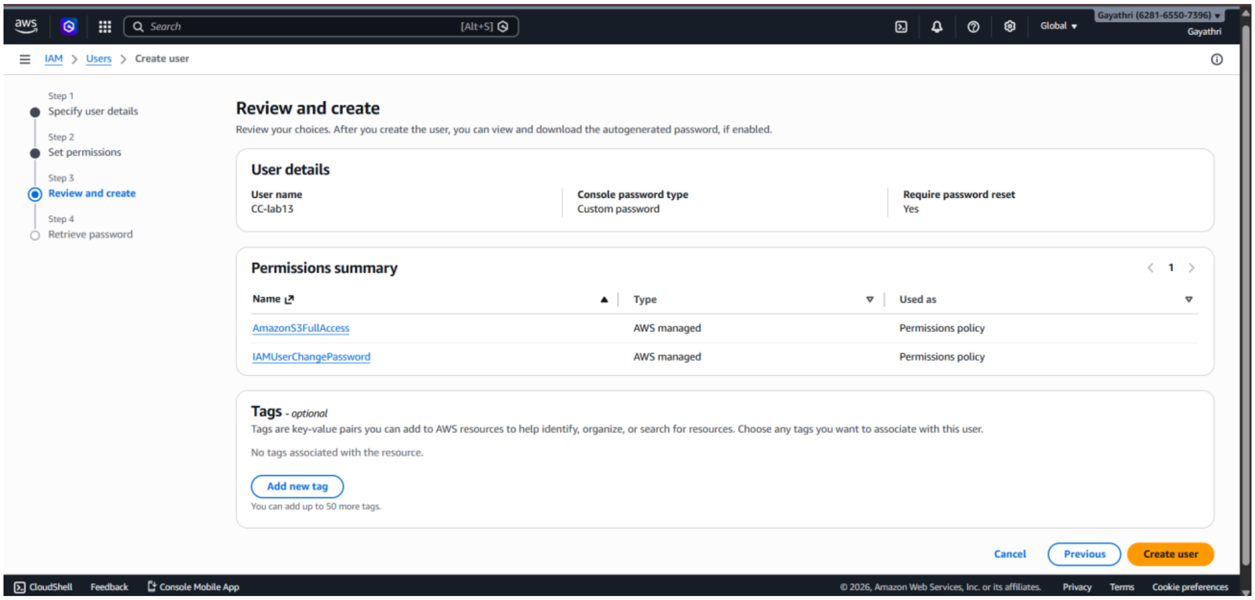


Set Permissions: Select "Attach policies directly." Search for and select AmazonS3FullAccess.

Click Next

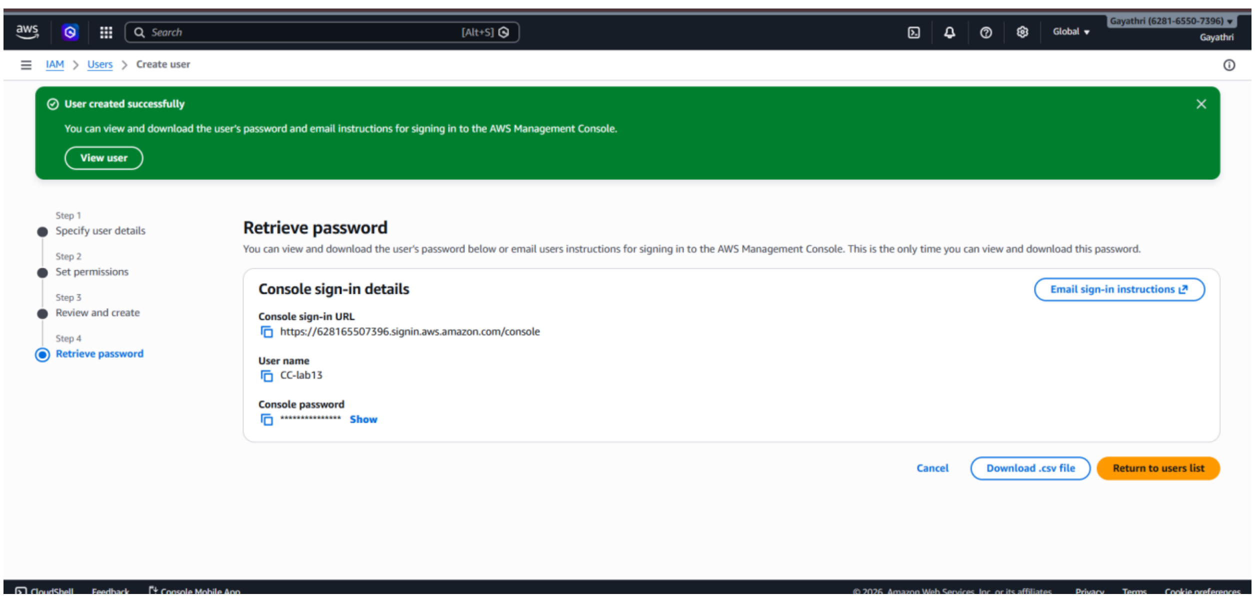


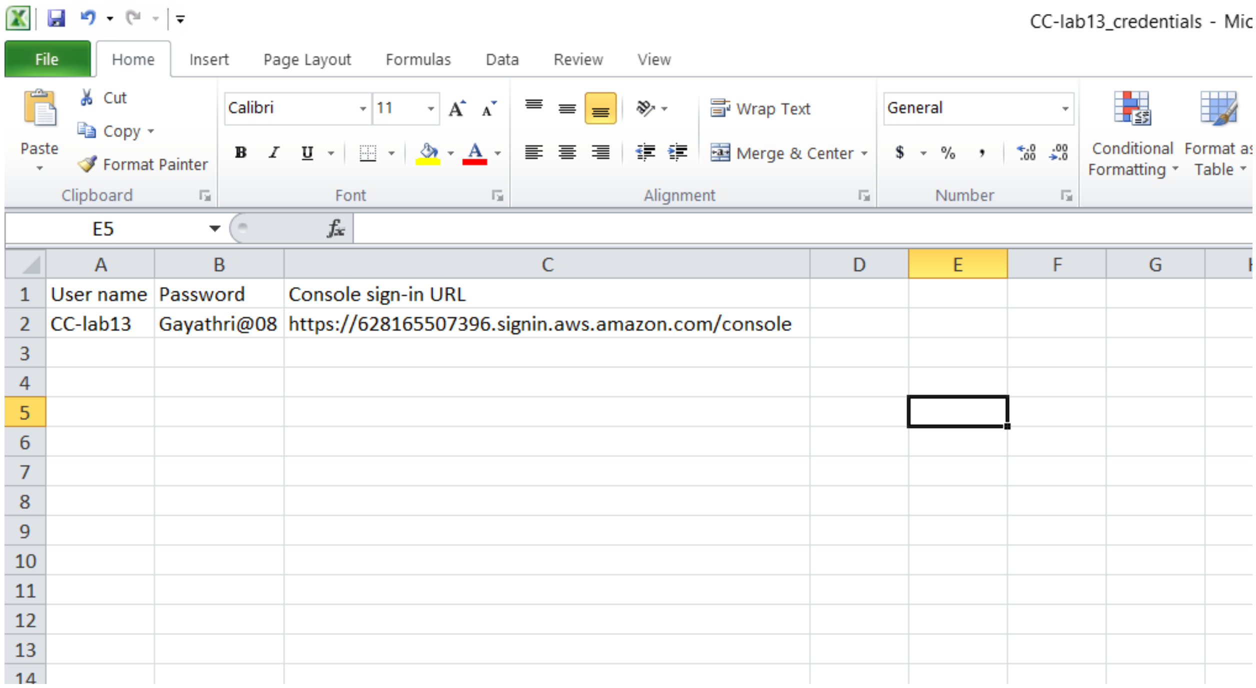
- Review & Create: Complete the wizard. On the final page, click Download .csv file.



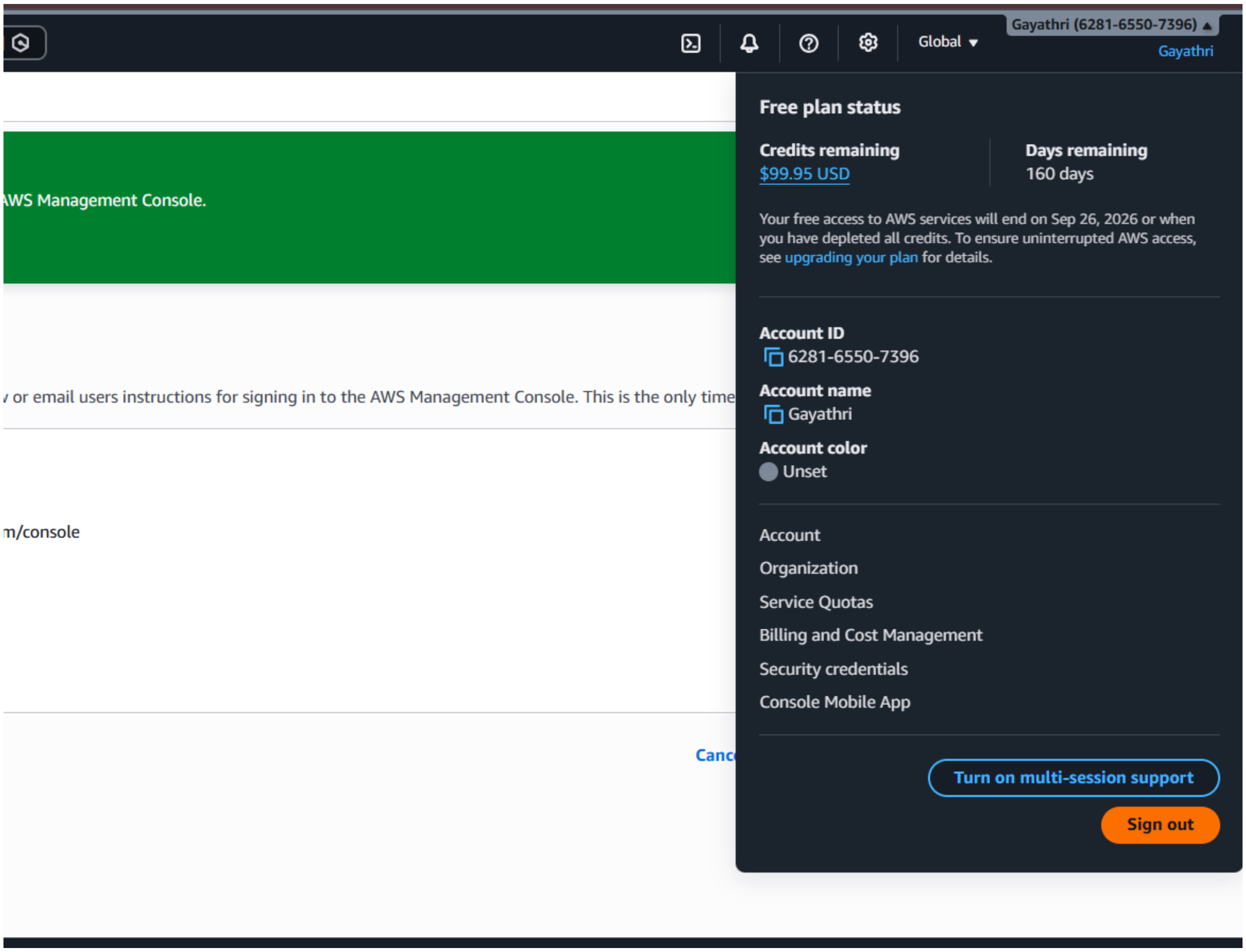
Click on download .csv file

Copy your 12-digit Account ID (found in the top-right dropdown or the .csv) .

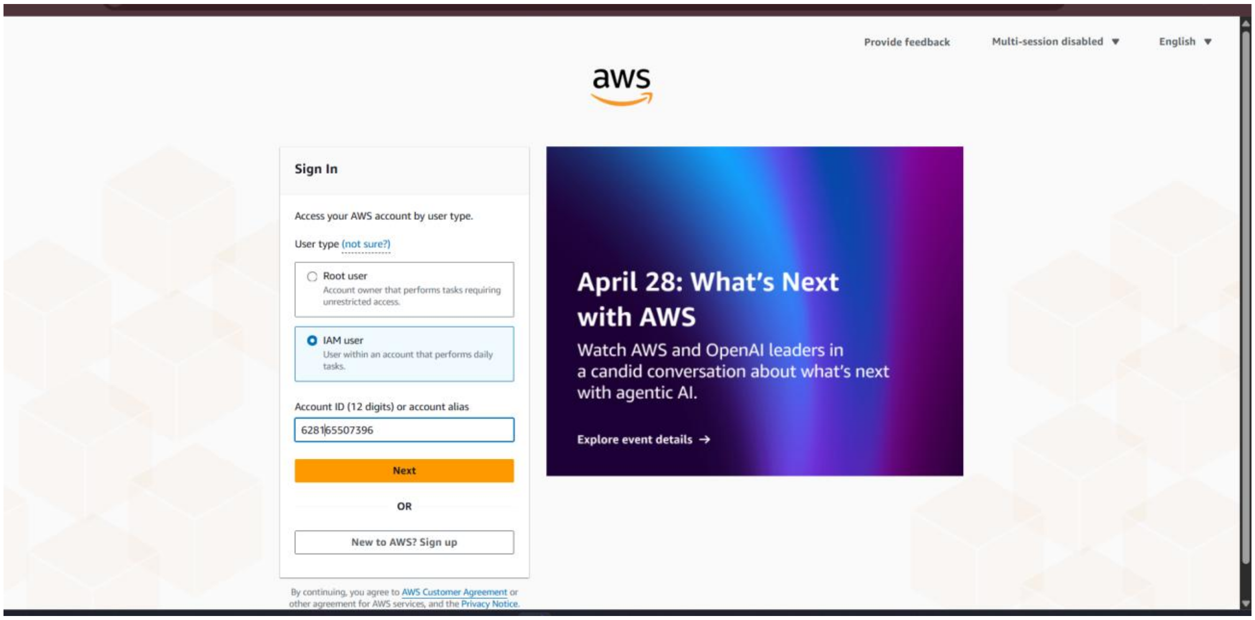


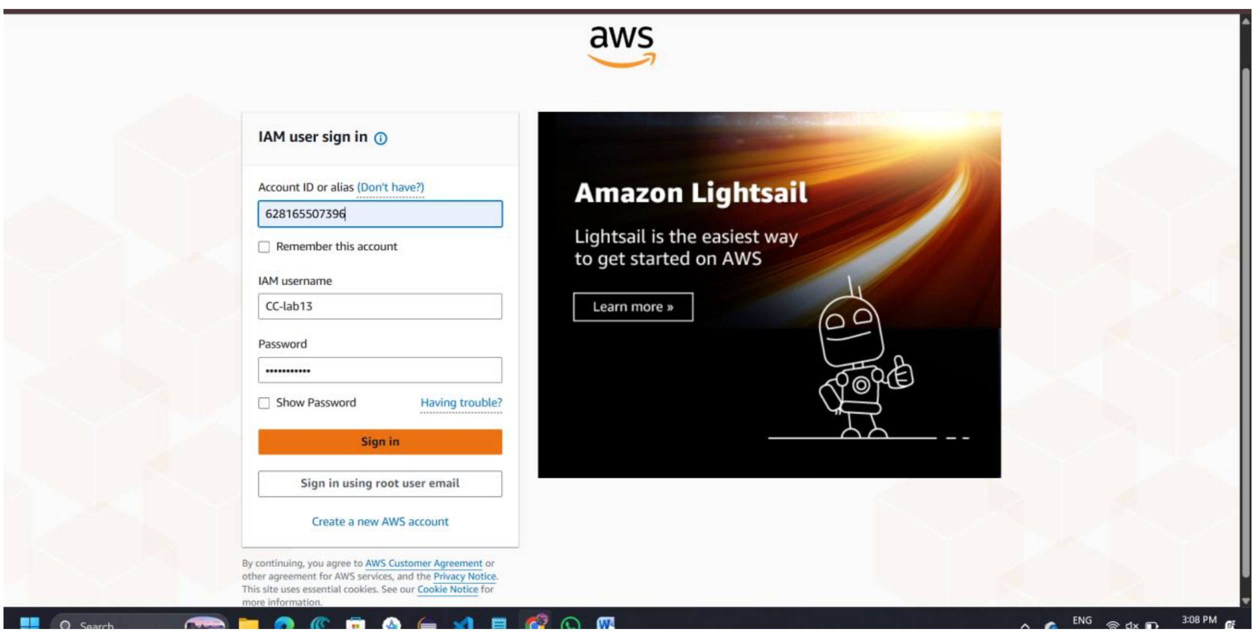

CC-lab13_credentials - Mic
File
Home
Insert
Page Layout
Formulas
Data
Review
View
Paste
Cut
Copy
Format Painter
Clipboard
Calibri
11
Font
Wrap Text
Merge & Center
Alignment
General
Number
Conditional Formatting
Format as Table
E5
User name
Password
Console sign-in URL
CC-lab13
Gayathri@08
https://628165507396.signin.aws.amazon.com/console

Sign out the root account

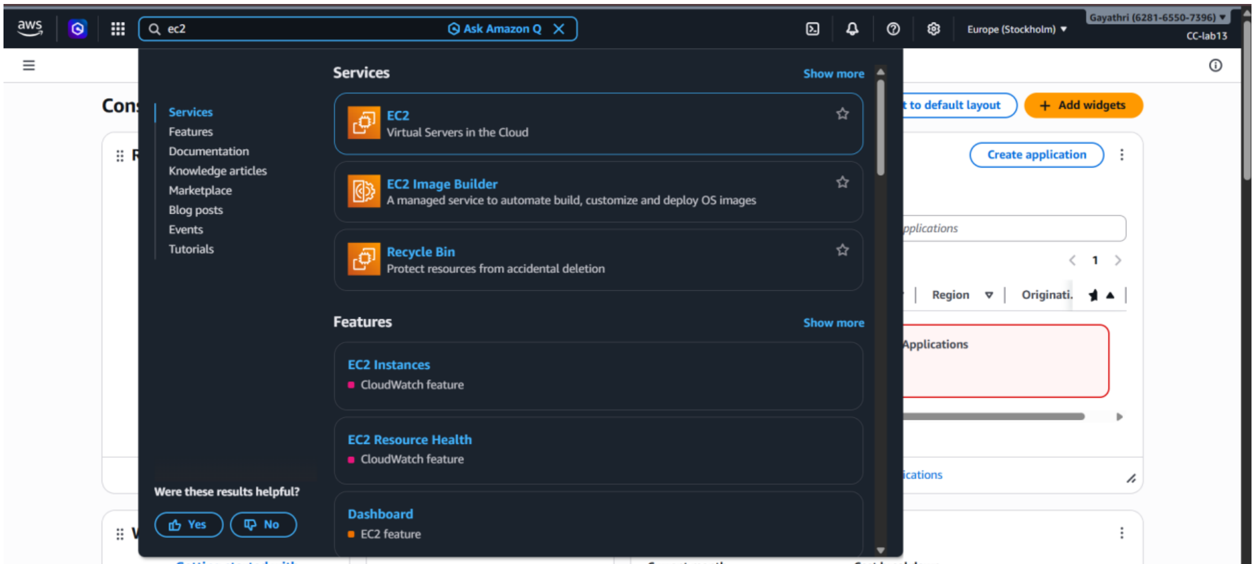


- Use the Sign-in URL from the .csv (or go to the AWS sign-in page and select **IAM User**).
- Enter the **Account ID**, then the **IAM Username** and **Password** from the .csv.

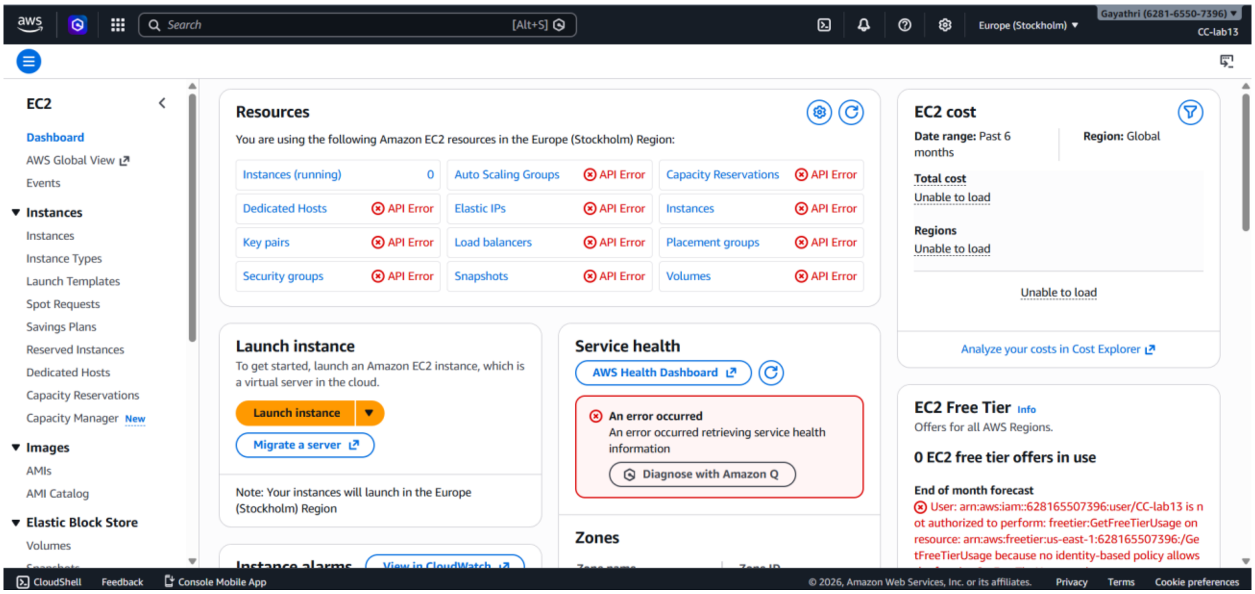




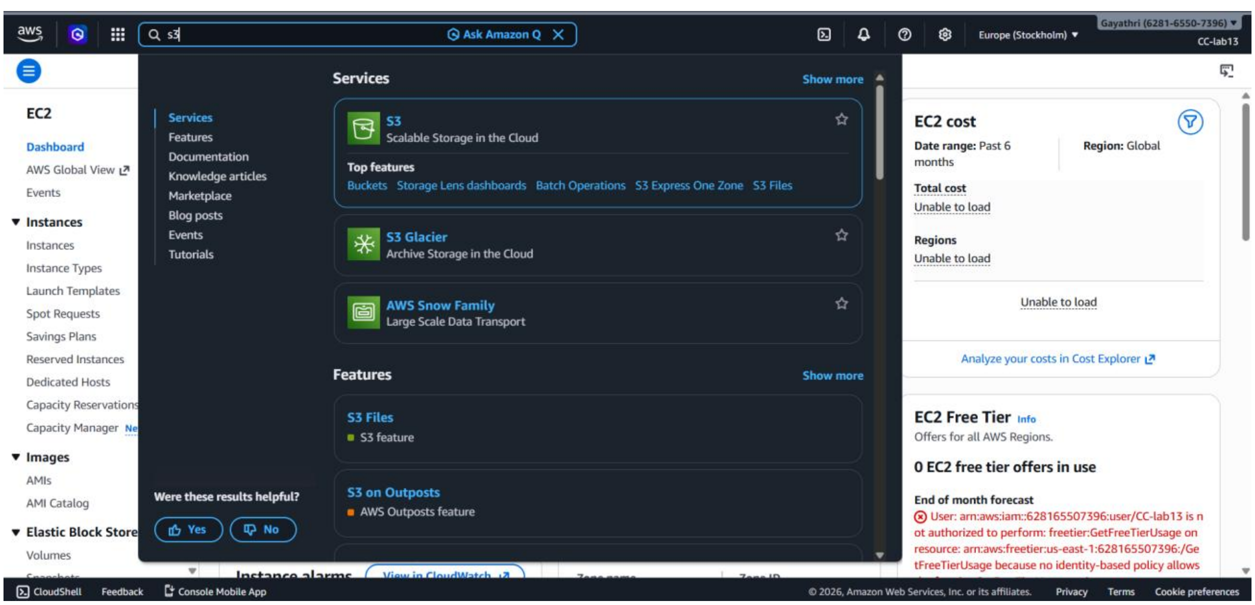
Test 1 (Failure): Navigate to **EC2.**



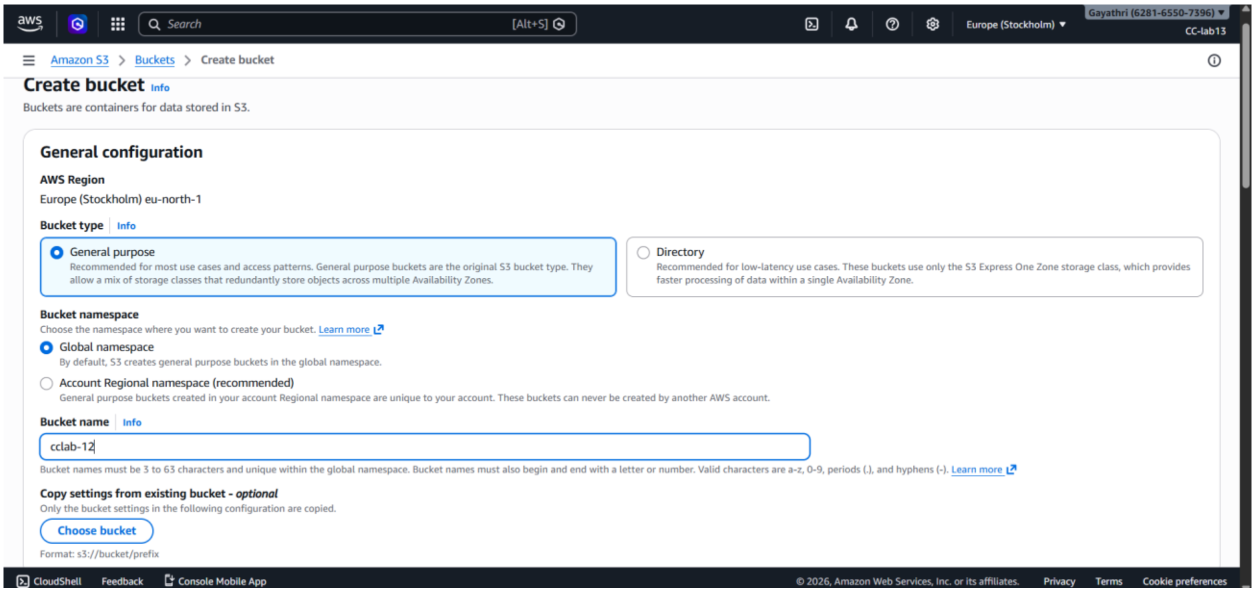
- Try to view instances. You will see "API Error" or "Access Denied." This proves the user is restricted

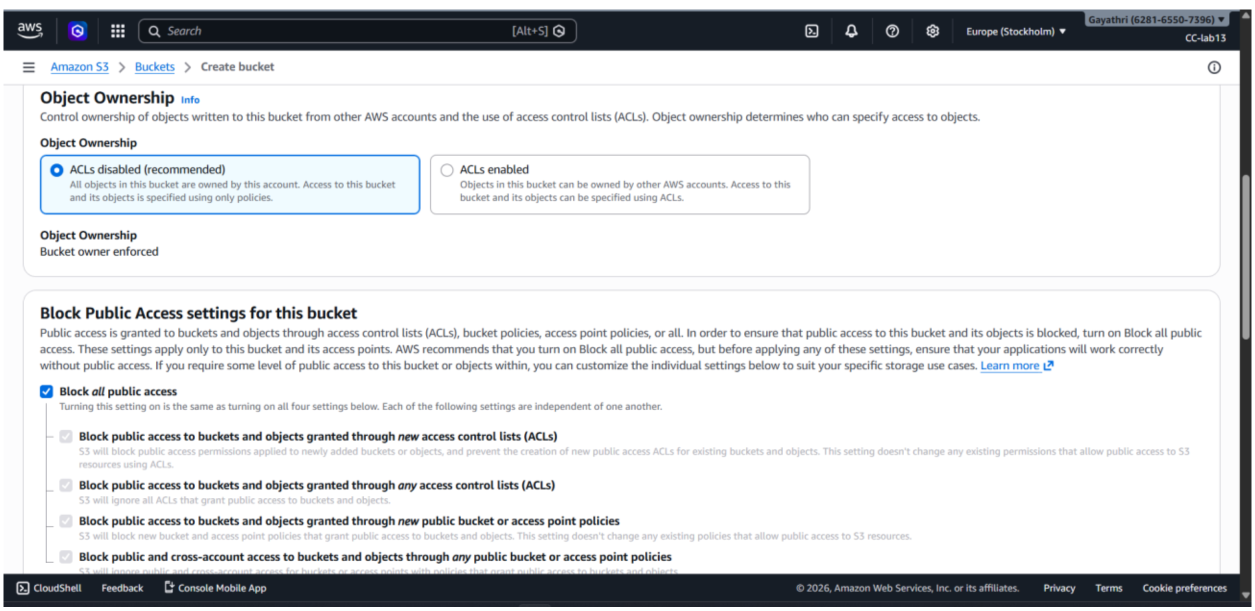


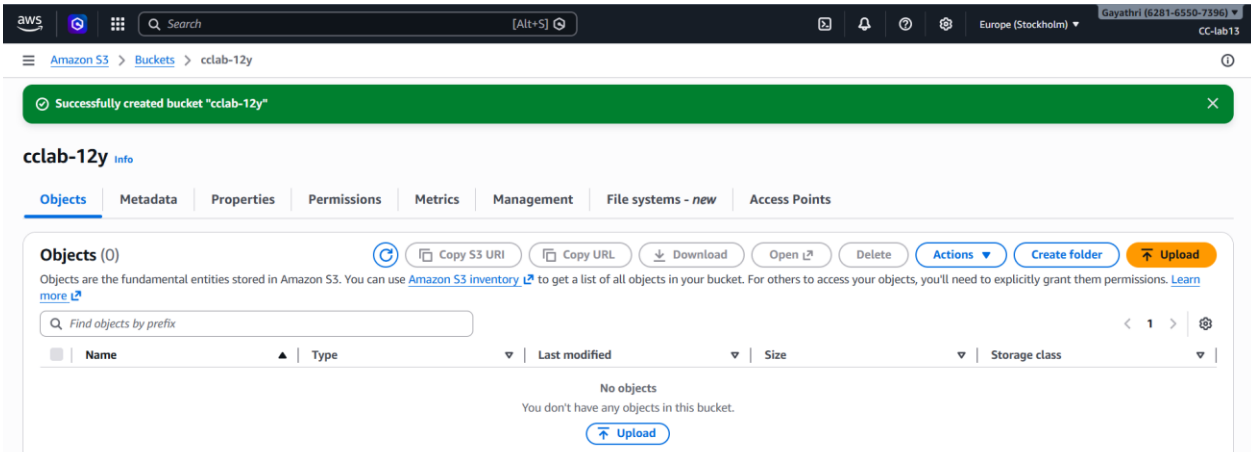
Test 2 (Success): Navigate to **S3**.



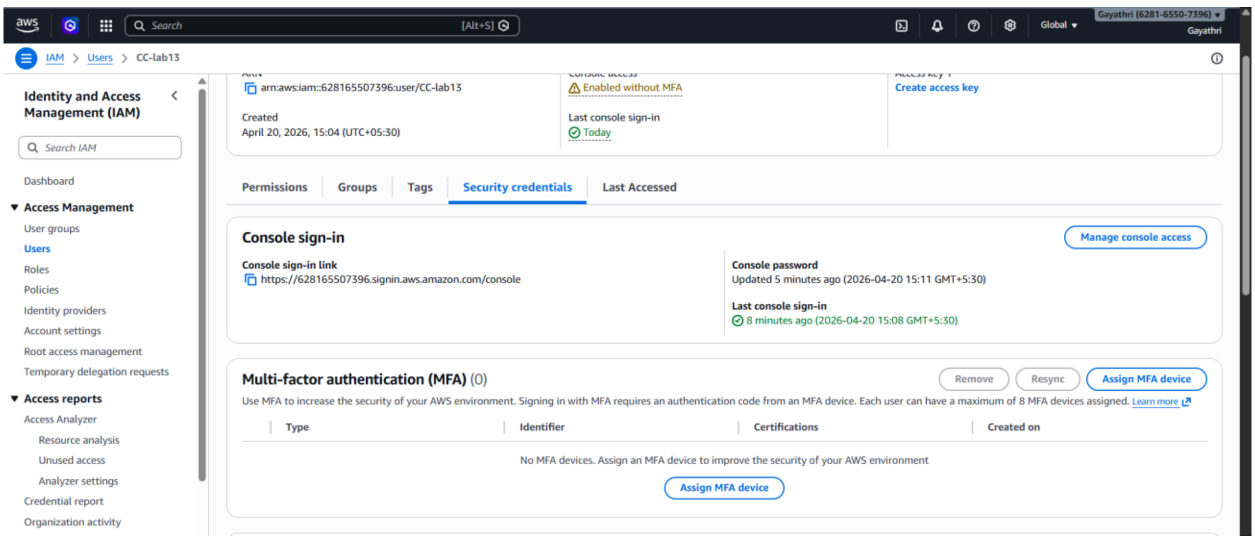
- Create a new bucket. It will work perfectly because of the attached policy.



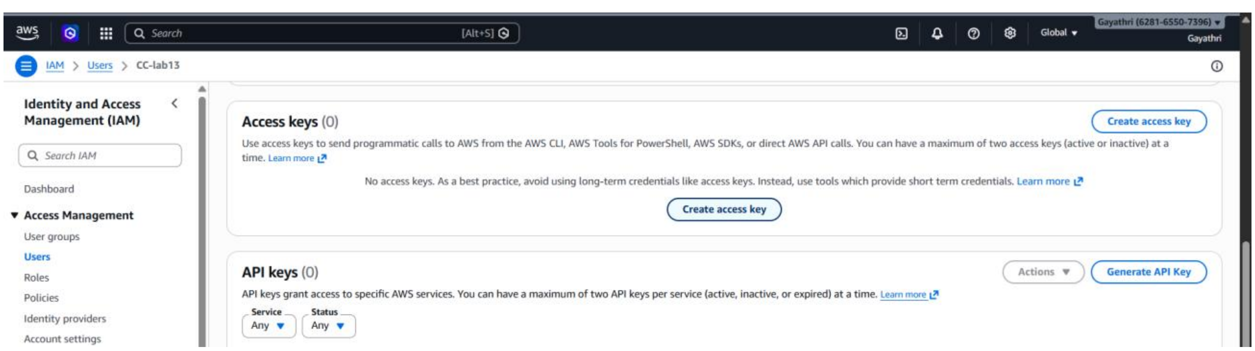




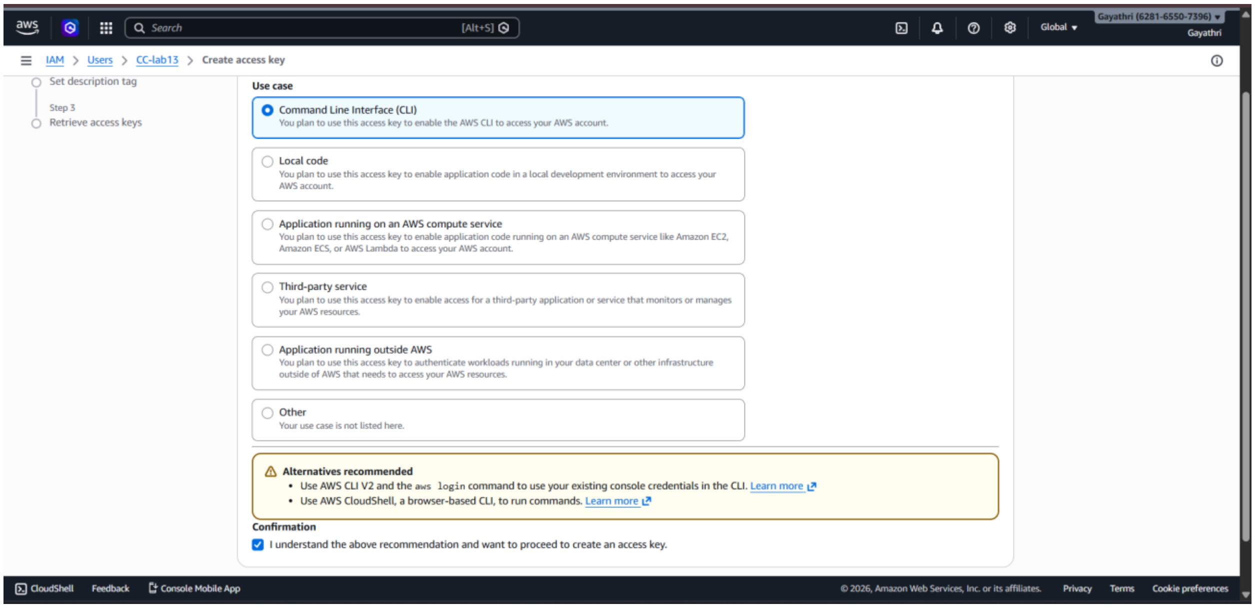
Login to the root user -> click on users -> click on security credentials



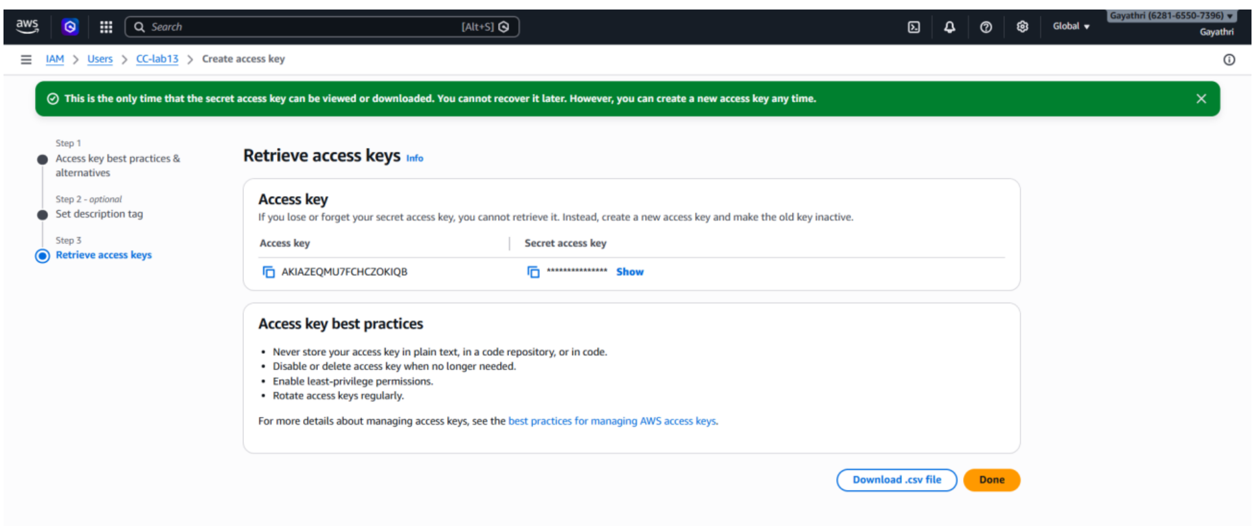
Navigate to Access keys -> click on create access key



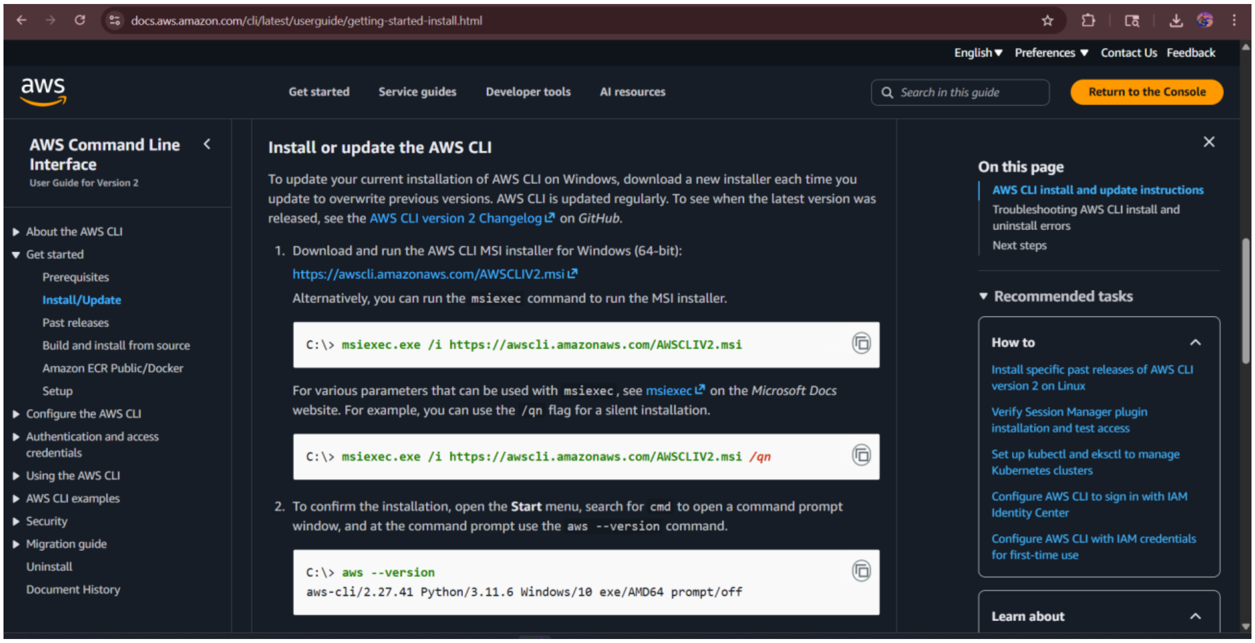
Select command line interface -> Acknowledge the warning and click **Next.**



Download the .csv file. > **Critical:** This contains the **Access Key ID** and **Secret Access Key**.



Install the **AWS CLI Tool** from google on your computer.

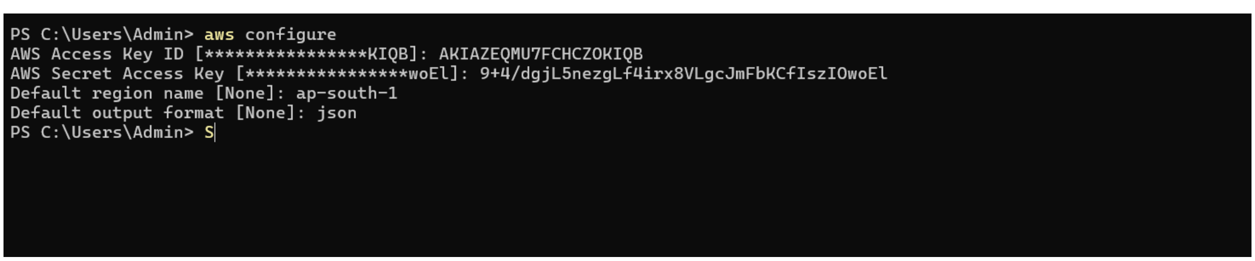


Open your Command Prompt (CMD) or Powershell.

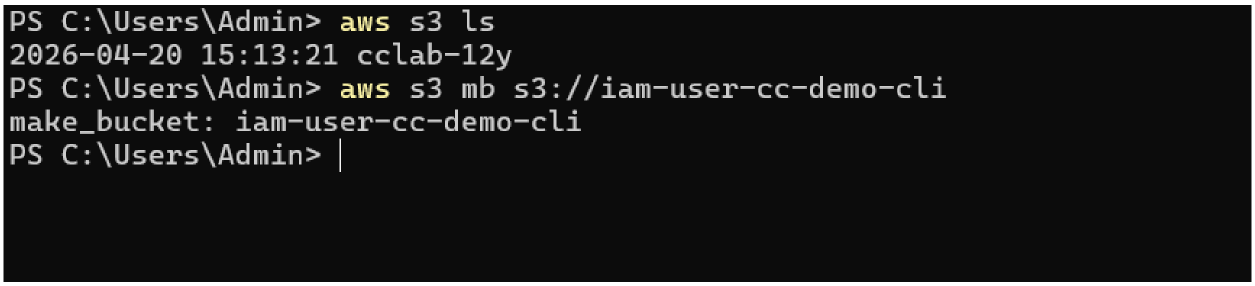
Type aws configure and press Enter.

Provide the following from your .csv:

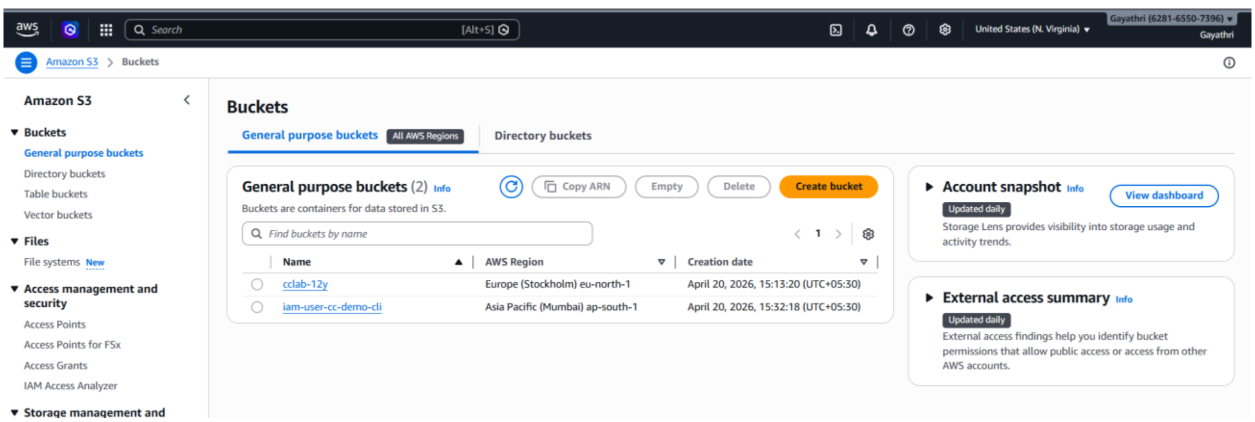
- **Access Key ID**
- **Secret Access Key**
- **Default region:** (e.g., ap-south-1)
- **Default output:** json



- Run aws s3 ls. This should list your buckets.
- Run aws s3 mb s3://your-unique-bucket-name. (Success)



Login to the root user the bucket will be created .



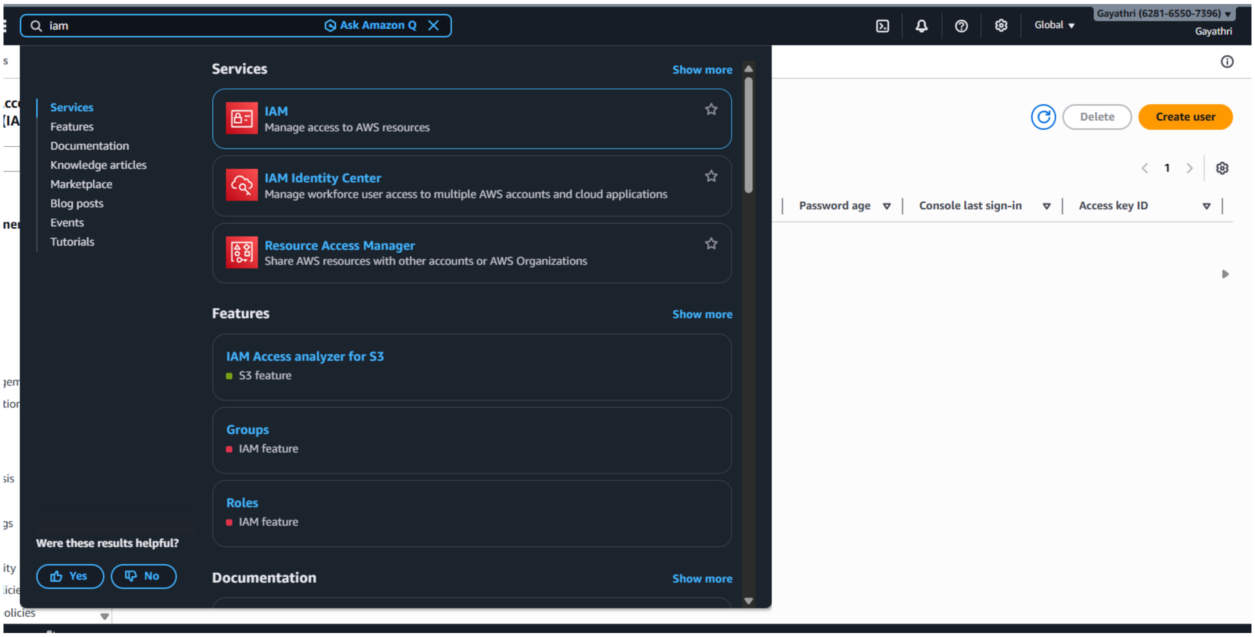
- Run aws iam list-users.
    - **Result:** You will get an **(AccessDenied)** error.

This confirms that the CLI is respecting the same IAM policy as the GUI.

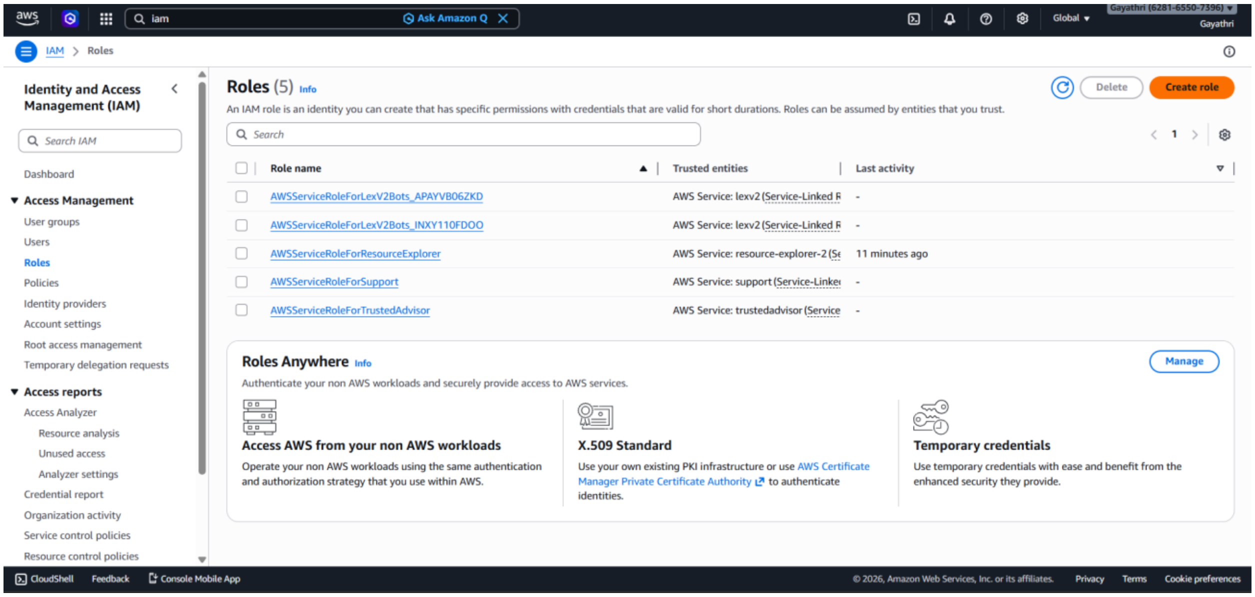
```
PS C:\Users\Admin> aws iam list-users

aws: [ERROR]: An error occurred (AccessDenied) when calling the ListUsers operation: User: arn:aws:iam::628165507396:use
r/CC-lab13 is not authorized to perform: iam:ListUsers on resource: arn:aws:iam::628165507396:user/ because no identity-
based policy allows the iam:ListUsers action
PS C:\Users\Admin>
```

WEEk-13 : Attach an IAM Role to an EC2 Instance (S3 access without keys)

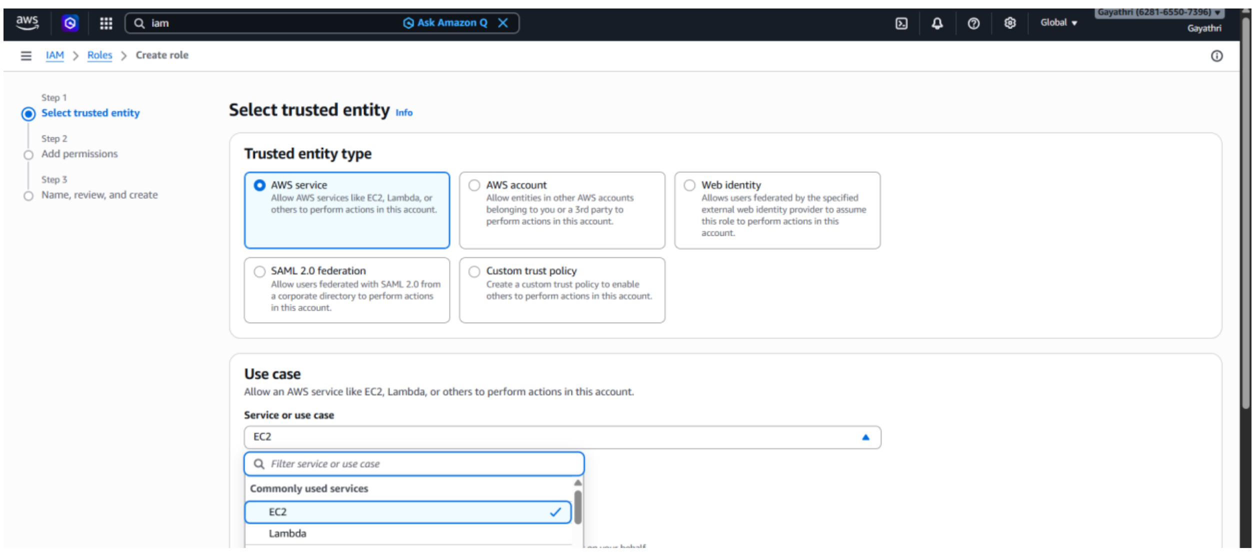
Login in to the aws account -> search for iam service in console



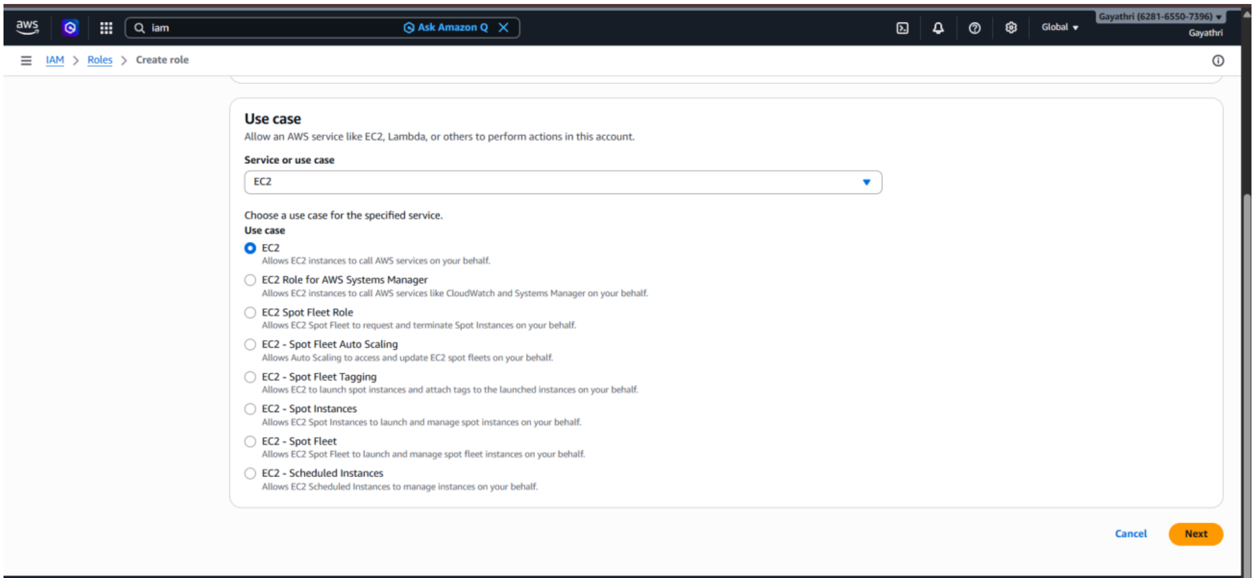
Open IAM Console → Roles → Create role



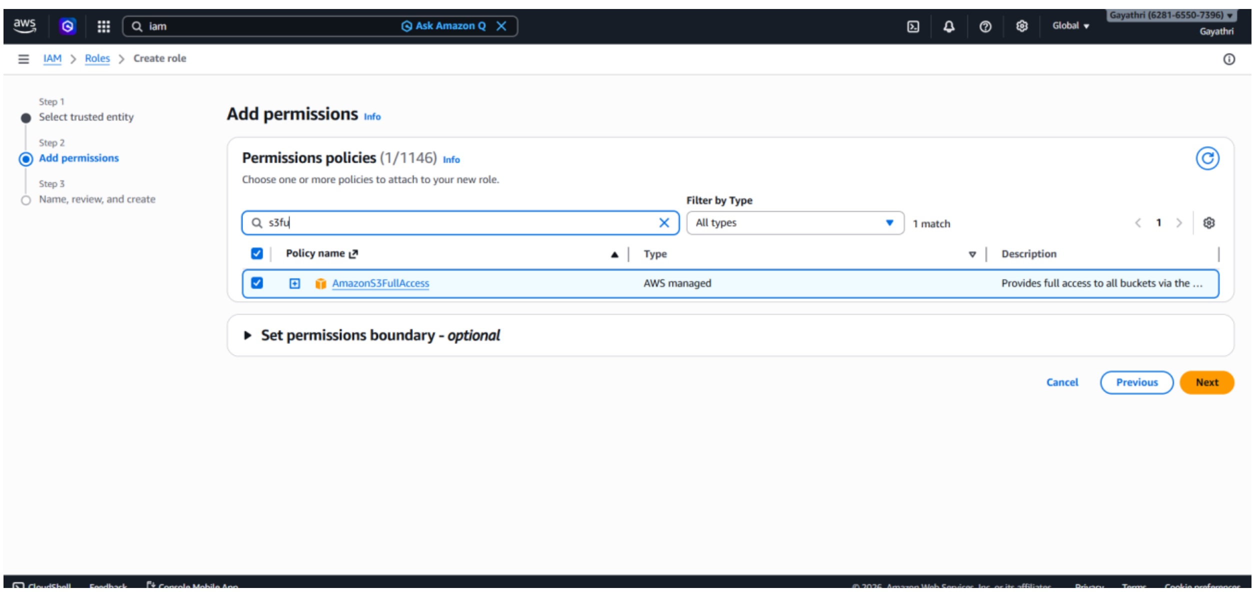
- Select AWS Service
- Use case: Choose EC2



Click Next

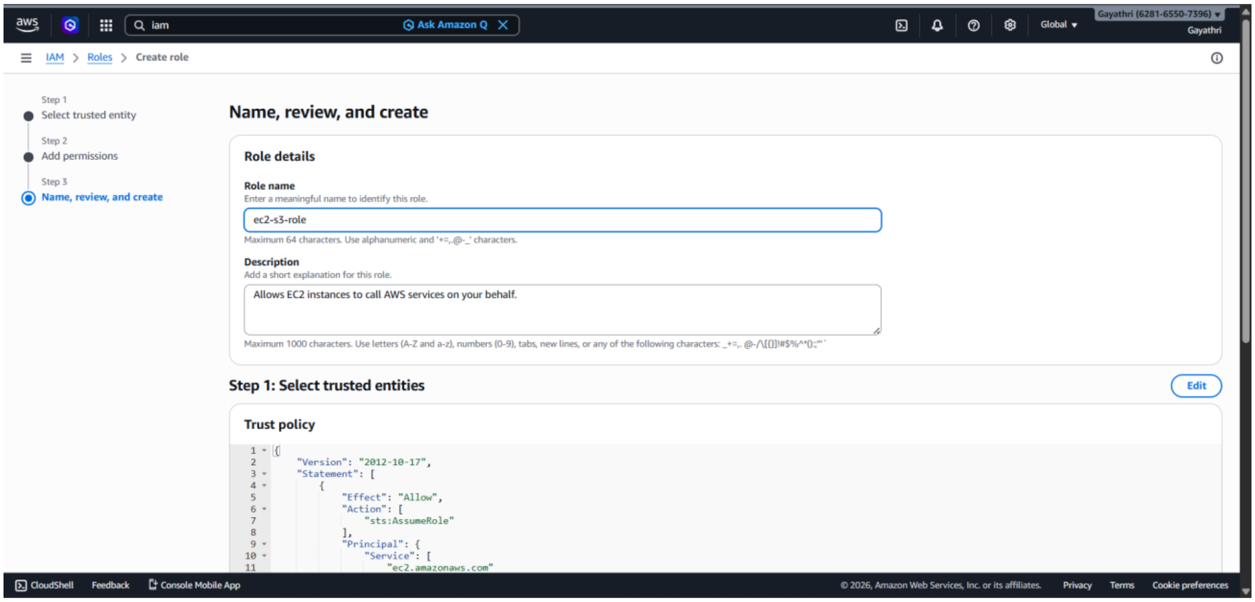


- Search policy: AmazonS3FullAccess
- Select it → Next



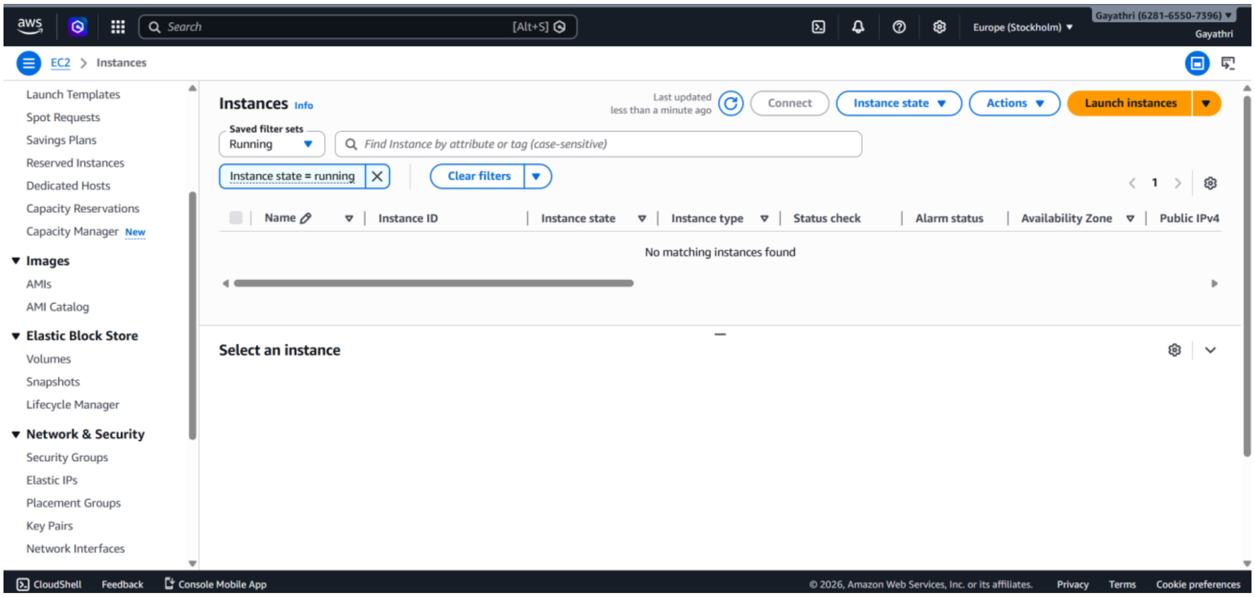
- Name: EC2-S3-Role

- Click Create role

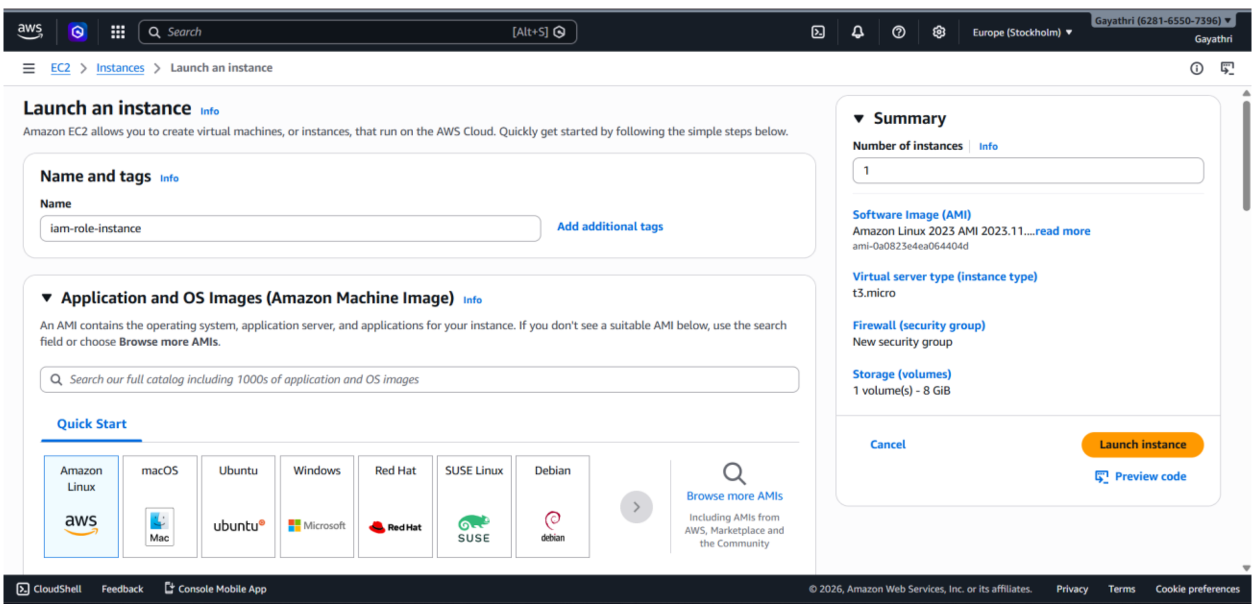


Go to EC2 Dashboard

Click on launch instance

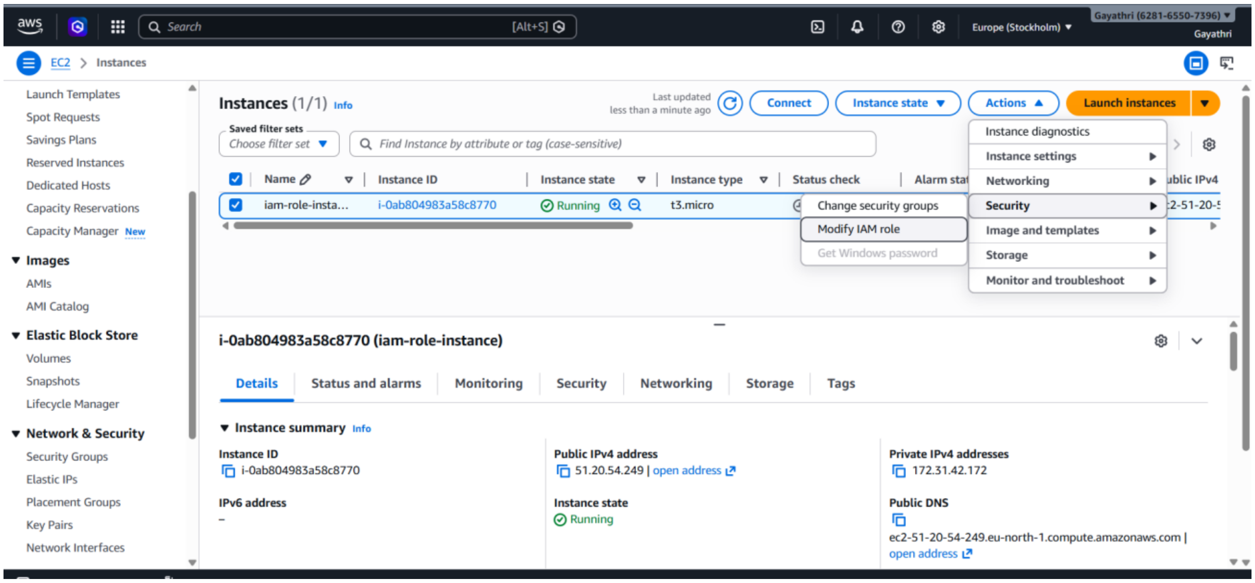


Name the instance and click on launch instance

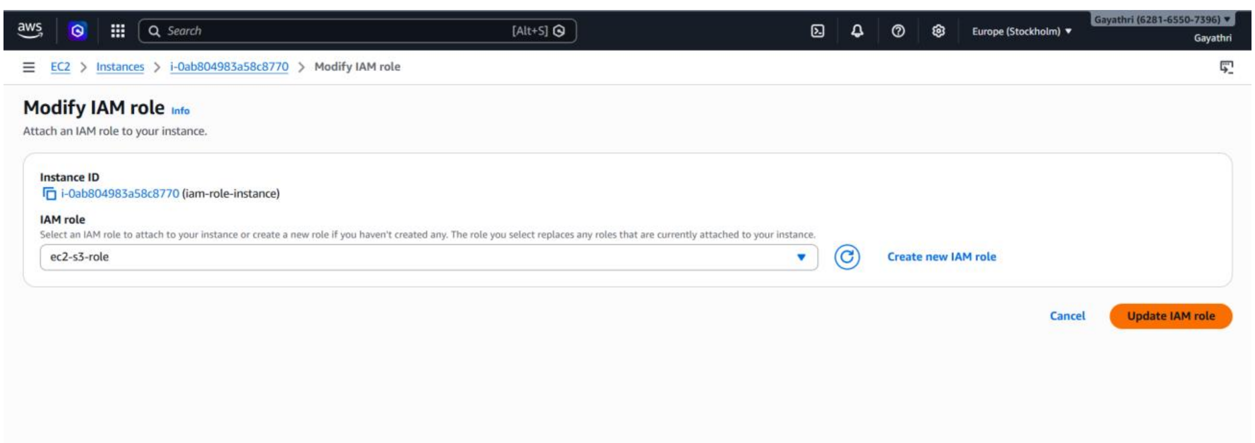


Select the running instance

Click on Actions → Security → Modify IAM role



- Select EC2-S3-Role → Update



Run the instance using ssh key

```
PS C:\Users\Admin\Downloads> ssh -i "demo.pem" ec2-user@ec2-51-20-54-249.eu-north-1.compute.amazonaws.com
The authenticity of host 'ec2-51-20-54-249.eu-north-1.compute.amazonaws.com (51.20.54.249)' can't be established.
ED25519 key fingerprint is SHA256:dkAleEIPxh45sQ0Z2xRydCld3vzl4V5y3AlhA2dDZdA.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-51-20-54-249.eu-north-1.compute.amazonaws.com' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-172-31-42-172 ~]$
```

aws s3 ls

aws s3 ls Works

```
[ec2-user@ip-172-31-42-172 ~]$ aws s3 ls
2026-04-21 09:42:23 week-13-demo
[ec2-user@ip-172-31-42-172 ~]$
```